RBC_XCPC_算法模板

C++17 • 黑白打印版

打印说明：A4，双面长边翻转，按实际大小（100%）打印。内侧预留装订边距。目录及依赖引用使用本册页码。代码中的 ↪ 为排版续行标记，不属于代码；续行时仍属于上一条源代码行。

阅读约定

- 普通数组、图和树顶点使用 1..n，数组查询默认闭区间 [l,r]；字符串、多项式系数等例外按各节说明。
- 起手式保留 `i64`、`i128`、`u64`；`// #define int long long` 默认注释。使用前检查数值范围、前置条件和复杂度。
- 同一问题的不同方法放在同一节的不同小标题下，每个方法各有独立 `.cpp`。背包、博弈小标题注明不同问题变体。
- 每份程序包含完整起手式和必要依赖，在 `solve()` 中写题后单独编译；手册片段省略起手式及已列出的依赖。
- 几何片段共用点、叉积等基础定义；完整文件已包含这些定义。文件名为对应手册编号和算法名，目录保留章节大标题。

目录

1. 基础与技巧 4
1.1 比赛起手式与整数安全 4
1.2 二分边界与答案二分 4
1.3 快速排序 5
1.4 离散化 5
1.5 前缀和 6
1.5.1 一维前缀和 6
1.5.2 二维前缀和 6
1.6 差分 7
1.6.1 一维差分 7
1.6.2 二维差分 7
1.7 单调队列：滑动窗口最小值 8
1.8 随机数与防构造哈希 9
2. 数据结构 10
2.1 并查集 10
2.2 可回滚并查集 10
2.3 树状数组与第 k 小 11
2.4 ST 表：静态区间最小值 12
2.5 线段树 13
2.5.1 无懒标记线段树（SegTree） 13
2.5.2 带懒标记线段树（segmenttree） 15
2.6 主席树：静态区间第 k 小 18
2.7 异或线性基 19
2.8 莫队：区间不同数个数 20
2.9 隐式 FHQ Treap：序列翻转 21
2.10 01 Trie：二进制字典树 22
3. 图论与树 25
3.1 单源最短路 25
3.1.1 Dijkstra：非负权最短路 25
3.1.2 0-1 BFS 25
3.1.3 Bellman-Ford：负边与可达负环 26
3.2 拓扑排序与 DAG 动态规划 26
3.3 Kruskal：最小生成森林 27
3.4 强连通分量 27
3.4.1 迭代 Kosaraju 27
3.4.2 递归 Tarjan 29
3.5 2-SAT：布尔约束可满足性 30
3.6 无向图割点与桥 31
3.7 树链剖分、LCA 与子树区间 32
3.8 树上前缀和 33
3.8.1 点权前缀和（TreePointPrefix） 33
3.8.2 边权前缀和（TreeEdgePrefix） 34
3.9 树上差分 35
3.9.1 点差分（TreePointDiff） 35
3.9.2 边差分（TreeEdgeDiff） 36
3.10 虚树：关键点及其 LCA 36
3.11 二分图最大匹配：匈牙利算法（DFS 增广路） 37
3.12 Dinic 最大流与最小割 38
3.13 最小费用流：逐次最短增广路 39
4. 字符串 41
4.1 前缀函数与 KMP 匹配 41
4.2 Z 函数 41
4.3 Manacher：奇偶回文半径 42
4.4 双模字符串哈希 43
4.5 普通字典树 43
4.6 AC 自动机：多模式出现次数 44
4.7 后缀数组与 LCP：倍增版 46
4.8 后缀自动机：不同子串与出现次数 46
4.9 最小表示法 48
5. 数学 49
5.1 快速幂 49
5.2 逆元 49
5.2.1 费马小定理求逆元 49
5.2.2 扩展欧几里得求逆元 49
5.3 线性筛：最小质因子、欧拉函数与莫比乌斯函数 50
5.4 组合数 51
5.4.1 阶乘预处理求组合数 51
5.4.2 Lucas 定理 51
5.5 扩展中国剩余定理 52
5.6 Miller–Rabin：64 位素性测试 52
5.7 Pollard–Rho：质因数分解 53
5.8 数论分块 54

5.9 NTT：模 998244353 多项式卷积 54
5.10 FFT：实系数卷积 55
5.11 高斯消元：实数线性方程组 56
5.12 矩阵快速幂 57
5.13 线性递推第 n 项：多项式约简 58
5.14 博弈 59
5.14.1 Nim 59
5.14.2 反 Nim 59
5.14.3 减法游戏 SG 59
5.15 高精度整数：分块加减乘除 60
6. 动态规划与优化 64
6.1 背包 64
6.1.1 0/1 背包 64
6.1.2 完全背包 64
6.1.3 多重背包：二进制拆分 64
6.2 最长严格上升子序列及方案恢复 65
6.3 最长公共子序列：滚动数组 65
6.4 数位 DP：相邻数字不同 66
6.5 SOS DP：子集和与超集和 66
6.6 树形 DP：最大权独立集 67
6.7 李超线段树：直线最小值与 DP 优化 68
6.8 分治优化 DP 69
7. 计算几何 70
7.1 二维叉积 70
7.2 线段相交：整数精确判断 70
7.3 凸包：单调链 71
7.4 多边形面积 71
7.5 点与多边形的位置关系 72
7.6 旋转卡壳：凸包直径 72
7.7 直线交点 73
7.8 圆与圆交点 73
7.9 有界半平面交：逐边裁剪 74
8. 比赛速查与检查清单 76
8.1 算法选型 76
8.2 复杂度与内存估算 76
8.3 128 位整数输出 76
8.4 常用结论与适用条件 77
8.5 交题前检查 77
8.6 对拍方法 77
8.7 本版范围 77

1. 基础与技巧

1.1 比赛起手式与整数安全

用途与条件：所有代码使用 GNU C++17。复制模板时加入本节头文件；同时保留 i64、i128、u64 的类型别名定义。普通数组和图、树顶点使用 1..n，输入数组分配 n+1 个位置且忽略 a[0]；数组查询采用闭区间 [l,r]。字符串仍从 0 开始。

复杂度：输入输出 O(输入规模)；__int128 是 GNU 扩展，非 ISO C++17 标准类型。

易错点：// #define int long long 仅作为注释保留，默认不启用。下标用 int，大整数权值和答案写 i64；特殊运算使用 i128 或 u64。signed main() 保持合法入口。无穷大常量仅在需要的模板内定义；本版不保证取消宏注释后所有模板仍直接兼容。

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
using i128 = __int128_t;
using u64 = unsigned long long;
#define endl '\n'
// #define int long long

void solve() {
    // Read one test case. Arrays and vertices use 1..n.
}
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    cin >> T; // Remove for a single test case.
    while (T--) solve();
}
```

调用与说明：起手式统一定义 #define endl '\n'，cout << ans << endl; 仅换行而不强制刷新；不要写 std::endl。交互题使用 cout << ans << endl << flush; 显式刷新。GNU 编译示例：g++ -std=c++17 -O2 main.cpp -o main。调试时可增加 -Wall -Wextra -D_GLIBCXX_DEBUG。

1.2 二分边界与答案二分

用途与条件：使用开区间 (l,r) 二分，在先假后真的条件中找第一个 true。l 是不满足条件的边界，r 是满足条件的边界，两端可使用虚拟哨兵且不会调用 ok。对 1 下标数组 arr[1..n]，初始化 l=0、r=n+1；不存在则返回 n+1。

复杂度：O(log(r-l)) 次判定；空间 O(1)。

易错点：要求 l<r、r-l 不溢出，且区间内条件先假后真。初始 l、r 不参与检查；实际候选闭区间 [L,R] 应传入 L-1、R+1，并确保哨兵可表示。循环结束时 r 是第一个真，l 是最后一个假；返回初始 r 表示候选中不存在真。

```
// 在开区间 (l,r) 中寻找第一个使 ok(x) 为真的值，不存在则返回初始 r。
// l 为假边界，r 为真边界；两端可为虚拟哨兵，不会调用 ok。
// 要求 l<r、r-l 不溢出，条件先假后真；判定次数为 O(log(r-l))。
// 对 arr[1..n]，传入 l=0、r=n+1；返回 n+1 表示不存在。
template <class Pred>
i64 first_true(i64 l, i64 r, Pred ok) {
    while (l + 1 < r) {
        i64 mid = l + (r - l) / 2;
        if (ok(mid)) {
            r = mid;
        }
        else {
            l = mid;
        }
    }
    return r;
}
```

调用与说明：有序数组 arr[1..n]：first_true(0, n + 1, [&](i64 mid) { return arr[mid] >= x; }) 返回第一个 >=x 的位置，不存在返回 n+1。答案二分：first_true(-1, 11, [](i64 x) { return x*x >= 30; }) 在候选 [0,10] 中返回 6。

1.3 快速排序

用途与条件：将整数数组 a 的闭区间 [l,r] 原地按升序排序；数组采用 1 下标，a[0] 占位。排序整个数组调用 quick_sort(a,1,n)。

复杂度：平均时间 O(n log n)，最坏 O(n²)；递归栈平均 O(log n)，最坏 O(n)。

易错点：非稳定排序。固定选择中间位置的元素作为基准，特殊构造可能退化；保存基准值，不能引用交换过程中会改变的数组元素。递归区间为 [l,j] 和 [j+1,r]。l>=r 时直接返回，允许用 [1,0] 表示空数组。其他算法的排序继续使用标准库 sort。

```
// 将 a[l..r] 原地按升序排序，无返回值；调用时保证区间下标合法。
// 数组采用 1 下标，a[0] 占位；l>=r 时不操作。
// 平均时间 O(n log n)，最坏 O(n²)；递归栈平均 O(log n)，最坏 O(n)。
void quick_sort(vector<int>& a, int l, int r) {
    if (l >= r) return;

    int i = l - 1, j = r + 1;
    int pivot = a[l + (r - l) / 2];  // 保存基准值，避免交换后基准改变。

    while (i < j) {
        do {
            i++;
        } while (a[i] < pivot);

        do {
            j--;
        } while (a[j] > pivot);

        if (i < j) {
            swap(a[i], a[j]);
        }
    }

    // 分区后，左区间元素不大于基准，右区间元素不小于基准。
    quick_sort(a, l, j);
    quick_sort(a, j + 1, r);
}
```

调用与说明：vector<int> a = {0,3,1,2,1}; quick_sort(a,1,4); 得到 {0,1,1,2,3}。也可只排序合法子区间，区间外元素保持不变。

1.4 离散化

用途与条件：输入数组 a[1..n]，排序去重得到 xs[1..m]，将原数值映射为从 1 开始的秩；不修改原数组。

复杂度：构造 O(n log n)，单次查询秩 O(log m)，空间 O(n)。

易错点：离散化保留大小关系，不保留距离；几何长度使用原坐标之差。输入数组至少包含占位 a[0]，其值不参与计算。

```
// a[0] is unused; xs[0] is also unused.
// 将 a[1..n] 排序去重，返回带 0 号占位的有序值表 xs；不修改原数组。
// 要求 a 至少包含占位项；时间 O(n log n)，空间 O(n)。
template <class T>
vector<T> compress(const vector<T>& a) {
    vector<T> xs(a.begin() + 1, a.end());
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());
    xs.insert(xs.begin(), T{});
    return xs;
}
// 返回 x 在有序离散值表 xs[1..m] 中的 lower_bound 位置，作为 1 下标的秩。
// 通常要求 x 已在值表中；否则返回插入位置，可能为 m+1。时间 O(log m)。
```

```
template <class T>
int rank_of(const vector<T>& xs, T x) {
    return lower_bound(xs.begin() + 1, xs.end(), x) - xs.begin();
}
```

调用与说明：compress(vector<int>{0,30,10,30}) 得到 {0,10,30}，rank_of(xs,30)==2。

1.5 前缀和

1.5.1 一维前缀和

用途与条件：输入数组 a[1..n]，构造 pre[0]=0 的前缀和数组；闭区间 [l,r] 的和为 pre[r]-pre[l-1]，适合静态区间求和。

复杂度：构造时间和空间 O(n)，单次区间求和 O(1)。

易错点：输入数组至少包含占位 a[0]，其值不参与计算。要求 1≤l≤r≤n；原数组修改后前缀和不会自动更新，累加及区间和必须在 i64 范围内。

```
// 由 a[1..n] 构造前缀和 pre，pre[0]=0；[l,r] 的和为 pre[r]-pre[l-1]。
// 忽略 a[0]，不修改 a；要求累加不溢出 i64，时间和空间均为 O(n)。
vector<i64> prefix_sum(const vector<i64>& a) {
    vector<i64> pre(a.size()); // pre[i]：a[1..i] 的和，pre[0]=0。
    for (int i = 1; i < (int)a.size(); ++i) {
        pre[i] = pre[i - 1] + a[i];
    }
    return pre; // sum[l,r] = pre[r]-pre[l-1]
}
```

调用与说明：auto pre = prefix_sum({0,1,2,3}); 得到 {0,1,3,6}；区间 [2,3] 的和为 pre[3]-pre[1]=5。

1.5.2 二维前缀和

用途与条件：输入矩形数组 a[1..n][1..m]，构造二维前缀和，通过 query 查询闭矩形 [x1,x2] × [y1,y2] 的元素和；x 表示行，y 表示列。

复杂度：构造时间和空间 O(nm)，单次矩形求和 O(1)。

易错点：要求 n,m≥1，输入包含第 0 行和第 0 列占位，其值不参与计算。查询矩形非空且下标有效；修改原数组后需要重新构造 pre。所有中间结果必须在 i64 范围内。

```
// 由矩形数组 a[1..n][1..m] 构造二维前缀和，要求 n,m>=1。
// 忽略第 0 行和第 0 列，不修改 a；返回 pre，时间和空间均为 O(nm)。
vector<vector<i64>> prefix_sum(const vector<vector<i64>>& a) {
    assert(a.size() >= 2 && a[0].size() >= 2);
    int n = (int)a.size() - 1;
    int m = (int)a[0].size() - 1;
    // pre[i][j]：矩形 [1,i] × [1,j] 的元素和，第 0 行和第 0 列为 0。
    vector<vector<i64>> pre(n + 1, vector<i64>(m + 1));
    for (int i = 1; i <= n; ++i) {
        assert((int)a[i].size() == m + 1);
        for (int j = 1; j <= m; ++j) {
            pre[i][j] = pre[i - 1][j] + pre[i][j - 1]
                      - pre[i - 1][j - 1] + a[i][j];
        }
    }
    return pre;
}

// 返回闭矩形 [x1,x2] × [y1,y2] 的元素和，x 为行，y 为列。
// pre 由 prefix_sum 构造；要求矩形非空且下标有效，时间 O(1)。
i64 query(const vector<vector<i64>>& pre,
          int x1, int y1, int x2, int y2) {
    assert(1 <= x1 && x1 <= x2 && x2 < (int)pre.size());
    assert(1 <= y1 && y1 <= y2 && y2 < (int)pre[0].size());
    return pre[x2][y2] - pre[x1 - 1][y2]
         - pre[x2][y1 - 1] + pre[x1 - 1][y1 - 1];
}
```

调用与说明：vector<vector<i64>> a = {{0,0,0,0},{0,1,2,3},{0,4,5,6}}; auto pre = prefix_sum(a); query(pre,1,2,2,3) 返回 16。

1.6 差分

1.6.1 一维差分

用途与条件：输入数组 a[1..n]，构造差分数组 diff；通过 range_add 记录闭区间 [l,r] 加 v，所有操作结束后调用 restore 还原。适合批量区间加、最后统一取值。

复杂度：初始化与还原各 O(n)，单次区间加 O(1)，空间 O(n)。

易错点：diff 必须保留 n+2 项，容纳 r+1；最后一项为边界标记，不属于原数组。a[0] 的值不参与计算。差分及累加结果必须在 i64 范围内；还原不会修改 diff。

```
// 由 a[1..n] 构造 n+2 项的差分数组，忽略占位 a[0]；不修改 a。
// 输入至少含 a[0]，允许 n=0；末项用于区间更新的边界标记，时间和空间 O(n)。
vector<i64> difference(const vector<i64>& a) {
    assert(!a.empty());
    int n = (int)a.size() - 1;
    vector<i64> diff(n + 2);
    for (int i = 1; i <= n; ++i) {
        diff[i] = a[i] - (i == 1 ? 0 : a[i - 1]);
    }
    return diff;
}
// 在差分数组 diff 上记录闭区间 [l,r] 加 v，不直接还原原数组；l>r 时不操作。
// 有效更新要求 l>=1 且 r+1 在数组内，通常分配 n+2 项；时间 O(1)。
void range_add(vector<i64>& diff, int l, int r, i64 v) {
    if (l > r) return;
    diff[l] += v;
    diff[r + 1] -= v;
}
// 将含 n+2 项的 diff 前缀累加，返回 a[1..n]，a[0]=0；忽略末项边界标记。
// 不修改 diff，可在后续继续更新并再次还原；时间和空间 O(n)。
vector<i64> restore(const vector<i64>& diff) {
    assert(diff.size() >= 2);
    int n = (int)diff.size() - 2;
    vector<i64> a(n + 1);
    for (int i = 1; i <= n; ++i) {
        a[i] = a[i - 1] + diff[i];
    }
    return a;
}
```

调用与说明：auto diff = difference({0,1,2,3}); range_add(diff,2,3,5); auto a = restore(diff); 得到 {0,1,7,8}。初始数组全零时，可直接使用 vector<i64> diff(n+2)。

1.6.2 二维差分

用途与条件：输入矩形数组 a[1..n][1..m]，通过 range_add 记录闭矩形 [x1,x2] × [y1,y2] 加 v，所有操作结束后调用 restore 还原；x 表示行，y 表示列。

复杂度：构造和还原各 O(nm)，单次矩形加 O(1)，空间 O(nm)。

易错点：要求 n,m≥1，diff 分配 (n+2) × (m+2)，额外一行、一列容纳更新边界。输入第 0 行和第 0 列的值不参与计算。还原不修改 diff，可继续更新；所有中间结果必须在 i64 范围内。

```
// 由矩形数组 a[1..n][1..m] 构造二维差分，要求 n,m>=1。
// 忽略第 0 行和第 0 列，不修改 a；返回 diff，时间和空间均为 O(nm)。
vector<vector<i64>> difference(const vector<vector<i64>>& a) {
    assert(a.size() >= 2 && a[0].size() >= 2);
    int n = (int)a.size() - 1;
    int m = (int)a[0].size() - 1;
    // diff：二维差分数组；第 n+1 行、第 m+1 列用于更新边界标记。
    vector<vector<i64>> diff(n + 2, vector<i64>(m + 2));
    for (int i = 1; i <= n; ++i) {
        assert((int)a[i].size() == m + 1);
        for (int j = 1; j <= m; ++j) {
```

```
            diff[i][j] = a[i][j];
            if (i > 1) diff[i][j] -= a[i - 1][j];
            if (j > 1) diff[i][j] -= a[i][j - 1];
            if (i > 1 && j > 1) {
                diff[i][j] += a[i - 1][j - 1];
            }
        }
    }
    return diff;
}

// 在 diff 上记录闭矩形 [x1,x2] × [y1,y2] 的元素加 v，无返回值。
// diff 由 difference 构造；要求矩形非空且下标有效，时间 O(1)。
void range_add(vector<vector<i64>>& diff,
               int x1, int y1, int x2, int y2, i64 v) {
    assert(1 <= x1 && x1 <= x2 && x2 < (int)diff.size() - 1);
    assert(1 <= y1 && y1 <= y2 && y2 < (int)diff[0].size() - 1);
    diff[x1][y1] += v;
    diff[x2 + 1][y1] -= v;
    diff[x1][y2 + 1] -= v;
    diff[x2 + 1][y2 + 1] += v;
}

// 将 difference 构造的 diff 还原为 a[1..n][1..m]，第 0 行、第 0 列为 0。
// 不修改 diff，允许继续更新后再次还原；返回 a，时间和空间均为 O(nm)。
vector<vector<i64>> restore(const vector<vector<i64>>& diff) {
    assert(diff.size() >= 3 && diff[0].size() >= 3);
    int n = (int)diff.size() - 2;
    int m = (int)diff[0].size() - 2;
    vector<vector<i64>> a(n + 1, vector<i64>(m + 1));
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            a[i][j] = a[i - 1][j] + a[i][j - 1]
                    - a[i - 1][j - 1] + diff[i][j];
        }
    }
    return a;
}
```

调用与说明：vector<vector<i64>> a = {{0,0,0,0},{0,1,2,3},{0,4,5,6}}; auto diff = difference(a); range_add(diff,1,2,2,3,10); a = restore(diff); 两行分别为 {1,12,13}、{4,15,16}。初始矩阵全零时可直接分配 vector<vector<i64>> diff(n+2,vector<i64>(m+2))。

1.7 单调队列：滑动窗口最小值

用途与条件：输入 a[1..n]，返回 ans[1..n-k+1]，ans[j] 是窗口 [j,j+k-1] 的最小值；a[0]、ans[0] 均为占位。要求 1≤k≤n。

复杂度：时间 O(n)，辅助空间 O(k)。

易错点：先删除已离开窗口的下标，再维护单调性；存下标才能判定过期。将 >= 改为 <= 可求最大值。

```
// 返回 a[1..n] 每个长度为 k 的窗口最小值，ans[j] 对应 [j,j+k-1]。
// 输入和输出的 0 号项均占位，要求 1<=k<=n；时间 O(n)，队列空间 O(k)。
vector<i64> window_min(const vector<i64>& a, int k) {
    assert(1 <= k && k < (int)a.size());
    deque<int> q;
    vector<i64> ans(1);
    for (int i = 1; i < (int)a.size(); ++i) {
        while (!q.empty() && q.front() <= i - k) {
            q.pop_front();
        }
        while (!q.empty() && a[q.back()] >= a[i]) {
            q.pop_back();
        }
```

```
        q.push_back(i);
        if (i >= k) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

调用与说明：window_min({0,4,2,3,1},2) 返回 {0,2,2,1}，第 0 项为占位。

1.8 随机数与防构造哈希

用途与条件：为 unordered_map 的整数键提供带随机种子的哈希，降低固定碰撞构造的影响。
复杂度：单次哈希 O(1)；哈希表平均操作 O(1)，最坏复杂度仍无保证。
易错点：随机化不是确定性最坏复杂度保证。需要有序遍历或严格界限时用 map。reserve 能减少扩容。

```
struct CustomHash {
    // 将 64 位整数 x 按 SplitMix64 步骤混合，返回确定性的 64 位散列值；时间 O(1)。
    static u64 mix(u64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    // 为整数键 x 计算带进程内固定随机种子的哈希值，供 unordered_map 等容器调用。
    // 同一进程内相同键得到相同结果；可降低固定碰撞构造的影响，但不保证无碰撞。
    size_t operator()(u64 x) const {
        static const u64 seed = chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }
};
```

调用与说明：unordered_map<i64,int,CustomHash> mp; mp.reserve(200000); mp.max_load_factor(0.7); 使用固定种子 mt19937_64 rng(12345) 可复现对拍。

2. 数据结构

2.1 并查集

用途与条件：维护无向连通分量，递归路径压缩配合按秩合并。使用全局 father、rk、sz 数组和 init、find、merge、size 函数，size(id) 查询所在集合的元素数量。元素编号为 1..n，0 号位置不使用。

复杂度：均摊 O(α(n)) 每次操作；空间 O(n)。

易错点：每组数据先调用 init(n)。rk 表示秩，路径压缩后不等于实际树高，也不是集合大小。只有根节点的 sz 才表示当前集合大小，应通过 size(id) 查询。merge 成功时可在调用处将连通块数量减一。普通并查集不能删除边；回滚版本不要使用路径压缩。

```
// DSU（Disjoint Set Union）：并查集。
// father[x]：父节点；根节点的父节点是自身。
// rk（rank）：合并使用的秩，不是集合大小；sz[root]：根对应的集合大小。
vector<int> father, rk, sz;

// 初始化编号 1..n 的独立集合，初始秩和集合大小均为 1；时间和空间 O(n)。
void init(int n) {
    father.resize(n + 1);
    rk.assign(n + 1, 1);
    sz.assign(n + 1, 1);
    for (int i = 1; i <= n; ++i) father[i] = i;
}

// 返回 id 所在集合的根，并通过递归压缩路径；要求 1<=id<=n。
int find(int id) {
    if (id != father[id]) father[id] = find(father[id]);
    return father[id];
}

// 按秩合并 u、v 所在集合并更新集合大小；成功返回 true，原本同集合返回 false。
// 要求顶点为 1..n；配合路径压缩，均摊时间 O(α(n))。
bool merge(int u, int v) {
    int a = find(u), b = find(v);
    if (a == b) return false;
    if (rk[a] > rk[b]) {
        father[b] = a;
        sz[a] += sz[b];
    }
    else if (rk[b] > rk[a]) {
        father[a] = b;
        sz[b] += sz[a];
    }
    else {
        father[b] = a;
        rk[a]++;
        sz[a] += sz[b];
    }
    return true;
}

// 返回 id 所在集合的元素数量；要求 1<=id<=n，均摊时间 O(α(n))。
int size(int id) {
    return sz[find(id)];
}
```

调用与说明：init(n); int num=n; 若 merge(u,v) 返回 true，则 --num；find(u)==find(v) 判断是否连通；size(u) 查询 u 所在集合的元素数量。

2.2 可回滚并查集

用途与条件：离线动态连通性、分治时间轴等场景。保存快照，在处理完一个分支后恢复。元素编号为 1..n，0 号位置不使用。

复杂度：find / merge O(log n)；回滚每次成功合并 O(1)；空间 O(n+操作数)。
易错点：禁止路径压缩。失败的 merge 不入栈，所以应记录 snapshot 而非假定每次操作都对应一条历史。

```
// DSU（Disjoint Set Union）：并查集；Rollback 表示支持回滚。
struct RollbackDSU {
    // father[x]：父节点；sz[root]：根对应的集合大小。
    vector<int> father, sz;
    // hist（history）：成功合并的历史，保存被挂接的根及合并前另一集合的大小。
    vector<pair<int, int>> hist;
    // 创建编号 1..n 的独立集合并清空合并历史，时间和空间 O(n)。
    RollbackDSU(int n) : father(n + 1), sz(n + 1, 1) {
        iota(father.begin(), father.end(), 0);
    }
    // 沿父指针返回 x 的代表元，不压缩路径以便回滚；时间 O(log n)。
    int find(int x) {
        while (x != father[x]) x = father[x];
        return x;
    }
    // 返回当前成功合并历史的长度，作为之后 rollback 的快照；不修改状态，O(1)。
    int snapshot() const { return hist.size(); }
    // 按大小合并 a、b 所在集合，并记录可撤销信息；同集合返回 false 且不记历史。
    // 有效合并返回 true；时间 O(log n)。
    bool merge(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        hist.push_back({b, sz[a]});
        father[b] = a;
        sz[a] += sz[b];
        return true;
    }
    // 撤销当前历史末尾的合并，恢复到快照 s；s 必须是当前历史中的有效长度。
    // 每撤销一次成功合并耗时 O(1)，不会恢复已被丢弃的未来快照。
    void rollback(int s) {
        while ((int)hist.size() > s) {
            auto [b, old] = hist.back();
            hist.pop_back();
            sz[father[b]] = old;
            father[b] = b;
        }
    }
};
```

调用与说明：int s=d.snapshot(); d.merge(u,v); ...; d.rollback(s);

2.3 树状数组与第 k 小

用途与条件：单点加、闭区间 [1,r] 求和；下标 1..n，sum(r) 为 [1,r] 之和，sum(0)=0。非负频数时 kth(k) 返回第 k 小的 1 下标。
复杂度：更新、查询、第 k 小均 O(log n)；空间 O(n)。
易错点：kth 要求 1≤k≤总频数，且所有单点值非负。区间和可存负数，但此时不能使用 kth。

```
struct Fenwick {
    int n;
    // t[i]：树状数组节点，维护原数组区间 [i-lowbit(i)+1,i] 的和。
    // lowbit(i) = i & -i，表示 i 的最低有效二进制位对应的值。
    vector<i64> t;
    // 初始化长度为 n、下标为 1..n 的全零树状数组，时间和空间 O(n)。
    Fenwick(int n) : n(n), t(n + 1) {}
    // 将第 x 项增加 v，并更新树状数组；要求 1<=x<=n，时间 O(log n)。
    void add(int x, i64 v) {
        assert(1 <= x && x <= n);
```

```
        for (; x <= n; x += x & -x) {
            t[x] += v;
        }
    }
    // 返回闭区间 [1,r] 的元素和；要求 0<=r<=n，r=0 返回 0，时间 O(log n)。
    i64 sum(int r) const {
        i64 s = 0;
        for (; r; r -= r & -r) {
            s += t[r];
        }
        return s;
    }
    // 返回闭区间 [l,r] 的元素和；非空查询要求 1<=l<=r<=n，l>r 返回 0。
    // 不修改数组，时间 O(log n)。
    i64 query(int l, int r) const { return l > r ? 0 : sum(r) - sum(l - 1); }
    // 返回最小的下标 x，使前缀和 sum(x)>=k；将单点值看作频数时即第 k 小的位置。
    // 要求所有单点频数非负且 1<=k<=sum(n)，返回 1 下标，时间 O(log n)。
    int kth(i64 k) const {
        assert(1 <= k && k <= sum(n));
        int x = 0, step = 1;
        while (step <= n / 2) step <<= 1;
        for (; step; step >>= 1) {
            int y = x + step;
            if (y <= n && t[y] < k) {
                x = y;
                k -= t[y];
            }
        }
        return x + 1;
    }
};
```

调用与说明：Fenwick f(4); f.add(3,3); f.query(2,4)==3，f.kth(2)==3。逆序对：依次累加已插入数量 - sum(rank)，然后 add(rank,1)。

2.4 ST 表：静态区间最小值

用途与条件：静态数组 a[1..n] 的非空闭区间 [l,r] 最小值。a[0] 占位；st[i][j] 保存从 i 开始、长度为 2^j 的区间最小值。

复杂度：预处理 O(n log n)，查询 O(1)，空间 O(n log n)。

易错点：输入至少包含 a[0] 占位和一个有效元素。构造时断言 a.size() >= 2，查询时断言 1 <= l <= r <= n；定义 NDEBUG 后断言禁用，调用者仍须保证这些条件。数组修改后需要重建。log2 的参数必须为正数。默认数据类型为 int。

```
// ST（Sparse Table）：稀疏表，用于静态区间查询。
// RMQ（Range Minimum Query）：区间最小值查询，本模板默认实现的操作。
class ST {
private:
    int n;
    // st[i][j]：从 i 开始、长度为 2^j 的区间最小值。
    vector<vector<int>> st;

public:
    // 由 a[1..n] 构造静态区间最小值表，忽略 a[0]，要求 n>=1。
    // 预处理时间和空间 O(n log n)；原数组修改后需重新构造。
    ST(const vector<int>& a) {
        assert(a.size() >= 2);
        n = (int)a.size() - 1;
        if (n <= 0) return;

        int K = log2((double)n) + 1;
        st.assign(n + 1, vector<int>(K));
```

```
        for (int i = 1; i <= n; ++i) st[i][0] = a[i];

        for (int j = 1; j < K; ++j) {
            for (int i = 1; i + (1LL << j) - 1 <= n; ++i) {
                st[i][j] = min(st[i][j - 1], st[i + (1LL << (j - 1))][j - 1]);
            }
        }
    }

    // 1 下标，查询非空闭区间 [l, r] 的最小值
    int query(int l, int r) const {
        assert(1 <= l && l <= r && r <= n);
        int j = log2((double)(r - l + 1));
        return min(st[l][j], st[r - (1LL << j) + 1][j]);
    }
};
```

调用与说明：vector<int> a = {0,3,1,4}; ST st(a); st.query(1,2) 返回 1。独立算法文件中直接使用 ST。

更换区间操作

需同时替换两处合并操作，其他建表和查询逻辑不变：

1. 构造函数中，`st[i][j] = min(...)` 的 `min`。
2. `query` 函数中，`return min(...)` 的 `min`。

目标操作	两处替换方式
最大值	将 min 改为 max
最大公约数（非负整数）	将 min 改为 std::gcd，使用 C++17
按位与	将两处 min(x, y) 改为 (x & y)，x、y 代表原来的两个参数
按位或	将两处 min(x, y) 改为 (x | y)，x、y 代表原来的两个参数

更换后同步修改查询注释。合并操作须满足结合律和幂等性（合并 x 与 x 仍为 x），因为查询用的两个区间可能重叠。不能直接替换为求和、乘积或异或。

数据超出 int 范围时，可在标准头文件和 `using namespace std;` 之后、此类及输入数组定义之前添加 `#define int long long`，主函数使用 `signed main()`。使用完整的 代码/02_数据结构/2.4 ST 表：静态区间最小值.cpp 写题时，在标准头文件之后取消整数宏的注释即可；此用法针对本 ST 模板。

2.5 线段树

2.5.1 无懒标记线段树（SegTree）

用途与条件：输入 a[1..n]，a[0] 占位，n≥1。支持单点赋值、单点加和闭区间求和；单点下标要求 1≤pos≤n，查询要求 1≤l≤r≤n。

复杂度：建树 O(n)，每次操作 O(log n)，空间 O(n)。

易错点：所有类型统一 int，需要时在标准头文件之后、类定义之前添加 #define int long long。构造函数和公开接口包含 assert 检查，定义 NDEBUG 后禁用。没有懒标记，add(pos,v) 只修改一个元素。

```
class SegTree {
private:
    int n;
    vector<int> tree; // 区间和

    // 合并两个区间的结果：参数 a、b 为各自区间和，返回合并后的和
    int merge(int a, int b) { return a + b; }

    // 向上合并：通过 merge 用两个子节点的结果更新节点 p
    void pushup(int p) {
        tree[p] = merge(tree[p * 2], tree[p * 2 + 1]);
    }

    // 建树：节点 p 管理闭区间 [l,r]
```

```
    void build(const vector<int>& a, int p, int l, int r) {
        if (l == r) {
            tree[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(a, p * 2, l, m);
        build(a, p * 2 + 1, m + 1, r);
        pushup(p);
    }

    // 单点赋值的递归实现：将 a[pos] 改为 v
    void set(int p, int l, int r, int pos, int v) {
        if (l == r) {
            tree[p] = v;
            return;
        }
        int m = (l + r) / 2;
        if (pos <= m) {
            set(p * 2, l, m, pos, v);
        }
        else {
            set(p * 2 + 1, m + 1, r, pos, v);
        }
        pushup(p);
    }

    // 单点加的递归实现：将 a[pos] 增加 v，再向上更新区间和
    void add(int p, int l, int r, int pos, int v) {
        if (l == r) {
            tree[p] += v;
            return;
        }
        int m = (l + r) / 2;
        if (pos <= m) {
            add(p * 2, l, m, pos, v);
        }
        else {
            add(p * 2 + 1, m + 1, r, pos, v);
        }
        pushup(p);
    }

    // 区间求和的递归实现：完全覆盖直接返回，否则累加子区间答案
    int query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tree[p];
        int m = (l + r) / 2;
        int sum = 0;
        if (ql <= m) {
            sum = merge(sum, query(p * 2, l, m, ql, qr));
        }
        if (qr > m) {
            sum = merge(sum, query(p * 2 + 1, m + 1, r, ql, qr));
        }
        return sum;
    }

public:
    // 构造函数：a[0] 占位，要求 n >= 1，建树 O(n)
    SegTree(const vector<int>& a) {
        assert(a.size() >= 2); // 至少包含占位和一个有效元素
```

```
        n = (int)a.size() - 1;
        tree.assign(4 * n + 5, 0);
        build(a, 1, 1, n);
    }

    // 单点赋值：a[pos] = v，要求 1 <= pos <= n，O(log n)
    void set(int pos, int v) {
        assert(1 <= pos && pos <= n);
        set(1, 1, n, pos, v);
    }

    // 单点加：a[pos] += v，要求 1 <= pos <= n，O(log n)
    void add(int pos, int v) {
        assert(1 <= pos && pos <= n);
        add(1, 1, n, pos, v);
    }

    // 区间求和：返回 [l,r] 的和，要求 1 <= l <= r <= n，O(log n)
    int query(int l, int r) {
        assert(1 <= l && l <= r && r <= n);
        return query(1, 1, n, l, r);
    }
};
```

调用与说明：SegTree s({0,1,2,3}); s.add(2,4); s.set(3,10); s.query(1,3) 返回 17。

改为区间最大值或最小值

以本节的求和代码为起点，最大值和最小值版本分别按下表修改；应在建树前改好代码，再重新构造线段树。

修改位置	区间和（当前）	区间最大值	区间最小值
merge 的返回值	return a + b	return max(a,b)	return min(a,b)
内部 query 的初始答案	int sum = 0	int sum = numeric_limits<int>::lowest()	int sum = numeric_limits<int>::max()
pushup 与 query 的合并调用	merge(a,b)	保持不变	保持不变

pushup 和 query 已统一调用 merge，合并调用不用改；只需改 merge 的返回值和 query 的初始答案。变量 sum 可重命名为 ans。不能将最大值查询的初始答案设为 0，因为元素可能全为负数。numeric_limits<int> 会随 #define int long long 使用对应类型的边界。

只需修改 merge 的返回值和内部 query 的初始答案。pushup、build、set、单点 add、构造函数和公开接口不变。

2.5.2 带懒标记线段树（segmenttree）

两版只需 `#include <bits/stdc++.h>` 和 `using namespace std;`，类名直接使用，无命名空间。两版分别位于 代码/02_数据结构/2.5.2 带懒标记线段树（segmenttree）.cpp 和 代码/02_数据结构/2.5.1 无懒标记线段树（SegTree）.cpp，每份文件只包含对应的类，均带完整起手式，可单独复制写题。需要 64 位时先包含标准头文件，再写 `#define int long long`，再放类和 `vector<int>` 输入数组，主函数用 `signed main()`；不要在宏之后首次包含标准头文件。

只需要单点修改时选择 SegTree；需要区间加时选择 segmenttree。两个版本的 add 参数不同：前者 add(pos,v)，后者 add(l,r,v)。

两个版本的区别

对比项	segmenttree（带懒标记）	SegTree（无懒标记）
区间查询	query(l,r)，区间和	query(l,r)，区间和
单点赋值	set(pos,v)	set(pos,v)
加法修改	add(l,r,v)，整个区间加 v	add(pos,v)，单个元素加 v
懒标记	lazy 保存已计入当前节点、尚未下传的增量	没有 lazy 和 pushdown
使用场景	需要区间加	只需要单点修改，代码更简单

两版建树 O(n)、空间 O(n)，各自提供的修改和查询均为 O(log n)。segmenttree 可以用 add(pos,pos,v) 实现单点加；SegTree 若循环进行 k 次单点加，需 O(k log n)，不能当成高效区间加。

用途与条件：输入 a[1..n]，a[0] 占位，n≥1。内外均为闭区间，要求 1≤l≤r≤n。支持区间加、单点赋值、区间求和。

复杂度：建树 O(n)，每次操作 O(log n)，空间 O(n)。

易错点：所有类型统一 int；区间和、增量乘长度和中间结果都必须在类型范围内。需要时在标准头文件之后、类定义之前添加 #define int long long。lazy 只记录每个元素的增量，不乘长度；单点赋值前下传，叶子赋值后清空标记。构造函数和公开接口使用 assert 检查数组非空、单点下标及区间范围；定义 NDEBUG 后断言禁用，调用方仍须保证参数合法。

```
class segmenttree {
private:
    int n;
    vector<int> tree, lazy; // 区间和、尚未下传的加法标记

    // 合并两个区间的结果：参数 a、b 为各自区间和，返回合并后的和
    int merge(int a, int b) { return a + b; }

    // 向上合并：通过 merge 用两个子节点的结果更新节点 p
    void pushup(int p) {
        tree[p] = merge(tree[p * 2], tree[p * 2 + 1]);
    }

    // 建树：节点 p 管理闭区间 [l,r]
    void build(const vector<int>& a, int p, int l, int r) {
        if (l == r) {
            tree[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(a, p * 2, l, m);
        build(a, p * 2 + 1, m + 1, r);
        pushup(p);
    }

    // 下传标记：更新孩子的区间和与标记，再清空当前标记
    void pushdown(int p, int l, int r) {
        if (l == r || lazy[p] == 0) return;
        int m = (l + r) / 2;
        tree[p * 2] += lazy[p] * (m - l + 1);
        tree[p * 2 + 1] += lazy[p] * (r - m);
        lazy[p * 2] += lazy[p];
        lazy[p * 2 + 1] += lazy[p];
        lazy[p] = 0;
    }

    // 单点赋值的递归实现：将 a[pos] 改为 v
    void set(int p, int l, int r, int pos, int v) {
        if (l == r) {
            tree[p] = v;
            lazy[p] = 0;
            return;
        }
        pushdown(p, l, r);
        int m = (l + r) / 2;
        if (pos <= m) {
            set(p * 2, l, m, pos, v);
        }
        else {
            set(p * 2 + 1, m + 1, r, pos, v);
        }
```

```
        pushup(p);
    }

    // 区间加的递归实现：将 [ql,qr] 内每个数增加 v
    void add(int p, int l, int r, int ql, int qr, int v) {
        if (ql <= l && r <= qr) {
            tree[p] += v * (r - l + 1);
            lazy[p] += v;
            return;
        }
        pushdown(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) {
            add(p * 2, l, m, ql, qr, v);
        }
        if (qr > m) {
            add(p * 2 + 1, m + 1, r, ql, qr, v);
        }
        pushup(p);
    }

    // 区间求和的递归实现：完全覆盖直接返回，否则累加子区间答案
    int query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tree[p];
        pushdown(p, l, r);
        int m = (l + r) / 2;
        int sum = 0;
        if (ql <= m) {
            sum = merge(sum, query(p * 2, l, m, ql, qr));
        }
        if (qr > m) {
            sum = merge(sum, query(p * 2 + 1, m + 1, r, ql, qr));
        }
        return sum;
    }

public:
    // 构造函数：a[0] 占位，要求 n >= 1，建树 O(n)
    segmenttree(const vector<int>& a) {
        assert(a.size() >= 2); // 至少包含占位和一个有效元素
        n = (int)a.size() - 1;
        tree.assign(4 * n + 5, 0);
        lazy.assign(4 * n + 5, 0);
        build(a, 1, 1, n);
    }

    // 单点赋值：a[pos] = v，要求 1 <= pos <= n，O(log n)
    void set(int pos, int v) {
        assert(1 <= pos && pos <= n);
        set(1, 1, n, pos, v);
    }

    // 区间加：[l,r] 内每个数增加 v，要求 1 <= l <= r <= n，O(log n)
    void add(int l, int r, int v) {
        assert(1 <= l && l <= r && r <= n);
        add(1, 1, n, l, r, v);
    }

    // 区间求和：返回 [l,r] 的和，要求 1 <= l <= r <= n，O(log n)
    int query(int l, int r) {
        assert(1 <= l && l <= r && r <= n);
```

```
        return query(1, 1, n, l, r);
    }
};
```

调用与说明：segmenttree s({0,1,2,3})；s.add(1,2,4)；s.set(2,10)；s.query(1,3) 返回 18。

改为区间最大值或最小值

以本节的求和代码为起点，最大值和最小值版本分别按下表修改；应在建树前改好代码，再重新构造线段树。

修改位置	区间和（当前）	区间最大值	区间最小值
merge 的返回值	return a + b	return max(a,b)	return min(a,b)
内部 query 的初始答案	int sum = 0	int sum = numeric_limits<int>::lowest()	int sum = numeric_limits<int>::max()
pushup 与 query 的合并调用	merge(a,b)	保持不变	保持不变

pushup 和 query 已统一调用 merge，合并调用不用改；只需改 merge 的返回值和 query 的初始答案。变量 sum 可重命名为 ans。不能将最大值查询的初始答案设为 0，因为元素可能全为负数。numeric_limits<int> 会随 #define int long long 使用对应类型的边界。

除上表外，还要改区间 add 和 pushdown，共四个函数。

区间 add 的完全覆盖分支，从 `tree[p] += v * (r-l+1);` 改为：

```
tree[p] += v;
lazy[p] += v;
```

pushdown 改为以下写法，最大值和最小值版本相同：

```
// 下传区间加标记：所有元素加 v，最大值和最小值也都加 v
void pushdown(int p, int l, int r)
{
    if (l == r || lazy[p] == 0) return;
    tree[p * 2] += lazy[p];
    tree[p * 2 + 1] += lazy[p];
    lazy[p * 2] += lazy[p];
    lazy[p * 2 + 1] += lazy[p];
    lazy[p] = 0;
}
```

build、单点 set、构造函数和公开接口不变。懒标记始终记录每个元素的增量；只有求和时，更新节点值才需要乘区间长度。若已改成最大值，再切换成最小值，只需改 merge 中的 max/min 及 query 的答案初始值。

2.6 主席树：静态区间第 k 小

用途与条件：值离散化为 1..m。root[i] 表示前 i 个元素的频数，root[0] 为空版本。查询原数组闭区间 [l,r] 第 k 小，k 从 1 开始。

复杂度：每次插入、查询 O(log m)；n 次插入空间 O(n log m)。

易错点：m≥1，1≤k≤r-l+1。输入值为 1..m 的秩。内部树递归区间 [L,R) 仅用于分割值域；0 号树节点为空节点。vector 扩容后不能继续使用旧引用。

```
struct PersistentKth {
    struct Node {
        // l、r：左右儿子的节点编号，0 表示空节点；cnt：该值域内的元素数量。
        int l = 0, r = 0, cnt = 0;
    };
    int m;
    // t：可持久化节点池，0 号为空节点。
    vector<Node> t{{}};
    // root[i]：插入前 i 个元素后的版本根；root[0] 表示空前缀。
    vector<int> root{0};
    // 设置离散值域为 1..m，创建空前缀版本 root[0]，要求 m>0。
    PersistentKth(int m) : m(m) {
        assert(m > 0);
    }
    // 内部持久化插入：复制 old 版本在值域 [l,r) 上通往秩 x 的路径，频数加一。
    // 返回新根编号，保留旧版本；要求 l<=x<r，新增节点数和时间均为 O(log m)。
```

```
    int insert(int old, int l, int r, int x) {
        int p = t.size();
        t.push_back(t[old]);
        ++t[p].cnt;
        if (r - l > 1) {
            int mid = (l + r) / 2;
            if (x < mid) {
                t[p].l = insert(t[old].l, l, mid, x);
            }
            else {
                t[p].r = insert(t[old].r, mid, r, x);
            }
        }
        return p;
    }
    // 向序列末尾追加离散秩 x，生成新的前缀频数版本；要求 1<=x<=m，O(log m)。
    void append(int x) {
        root.push_back(insert(root.back(), 1, m + 1, x));
    }
    // 返回原序列闭区间 [l,r] 的第 k 小离散秩，需再查离散值表取得实际值。
    // 要求 1<=l<=r<=已追加数量、1<=k<=r-l+1；时间 O(log m)。
    int kth(int l, int r, int k) {
        assert(1 <= k && k <= r - l + 1);
        int a = root[l - 1], b = root[r], L = 1, R = m + 1;
        while (R - L > 1) {
            int mid = (L + R) / 2, c = t[t[b].l].cnt - t[t[a].l].cnt;
            if (k <= c) {
                a = t[a].l;
                b = t[b].l;
                R = mid;
            }
            else {
                k -= c;
                a = t[a].r;
                b = t[b].r;
                L = mid;
            }
        }
        return L;
    }
};
```

调用与说明：依次 append(rank[i])（i=1..n），xs[tree.kth(l,r,k)] 即闭区间 [l,r] 第 k 小的实际值。

2.7 异或线性基

用途与条件：维护 64 位无符号整数的异或空间，支持插入、判断可表示性、最大异或和。 例外：二进制位编号为 0..63，基数组 b[i] 按位编号。

复杂度：每次操作 O(64)，空间 O(64)。

易错点：本模板允许选择空集。第 k 小异或值需要进一步消元及明确是否计入空集，不能直接把基按大小排列。

```
struct XorBasis {
    // b[i]：最高有效位为 i 的基向量，0 表示该位置尚无基向量。
    // XOR（Exclusive OR）：按位异或。
    array<u64, 64> b{};
    // 将 x 加入异或线性基；使线性空间维数增加时返回 true，否则返回 false。
    // 会修改基向量，处理完整 64 位无符号范围，时间 O(64)。
    bool insert(u64 x) {
        for (int i = 63; i >= 0; --i) {
            if ((x >> i) & 1) {
                if (!b[i]) {
                    b[i] = x;
```

```
                    return true;
                }
                x ^= b[i];
            }
        }
        return false;
    }
    // 判断 x 能否由已插入数的某个子集异或得到；允许空集，因此 0 总可表示，O(64)。
    bool contains(u64 x) const {
        for (int i = 63; i >= 0; --i) {
            if ((x >> i) & 1) {
                x ^= b[i];
            }
        }
        return x == 0;
    }
    // 返回 x 与线性空间中某个值异或后的最大无符号结果；默认 x=0 求最大子集异或和。
    // 不修改线性基，允许选择空集，时间 O(64)。
    u64 maximum(u64 x = 0) const {
        for (int i = 63; i >= 0; --i) {
            x = max(x, x ^ b[i]);
        }
        return x;
    }
};
```

调用与说明：插入 3 和 5 后，maximum()==6，contains(6)==true。

2.8 莫队：区间不同数个数

用途与条件：输入 a[1..n]，值离散化为 1..m。ranges 是无占位的查询列表，每项是闭区间 [l,r]；返回答案按查询列表顺序存放（列表位置从 0 开始）。

复杂度：块长约 sqrt(n)，时间 O((n+q)sqrt(n))，空间 O(n+m+q)。

易错点：本模板不支持修改。答案按原查询顺序返回。add/remove 必须互为逆操作；奇偶块排序降低常数。

```
// 离线统计 ranges 中各闭区间的不同值个数，答案按查询输入顺序返回，不带占位。
// a 使用 1 下标且值在 1..m；非空查询满足 1<=l<=r<=n，不支持修改。
vector<int> mo_distinct(const vector<int>& a, int m, const vector<pair<int, int>>& ranges) {
    int n = (int)a.size() - 1, q = ranges.size(), B = max(1, (int)sqrt(n));
    // ord：查询的处理顺序；ans：按原查询编号保存答案；cnt[x]：当前区间中值 x 的频数。
    vector<int> ord(q), ans(q), cnt(m + 1);
    iota(ord.begin(), ord.end(), 0);
    // 按左端点分块，块内右端点交替升降序排序，减少窗口指针移动；x、y 是查询编号。
    sort(ord.begin(), ord.end(), [&](int x, int y) {
        auto [l, r] = ranges[x];
        auto [L, R] = ranges[y];
        if (l / B != L / B) return l / B < L / B;
        return (l / B & 1) ? r > R : r < R;
    });
    // 当前窗口为 [l,r]，初始为空；cur：窗口内不同数值的数量。
    int l = 1, r = 0, cur = 0;
    // 把位置 i 纳入当前窗口，更新其频数；该值首次出现时增加不同值计数 cur。
    auto add = [&](int i) {
        if (cnt[a[i]]++ == 0) ++cur;
    };
    // 把当前窗口内的位置 i 移出，更新其频数；该值不再出现时减少 cur。
    auto del = [&](int i) {
        if (--cnt[a[i]] == 0) --cur;
    };
    for (int id : ord) {
        auto [L, R] = ranges[id];
        while (l > L) add(--l);
        while (r < R) add(++r);
```

```
        while (l < L) del(l++);
        while (r > R) del(r--);
        ans[id] = cur;
    }
    return ans;
}
```

调用与说明：mo_distinct({0,1,2,1},2,{{1,3},{2,2}}) 返回 {2,1}。

2.9 隐式 FHQ Treap：序列翻转

用途与条件：序列位置为 1..n，reverse(l,r) 翻转闭区间 [l,r]，要求 l≤r。values() 返回带下标 0 占位的数组。split 的 k 仍表示元素个数，节点 0 为空。

复杂度：期望单次 O(log n)，空间 O(n)；随机优先级下深度期望 O(log n)。

易错点：split 按元素个数分割。访问儿子前 push。合并要求 a 的全部元素在 b 之前。本实现不回收删除节点。

```
// FHQ Treap：基于 split/merge 的无旋 Treap；FHQ 来自作者名 fhq。
// Treap 由 Tree（树）和 Heap（堆）合成；隐式版本用中序位置表示序列下标。
struct ImplicitTreap {
    struct Node {
        // l、r：左右儿子编号，0 表示空树；sz：以当前节点为根的子树大小。
        int l = 0, r = 0, sz = 1;
        // pri（priority）：随机优先级，维护堆性质。
        u64 pri = 0;
        i64 val = 0;
        // rev（reverse）：待下传的区间翻转标记，不是反向下标。
        bool rev = false;
    };
    // t：节点池，0 号为空节点；有效节点从 1 开始。
    vector<Node> t{{}};
    int root = 0;
    mt19937_64 rng{712367};
    // 返回节点 p 的子树大小；节点编号 0 表示空树，返回 0，时间 O(1)。
    int size(int p) { return p ? t[p].sz : 0; }
    // 内部维护：根据左右孩子重新计算非空节点 p 的子树大小，时间 O(1)。
    void pull(int p) {
        t[p].sz = 1 + size(t[p].l) + size(t[p].r);
    }
    // 为子树 p 应用翻转：交换左右孩子并切换延迟翻转标记；空树不操作，O(1)。
    void flip(int p) {
        if (p) {
            swap(t[p].l, t[p].r);
            t[p].rev ^= 1;
        }
    }
    // 将非空节点 p 的延迟翻转标记下传给孩子，并清除自身标记，时间 O(1)。
    void push(int p) {
        if (t[p].rev) {
            flip(t[p].l);
            flip(t[p].r);
            t[p].rev = 0;
        }
    }
    // 按元素个数把子树 p 分为前 k 项与剩余项，返回两棵树的根；要求 0<=k<=size(p)。
    // 会重接子树并下传标记，期望时间 O(log n)。
    pair<int, int> split(int p, int k) {
        if (!p) return {0, 0};
        push(p);
        if (size(t[p].l) >= k) {
            auto [a, b] = split(t[p].l, k);
            t[p].l = b;
            pull(p);
```

```
            return {a, p};
        }
        auto [a, b] = split(t[p].r, k - size(t[p].l) - 1);
        t[p].r = a;
        pull(p);
        return {p, b};
    }
    // 把序列树 a 接在 b 前面合并，返回新根；要求 a 的所有项排在 b 的所有项之前。
    // 会修改树的连接关系，期望时间 O(log n)。
    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (t[a].pri > t[b].pri) {
            push(a);
            t[a].r = merge(t[a].r, b);
            pull(a);
            return a;
        }
        push(b);
        t[b].l = merge(a, t[b].l);
        pull(b);
        return b;
    }
    // 在当前序列末尾添加值 v，并更新 root；期望时间 O(log n)，新增一个节点。
    void append(i64 v) {
        int p = t.size();
        Node x;
        x.pri = rng();
        x.val = v;
        t.push_back(x);
        root = merge(root, p);
    }
    // 原地翻转序列的闭区间 [l,r]，要求 1<=l<=r<=序列长度；期望时间 O(log n)。
    void reverse(int l, int r) {
        auto [a, c] = split(root, r);
        auto [x, b] = split(a, l - 1);
        flip(b);
        root = merge(merge(x, b), c);
    }
    // 内部中序遍历：把子树 p 的值依次追加到 a，遍历时下传翻转标记。
    // 不清空 a；耗时与子树大小成正比，递归空间与树高成正比。
    void collect(int p, vector<i64>& a) {
        if (!p) return;
        push(p);
        collect(t[p].l, a);
        a.push_back(t[p].val);
        collect(t[p].r, a);
    }
    // 导出当前完整序列，返回 a[1..n]，a[0] 为占位；时间和输出空间 O(n)。
    vector<i64> values() {
        vector<i64> a(1);
        collect(root, a);
        return a;
    }
};
```

调用与说明：依次 append(1),append(2),append(3)，reverse(1,2) 后 values() 为 {0,2,1,3}。

2.10 01 Trie：二进制字典树

用途与条件：维护非负整数多重集合，支持插入、删除一个数、计数和最大异或查询。构造参数 max_bit 是最高位编号（0..63），默认 63；节点从 1 开始，根为 1，0 表示空子节点。二进制位从 0 开始编号。

复杂度：插入、删除、计数和最大异或查询均为 O(max_bit+1)，size 为 O(1)。空间 O(1+历史插入次数×(max_bit+1))；clear 清空节点但不保证释放容量。

易错点：所有操作的参数 x 必须在设定的位宽内；max_bit=63 时支持完整 u64 范围。max_xor 要求集合非空，返回异或值而非选中的数。删除只减少计数，不回收节点，查询必须跳过计数为 0 的分支。节点编号和计数使用 int，规模须在其范围内。

```
// 01 Trie（Binary Trie）：二进制字典树，支持重复数。
class BinaryTrie {
private:
    struct Node {
        int ch[2] = {};  // 当前位为 0、1 的子节点，0 表示不存在
        int cnt = 0;     // 经过该节点的数的数量，包含重复数
    };

    int max_bit;      // 最高二进制位编号，共处理 max_bit+1 位
    vector<Node> tr;  // tr[0] 占位，tr[1] 为根；删除后保留节点

public:
    // 建立空字典树；最高位编号 max_bit 必须在 0..63 内。
    BinaryTrie(int max_bit = 63) : max_bit(max_bit), tr(2) {
        assert(0 <= max_bit && max_bit <= 63);
    }

    // 清空所有数并恢复根节点，保留设定的最高位编号。
    void clear() {
        tr.assign(2, Node{});
    }

    // 返回当前数的数量，包含重复数。
    int size() const {
        return tr[1].cnt;
    }

    // 返回 x 的出现次数；x 必须在设定位宽内，不存在时返回 0。
    int count(u64 x) const {
        assert(max_bit == 63 || (x >> (max_bit + 1)) == 0);
        int u = 1;
        for (int i = max_bit; i >= 0; i--) {
            int bit = (x >> i) & 1ULL;
            int v = tr[u].ch[bit];
            if (v == 0 || tr[v].cnt == 0) return 0;
            u = v;
        }
        return tr[u].cnt;
    }

    // 插入位宽范围内的 x，允许重复插入。
    void insert(u64 x) {
        assert(max_bit == 63 || (x >> (max_bit + 1)) == 0);
        int u = 1;
        tr[u].cnt++;
        for (int i = max_bit; i >= 0; i--) {
            int bit = (x >> i) & 1ULL;
            if (tr[u].ch[bit] == 0) {
                int v = (int)tr.size();
                tr.emplace_back();
                tr[u].ch[bit] = v;
            }
            u = tr[u].ch[bit];
            tr[u].cnt++;
        }
```

```
    }

    // 删除位宽范围内的一个 x；成功返回 true，不存在则返回 false。
    bool erase(u64 x) {
        if (count(x) == 0) return false;
        int u = 1;
        tr[u].cnt--;
        for (int i = max_bit; i >= 0; i--) {
            int bit = (x >> i) & 1ULL;
            u = tr[u].ch[bit];
            tr[u].cnt--;
        }
        return true;
    }

    // 返回集合中各个 y 与 x 异或后的最大值，不是 y 本身。
    // 集合必须非空，x 必须在设定位宽内。
    u64 max_xor(u64 x) const {
        assert(size() > 0);
        assert(max_bit == 63 || (x >> (max_bit + 1)) == 0);
        int u = 1;
        u64 ans = 0;
        for (int i = max_bit; i >= 0; i--) {
            int bit = (x >> i) & 1ULL;
            int v = tr[u].ch[bit ^ 1];
            // 高位优先选择相反位，使异或结果尽量大；跳过已删除的分支。
            if (v != 0 && tr[v].cnt > 0) {
                ans |= 1ULL << i;
                u = v;
            }
            else {
                u = tr[u].ch[bit];
            }
        }
        return ans;
    }
};
```

调用与说明：BinaryTrie trie(30); 处理第 30..0 位，要求 x<2^31。依次 insert(5)、insert(5)、insert(3)，erase(5) 后 count(5)==1、max_xor(2)==7；再 erase(5) 后 max_xor(2)==1；再次 erase(5) 返回 false。默认 BinaryTrie trie; 支持完整 64 位范围。

3. 图论与树

3.1 单源最短路

3.1.1 Dijkstra：非负权最短路

用途与条件：有向或无向非负权图，从 s 出发求最短路。g[u] 中保存 {v,w}。 顶点编号从 1 开始；传入邻接表时分配 n+1 行，第 0 行不使用。

复杂度：二叉堆实现 O((n+m)log(n+m))；空间 O(n+m)。

易错点：禁止负边。不可达返回 INF。要求有限答案严格小于 INF；本实现跳过达到哨兵的距离。无向边需添加两个方向。

```
const i64 INF = (1LL << 62);

// 求源点 s 到各点的最短距离，g[u] 存 {终点,边权}，顶点使用 1..n。
// 要求边权非负，返回 dist[1..n]，不可达为 INF，有限答案须小于 INF；不修改图。
// 堆实现时间 O((n+m)log(n+m))，空间 O(n+m)。
vector<i64> dijkstra(const vector<vector<pair<int, i64>>>& g, int s) {
    vector<i64> dist(g.size(), INF);
    dist[s] = 0;
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<pair<i64, int>>> q;
    q.push({0, s});
    while (!q.empty()) {
        auto [du, u] = q.top();
        q.pop();
        if (du != dist[u]) continue;
        for (auto [v, w] : g[u]) {
            assert(w >= 0);
            if (w >= INF - du) continue;
            if (dist[v] > du + w) {
                dist[v] = du + w;
                q.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

调用与说明：vector<vector<pair<int,i64>>> g(n); g[u].push_back({v,w}); auto dist=dijkstra(g,s);

3.1.2 0-1 BFS

用途与条件：边权只有 0 或 1 时，用双端队列求最短路。0 权边入队首，1 权边入队尾。 顶点编号从 1 开始；传入邻接表时分配 n+1 行，第 0 行不使用。

复杂度：时间 O(n+m)，空间 O(n+m)。

易错点：不要在入队时标记已访问，距离仍可能改进。这里在第一次有效出队时确定最短距离。

```
// 求源点 s 的 0/1 边权最短路；g 使用 1 下标，边记录为 {终点,0或1}。
// 返回各点距离，不可达为 1LL<<62；不修改图，时间 O(n+m)。
// BFS（Breadth-First Search）：广度优先搜索；0-1 BFS 用双端队列处理 0/1 边权。
vector<i64> bfs01(const vector<vector<pair<int, int>>>& g, int s) {
    const i64 INF = (1LL << 62);
    vector<i64> dist(g.size(), INF);
    // done[u]：顶点 u 是否已经确定最短距离并展开，防止重复处理。
    vector<char> done(g.size());
    deque<int> q;
    dist[s] = 0;
    q.push_front(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop_front();
        if (done[u]) continue;
        done[u] = 1;
        for (auto [v, w] : g[u]) {
            assert(w == 0 || w == 1);
```

```
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                if (w) {
                    q.push_back(v);
                }
                else {
                    q.push_front(v);
                }
            }
        }
    }
    return dist;
}
```

调用与说明：不可达点距离为 (1LL << 62)。无权图直接使用普通队列 BFS。

3.1.3 Bellman-Ford：负边与可达负环

用途与条件：允许负边，检测从 s 可达的负环。若返回 true，距离数组不再作为有效最短路结果使用。 顶点编号从 1 开始；传入邻接表时分配 n+1 行，第 0 行不使用。

复杂度：时间 O(nm)，空间 O(n)。

易错点：只检查源点可达负环。若需检查任意负环，可将所有初始距离设为 0，等价于加入超级源点。使用 i128 避免多轮松弛过早溢出。

```
struct Edge {
    int u, v;
    i64 w;
};
// 在 n 个点、边列表 e 中求 s 出发的最短路，并检查可达负环；点编号 1..n。
// 返回 {有无可达负环,距离}，有负环时距离无效，不可达为 i128(1)<<120。
// 允许负权，不修改输入；时间 O(nm)，空间 O(n)。
pair<bool, vector<i128>> bellman_ford(int n, const vector<Edge>& e, int s) {
    const i128 INF = (i128(1) << 120);
    vector<i128> dist(n + 1, INF);
    dist[s] = 0;
    for (int it = 0; it < n; ++it) {
        bool changed = false;
        for (auto [u, v, w] : e) {
            if (dist[u] != INF && dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                changed = true;
            }
        }
        if (!changed) return {false, dist};
        if (it == n - 1) return {true, dist};
    }
    return {false, dist};
}
```

调用与说明：auto [negative,dist]=bellman_ford(n,edges,s); 仅当 negative==false 时使用 dist；不可达为 i128(1)<<120。

3.2 拓扑排序与 DAG 动态规划

用途与条件：Kahn 算法返回有向图拓扑序。返回长度小于 n 说明存在有向环。 顶点编号从 1 开始；传入邻接表时分配 n+1 行，第 0 行不使用。

复杂度：时间 O(n+m)，空间 O(n)。

易错点：拓扑序不一定唯一。在拓扑序中按边转移即可做 DAG DP；最长路初始化要区分不可达与 0。

```
// 对 g[1..n] 做拓扑排序，返回无占位的顶点顺序列表；不修改图。
// 返回长度不足 n 表示存在有向环，此时列表并非全图拓扑序；时间 O(n+m)。
// DAG（Directed Acyclic Graph）：有向无环图。
// DP（Dynamic Programming）：动态规划，可按 DAG 的拓扑序转移。
vector<int> topo_sort(const vector<vector<int>>& g) {
    int n = (int)g.size() - 1;
```

```
    // deg[u]：尚未删除的入边数量；ans：依次取出的拓扑序顶点。
    vector<int> deg(n + 1), ans;
    queue<int> q;
    for (auto& es : g) {
        for (int v : es) ++deg[v];
    }
    for (int i = 1; i <= n; ++i) {
        if (!deg[i]) q.push(i);
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);
        for (int v : g[u]) {
            if (--deg[v] == 0) q.push(v);
        }
    }
    return ans;
}
```

调用与说明：auto ord=topo_sort(g); if(ord.size()!=g.size()-1) 则不是 DAG。ord 为无占位的顶点顺序列表。

3.3 Kruskal：最小生成森林

用途与条件：无向带权图，可含负权、重边、自环。返回最小生成森林权值与选中边数。顶点编号为 1..n；Edge 保存起点、终点和 i64 边权，边列表从 0 下标开始。依赖 2.1 并查集，独立代码中已展开。

复杂度：时间 O(m log m)，空间 O(n+m)。

易错点：选中边数为 n-1 才是整张图的一棵生成树。权值之和须在 i64 范围内。不要把断开图的森林权值误当作 MST。

依赖：查看 2.1「并查集」（第 10 页）；完整代码已展开依赖，可单独使用。

```
struct Edge {
    int u, v;
    i64 w;
};
// 使用 2.1 并查集的 init、find、merge；独立文件中已展开依赖。
// 求 n 个顶点的最小生成森林，es 为无向边列表，可有负权、重边、自环。
// 返回 {总权值,选中边数}；选中 n-1 条边才表示整图连通，权值和须在 i64 范围内。
// 要求 n>=1，顶点为 1..n；不修改调用者的 es，会重置全局并查集。
// 时间 O(n + m log(m+1))，空间 O(n+m)。
pair<i64, int> kruskal(int n, vector<Edge> es) {
    init(n);
    // 按边权从小到大比较两条边，供 Kruskal 依次尝试选边。
    sort(es.begin(), es.end(), [](const Edge& a, const Edge& b) { return a.w < b.w; });
    int num = n;
    i64 result = 0;
    for (int i = 0; i < (int)es.size(); ++i) {
        if (!merge(es[i].u, es[i].v)) continue;
        --num;
        result += es[i].w;
        if (num == 1) break;
    }
    return {result, n - num};
}
```

调用与说明：auto [weight,used]=kruskal(n,edges); 检查 used==n-1。

3.4 强连通分量

3.4.1 迭代 Kosaraju

用途与条件：迭代 Kosaraju 求有向图强连通分量；顶点 1..n，使用原图和反图，两轮遍历，无递归调用栈。comp 为 0..cc-1 的分量编号。

复杂度：时间 O(n+m)，总空间 O(n+m)，保存原图和反图。

易错点：跨分量边 u→v 满足 comp[u]<comp[v]；接口 add_edge、build、comp、cc。与 Tarjan 编号方向不同。

```
// SCC（Strongly Connected Component）：强连通分量；此处采用 Kosaraju 算法。
// DFS（Depth-First Search）：深度优先搜索，本实现用显式栈模拟。
struct SCC {
    // cc：强连通分量数量，已分配的分量编号为 0..cc-1。
    int n, cc = 0;
    // g：原图；rg（reverse graph）：将全部有向边反转后的图。
    vector<vector<int>> g, rg;
    // comp[u]：顶点 u 所属分量的编号，-1 表示尚未分配。
    vector<int> comp;
    // 初始化编号 1..n 的有向图和反图；时间和空间 O(n)。
    SCC(int n) : n(n), g(n + 1), rg(n + 1) {}
    // 添加有向边 u→v，同时在反图加入 v→u；要求端点在 1..n，均摊 O(1)。
    void add_edge(int u, int v) {
        g[u].push_back(v);
        rg[v].push_back(u);
    }
    // 用迭代 Kosaraju 重算强连通分量，将数量写入 cc、编号写入 comp[1..n]。
    // 编号为 0..cc-1，跨分量边 u→v 满足 comp[u]<comp[v]；可加边后重新调用。
    // 时间 O(n+m)，无需递归调用栈。
    void build() {
        cc = 0;
        vector<char> vis(n + 1);
        // order：第一遍 DFS 的完成顺序，反转后用于第二遍遍历。
        vector<int> order;
        for (int s = 1; s <= n; ++s) {
            if (!vis[s]) {
                // st：模拟递归的栈；每项保存顶点和下一条待处理邻边的下标。
                vector<pair<int, int>> st{{s, 0}};
                vis[s] = 1;
                while (!st.empty()) {
                    int u = st.back().first;
                    int& i = st.back().second;
                    if (i == (int)g[u].size()) {
                        order.push_back(u);
                        st.pop_back();
                    }
                    else {
                        int v = g[u][i++];
                        if (!vis[v]) {
                            vis[v] = 1;
                            st.push_back({v, 0});
                        }
                    }
                }
            }
        }
        comp.assign(n + 1, -1);
        reverse(order.begin(), order.end());
        for (int s : order) {
            if (comp[s] == -1) {
                vector<int> st{s};
                comp[s] = cc;
                while (!st.empty()) {
                    int u = st.back();
                    st.pop_back();
                    for (int v : rg[u]) {
                        if (comp[v] == -1) {
                            comp[v] = cc;
                            st.push_back(v);
```

```
                        }
                    }
                }
                ++cc;
            }
        }
    }
};
```

调用与说明：SCC s(n); s.add_edge(u,v); s.build(); s.comp[u] 为分量编号，s.cc 为分量数量。

3.4.2 递归 Tarjan

用途与条件：递归 Tarjan 求有向图强连通分量；顶点 1..n，使用 dfn、low 和节点栈，不需要反图。comp 为 0..cc-1 的分量编号。

复杂度：时间 O(n+m)，总空间 O(n+m)，递归深度最坏 O(n)。

易错点：跨分量边 u→v 满足 comp[u]>comp[v]；长链可能使递归栈溢出。缩点后重边是否去重按题意决定。

```
// SCC（Strongly Connected Component）：强连通分量；此处采用 Tarjan 算法。
// DFS（Depth-First Search）：深度优先搜索。
struct TarjanSCC {
    int n, cc = 0, timer = 0;
    vector<vector<int>> g;
    // dfn[u]：首次访问时间戳，0 表示未访问；low[u]：按 Tarjan 转移维护的可达最小时间戳。
    // st：尚未归属分量的顶点栈；comp[u]：所属强连通分量编号。
    // cc：分量数量；timer：DFS 时间戳计数器。
    vector<int> dfn, low, st, comp;
    // in_stack[u]：u 是否仍在分量栈中；已出栈顶点不能用于降低 low。
    vector<char> in_stack;
    // 初始化编号 1..n 的有向图；遍历数组在 build 中初始化。
    TarjanSCC(int n) : n(n), g(n + 1) {}
    // 添加有向边 u→v，端点需在 1..n；均摊 O(1)。
    void add_edge(int u, int v) {
        g[u].push_back(v);
    }
    // 内部 Tarjan 遍历：访问尚未访问的 u，维护 dfn、low 和栈，并弹出完整强连通分量。
    // 由 build 初始化状态后调用，可能递归至 O(n) 深度。
    void dfs(int u) {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        in_stack[u] = 1;
        for (int v : g[u]) {
            if (!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            }
            else if (in_stack[v]) {
                // 只有仍在分量栈中的顶点才能用于降低 low[u]，避免合并已完成的分量。
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (low[u] == dfn[u]) {
            while (true) {
                int v = st.back();
                st.pop_back();
                in_stack[v] = 0;
                comp[v] = cc;
                if (v == u) break;
            }
            ++cc;
        }
    }
    // 清空遍历状态并重算全部强连通分量，写入 comp[1..n] 与数量 cc。
```

```
    // 编号为 0..cc-1，跨分量边 u→v 满足 comp[u]>comp[v]；时间 O(n+m)。
    void build() {
        cc = timer = 0;
        st.clear();
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        comp.assign(n + 1, -1);
        in_stack.assign(n + 1, 0);
        for (int u = 1; u <= n; ++u) {
            if (!dfn[u]) dfs(u);
        }
    }
};
```

调用与说明：TarjanSCC s(n); s.add_edge(u,v); s.build(); 用 s.comp[u]==s.comp[v] 判断两点是否属于同一 SCC；s.cc 是分量数。修改图后可以重新 build()。

两种实现的区别

对比项	迭代 Kosaraju（SCC）	递归 Tarjan（TarjanSCC）
核心思路	原图求完成顺序，再按完成顺序的逆序遍历反图	一轮 DFS，用 dfn、low 和待归属节点栈识别 SCC
遍历	两轮遍历，每轮可启动多次 DFS	一轮遍历，每个未访问起点启动 DFS
反图	需要；额外保存 m 条反向邻接记录	不需要
时间 / 总空间	O(n+m) / O(n+m)	O(n+m) / O(n+m)
栈	显式栈，无递归调用栈风险	节点栈加递归调用栈；长链递归深度可达 n
本版分量编号	跨分量边 u→v 满足 comp[u]<comp[v]	跨分量边 u→v 满足 comp[u]>comp[v]
选用场景	图很深、希望避免递归时可直接采用	希望省去反图且递归深度可控时可采用

两种算法得到相同的分组，但编号不必相同，也不保证两组编号逐项互为逆序；无依赖关系的分量顺序可以不同。例如图中只有边 1→2 时，本版 Kosaraju 得到 comp[1..2]={0,1}，Tarjan 得到 comp[1..2]={1,0}。编号方向来自这里的具体实现，不应仅凭算法名称推断其他模板的编号规则。

Tarjan 中 dfn、low 与栈的含义

- `dfn[u]`：u 第一次被访问的时间戳，从 1 开始；0 表示未访问。
- `low[u]`：通过 DFS 子树及指向仍在栈中节点的边，能够追溯到的最小时间戳。
- `st` / `in_stack`：记录已访问但尚未归入某个 SCC 的节点，不等同于递归调用栈。
- 若 v 未访问，先递归再用 `low[v]` 更新；若 v 已访问且仍在栈中，用 `dfn[v]` 更新；若 v 已出栈，则忽略这条边对 low 的影响。
- 当 `low[u]==dfn[u]` 时，从栈顶弹出直到 u，这些点恰好组成一个 SCC。有向图 SCC 不使用无向图“跳过父边”的写法。

与下一节 2-SAT 的配合

下一节默认保留 `SCC`（Kosaraju），赋值为 `value[i] = comp[2*i-1] < comp[2*i]`。

若换成本节 `TarjanSCC`，应同时做两处修改：成员 `SCC s` 改成 `TarjanSCC s`，赋值比较改成 `value[i] = comp[2*i-1] > comp[2*i]`。无解判定 `comp[2*i-1]==comp[2*i]` 保持不变。使用独立算法文件时，成员可直接写 `TarjanSCC s`；其完整代码位于 `代码/03_图论与树/3.4.2 递归 Tarjan.cpp`；Kosaraju 位于 `代码/03_图论与树/3.4.1 迭代 Kosaraju.cpp`，两种方法分别存放。

3.5 2-SAT：布尔约束可满足性

用途与条件：变量编号 1..n，字面量 node(i,v)=2i-1+v：false 为 2i-1，true 为 2*i。默认依赖 Kosaraju SCC，clause(a,av,b,bv) 添加 (a==av) 或 (b==bv)。

复杂度：n 个变量、m 个子句：时间和空间 O(n+m)。

易错点：赋值方向依赖 SCC 编号顺序：默认 Kosaraju 用 <；若成员改为上一节 TarjanSCC，必须同时把赋值比较改为 >。强制 x=v 使用 clause(x,v,x,v)。

依赖：查看 3.4.1「迭代 Kosaraju」（第 27 页）；完整代码已展开依赖，可单独使用。

```
// Dependency: SCC from the previous entry.
// 2-SAT（2-Satisfiability）：每个子句至多包含两个文字的布尔可满足性问题。
struct TwoSAT {
    int n;
    SCC s;
    // value[i]：求解成功后第 i 个布尔变量的取值，0 为假、1 为真。
    vector<int> value;
    // 创建 n 个布尔变量的蕴含图，变量编号 1..n，每个变量对应两个图节点。
    TwoSAT(int n) : n(n), s(2 * n) {}
    // 加入子句 (变量 a 等于 av) 或 (变量 b 等于 bv)，转成两条蕴含边。
    // 变量编号为 1..n；clause(x,v,x,v) 可强制 x=v，均摊 O(1)。
    void clause(int a, bool av, int b, bool bv) {
        int x = 2 * a - 1 + av, y = 2 * b - 1 + bv;
        s.add_edge((x % 2 ? x + 1 : x - 1), y);
        s.add_edge((y % 2 ? y + 1 : y - 1), x);
    }
    // 判断全部子句是否可同时满足；可满足返回 true，并把一组赋值写入 value[1..n]。
    // 返回 false 时不能使用 value 作为解；此处赋值比较配合 Kosaraju 编号，时间 O(n+m)。
    bool solve() {
        s.build();
        value.resize(n + 1);
        for (int i = 1; i <= n; ++i) {
            if (s.comp[2 * i - 1] == s.comp[2 * i]) return false;
            value[i] = s.comp[2 * i - 1] < s.comp[2 * i];
        }
        return true;
    }
};
```

调用与说明：TwoSAT t(2); t.clause(1,true,2,false); 若 t.solve() 为 true，t.value[1..n] 给出一组合法解。

3.6 无向图割点与桥

用途与条件：Tarjan low-link 求割点、桥，支持重边。每条无向边分配唯一 id，两个方向共享该 id。 顶点编号从 1 开始；传入邻接表时分配 n+1 行，第 0 行不使用。 边 id 使用互不相同的非负整数，返回 bridges 是无占位的 id 列表。

复杂度：时间 O(n+m)，空间 O(n+m)。

易错点：跳过父边 id 而非父顶点，否则重边会出错。桥条件 low[v]>dfn[u]；非根割点条件 low[v]≥dfn[u]。递归深度可达 n。

```
struct LowLink {
    int n, timer = 0;
    vector<vector<pair<int, int>>> g;
    // DFS（Depth-First Search）：深度优先搜索。
    // dfn[u]：首次访问时间戳；low[u]：从子树经树边及至多一条返祖边可达的最小时间戳。
    // bridges：桥的边编号列表；timer：访问时间戳计数器。
    vector<int> dfn, low, bridges;
    // cut[u]：顶点 u 是否为割点。
    vector<char> cut;
    // 创建 n 个顶点的无向图，初始化访问时间、low 值和割点标记；点编号为 1..n。
    LowLink(int n) : n(n), g(n + 1), dfn(n + 1), low(n + 1), cut(n + 1) {}
    // 添加无向边 u-v，两个方向使用同一 id；每条物理边需有独立非负 id，以支持重边。
    void add_edge(int u, int v, int id) {
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    }
    // 内部割点/桥搜索：从未访问顶点 u 出发，pe 为父边编号，根使用 -1。
    // 更新 low、cut 和 bridges，按父边 id 跳过反向边；递归深度最多 O(n)。
    void dfs(int u, int pe) {
        dfn[u] = low[u] = ++timer;
        int child = 0;
```

```
        for (auto [v, id] : g[u]) {
            if (id == pe) continue;
            if (!dfn[v]) {
                ++child;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) {
                    bridges.push_back(id);
                }
                if (pe != -1 && low[v] >= dfn[u]) {
                    cut[u] = 1;
                }
            }
            else {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (pe == -1 && child > 1) {
            cut[u] = 1;
        }
    }
    // 遍历所有连通块，写入割点标记 cut[1..n] 和桥的边编号列表 bridges，时间 O(n+m)。
    // 应在全部边添加完后调用一次；本实现不清空旧状态，修改图后需重建对象。
    void build() {
        for (int i = 1; i <= n; ++i) {
            if (!dfn[i]) dfs(i, -1);
        }
    }
};
```

调用与说明：对象建完边后调用一次 build()。删去所有桥后求连通块可得边双连通分量；点双连通分量还需维护边栈，不能直接套此结论。

3.7 树链剖分、LCA 与子树区间

用途与条件：输入 g[1..n] 为连通无向树，默认根 1。迭代预处理生成 1 下标 dfs 序。path(u,v,f) 将点路径拆成若干闭区间 [l,r]。

复杂度：预处理 O(n)；LCA O(log n)，路径拆分 O(log n) 段；结合线段树查询为 O(log² n)。

易错点：子树为 [pos[u],pos[u]+sz[u]-1]。内部 ord、栈是无占位列表。path 各段无方向保证，只适合交换运算；路径字符串需单独处理方向。

```
// HLD（Heavy-Light Decomposition）：树链剖分。
// DFS（Depth-First Search）：深度优先搜索；DFS 序使子树对应连续区间。
struct HLD {
    int n;
    vector<vector<int>> g;
    // father[u]：父节点，根的父节点是自身；dep（depth）：深度，根为 0。
    // sz（size）：子树大小，包含自身；heavy[u]：重儿子，-1 表示没有。
    // head[u]：所在重链的链头；pos[u]：从 1 开始的 DFS 序编号。
    // rev[i]：DFS 序编号为 i 的顶点，rev[pos[u]] = u。
    vector<int> father, dep, sz, heavy, head, pos, rev;
    // 以 root 为根对非空连通无向树 g[1..n] 做迭代树链剖分，构造父亲、深度和 dfs 序。
    // pos/rev 使用 1 下标，子树区间为 [pos[u],pos[u]+sz[u]-1]；时间和空间 O(n)。
    HLD(vector<vector<int>> g, int root = 1)
        : n((int)g.size() - 1), g(move(g)), father(n + 1, -1), dep(n + 1), sz(n + 1, 1),
          heavy(n + 1, -1), head(n + 1), pos(n + 1), rev(n + 1) {
        // ord（order）：父节点先于子节点的遍历顺序，逆序处理即可汇总子树大小。
        vector<int> ord{root};
        father[root] = root;
        for (int i = 0; i < n; ++i) {
            int u = ord[i];
            for (int v : this->g[u]) {
                if (v != father[u]) {
```

```
                    father[v] = u;
                    dep[v] = dep[u] + 1;
                    ord.push_back(v);
                }
            }
        }
        for (int i = n - 1; i > 0; --i) {
            int u = ord[i], p = father[u];
            sz[p] += sz[u];
            if (heavy[p] == -1 || sz[u] > sz[heavy[p]]) {
                heavy[p] = u;
            }
        }
        int timer = 1;
        vector<pair<int, int>> st{{root, root}};
        while (!st.empty()) {
            auto [u, h] = st.back();
            st.pop_back();
            for (; u != -1; u = heavy[u]) {
                head[u] = h;
                pos[u] = timer;
                rev[timer++] = u;
                for (int v : this->g[u]) {
                    if (v != father[u] && v != heavy[u]) {
                        st.push_back({v, v});
                    }
                }
            }
        }
    }
    // 返回树上顶点 u、v 的最近公共祖先，要求顶点有效且已完成构造；时间 O(log n)。
    // LCA（Lowest Common Ancestor）：最近公共祖先。
    int lca(int u, int v) {
        while (head[u] != head[v]) {
            if (dep[head[u]] < dep[head[v]]) swap(u, v);
            u = father[head[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }
    // 将 u 到 v 的点路径拆成若干 dfs 序闭区间，逐段调用 f(l,r)，包含两端与 LCA。
    // 共 O(log n) 段，但段间顺序和方向无保证；适合求和等可交换运算。
    template <class F>
    void path(int u, int v, F f) {
        while (head[u] != head[v]) {
            if (dep[head[u]] < dep[head[v]]) swap(u, v);
            f(pos[head[u]], pos[u]);
            u = father[head[u]];
        }
        if (pos[u] > pos[v]) swap(u, v);
        f(pos[u], pos[v]);
    }
};
```

调用与说明：先令 base[h.pos[u]]=weight[u]，用 base 建线段树。h.path(u,v,[&](int l,int r) { ans+=seg.query(l,r); }); 边权放在较深端点时，最后一段要排除 LCA。

3.8 树上前缀和

3.8.1 点权前缀和（TreePointPrefix）

树上前缀和用于静态路径求和，分别提供点权与边权两个类，统一使用 query(u,v)。两版复用手册 3.7 的 HLD 求 LCA，独立文件已展开依赖。与批量路径加、最后还原的树上差分不同，前缀和构造后不支持直接修改权值。

用途与条件：输入非空连通无向树 g[1..n] 和点权 a[1..n]，默认根为 1，查询任意两点路径上的点权和，包含两端及 LCA。
复杂度：预处理 O(n)，单次查询 O(log n)，空间 O(n)。
易错点：pre[u] 保存根到 u 的点权和。令 p=lca(u,v)，答案为 pre[u]+pre[v]-2*pre[p]+a[p]。支持负权；权值改变后需重新构造。所有中间结果须在 i64 范围内。
依赖：查看 3.7「树链剖分、LCA 与子树区间」（第 32 页）；完整代码已展开依赖，可单独使用。

```
// 依赖手册 3.7 的 HLD（树链剖分、LCA 与子树区间），独立文件已展开。
class TreePointPrefix {
private:
    HLD h;
    vector<i64> a; // a[u]：点 u 的权值，a[0] 不参与计算。
    vector<i64> pre; // pre[u]：根到 u 的点权和，包含根和 u。

public:
    // 用树 g[1..n]、点权 a[1..n] 和有效根 root 构造静态前缀和；时间 O(n)。
    // 要求 g 为非空连通无向树，a 与 g 大小相同；迭代处理，不使用递归。
    TreePointPrefix(const vector<vector<int>>& g, const vector<i64>& a, int root = 1)
        : h(g, root), a(a), pre(g.size(), 0) {
        assert(a.size() == g.size());
        pre[root] = a[root];
        for (int i = 2; i <= h.n; ++i) {
            int u = h.rev[i];
            pre[u] = pre[h.father[u]] + a[u];
        }
    }

    // 返回 u 到 v 的点权和，包含两端；u==v 时返回 a[u]，时间 O(log n)。
    i64 query(int u, int v) {
        assert(1 <= u && u <= h.n && 1 <= v && v <= h.n);
        int p = h.lca(u, v);
        return pre[u] + pre[v] - 2 * pre[p] + a[p];
    }
};
```

调用与说明：TreePointPrefix tree(g,a); cout << tree.query(u,v) << endl; 对于链 1—2—3，a={0,2,-1,4}，query(1,3)=5，query(2,2)=-1。

3.8.2 边权前缀和（TreeEdgePrefix）

用途与条件：输入非空连通无向树 g[1..n] 和父边权 a[1..n]，默认根为 1。a[u] 为指定根下父节点到 u 的边权，根的 a 值忽略；查询任意两点路径上的边权和。
复杂度：预处理 O(n)，单次查询 O(log n)，空间 O(n)。
易错点：pre[root]=0；令 p=lca(u,v)，答案为 pre[u]+pre[v]-2*pre[p]。支持负权，u==v 时为 0。a 必须与 root 的父子方向一致，换根时需重新分配父边权；权值改变后需重新构造。所有中间结果须在 i64 范围内。
依赖：查看 3.7「树链剖分、LCA 与子树区间」（第 32 页）；完整代码已展开依赖，可单独使用。

```
// 依赖手册 3.7 的 HLD（树链剖分、LCA 与子树区间），独立文件已展开。
class TreeEdgePrefix {
private:
    HLD h;
    vector<i64> pre; // pre[u]：根到 u 的边权和，根及 pre[0] 为 0。

public:
    // 用树 g[1..n] 和父边权 a[1..n] 构造静态前缀和；时间 O(n)，不使用递归。
    // 要求 g 为非空连通无向树，1<=root<=n，a 与 g 大小相同。
    // a[u] 是以 root 为根时父节点到 u 的边权；a[root] 和 a[0] 忽略。
    TreeEdgePrefix(const vector<vector<int>>& g, const vector<i64>& a, int root = 1)
        : h(g, root), pre(g.size(), 0) {
        assert(a.size() == g.size());
        for (int i = 2; i <= h.n; ++i) {
            int u = h.rev[i];
```

```
            pre[u] = pre[h.father[u]] + a[u];
        }
    }

    // 返回 u 到 v 的边权和；u==v 时返回 0，时间 O(log n)。
    i64 query(int u, int v) {
        assert(1 <= u && u <= h.n && 1 <= v && v <= h.n);
        int p = h.lca(u, v);
        return pre[u] + pre[v] - 2 * pre[p];
    }
};
```

调用与说明：TreeEdgePrefix tree(g,a,root); cout << tree.query(u,v) << endl; 输入边 (u,v,w) 可先用 HLD h(g,root) 确定父子关系，再令 a[h.dep[u]>h.dep[v]?u:v]=w。对于根为 1 的链 1—2—3，边权依次为 5、-2，a={0,0,5,-2}，query(1,3)=3，query(2,2)=0。

3.9 树上差分

3.9.1 点差分（TreePointDiff）

点差分与边差分分别提供独立类和独立代码文件，统一使用 add(u,v,val) 和 result()。两版复用手册 3.7 的 HLD 求 LCA，独立文件均已展开依赖。适合批量路径加后统一取值，不适合每次修改后立即高效查询。

用途与条件：输入非空连通无向树 g[1..n]，默认根为 1。批量给路径上的点加值，最后统一还原各点累计增量；包含路径两端和 LCA。

复杂度：借助树链剖分求 LCA：预处理 O(n)，单次路径加 O(log n)，还原 O(n)，空间 O(n)。

易错点：令 p=lca(u,v)，两端各加 val，p 和 p 的父亲各减 val；p 为根时不减父亲。现有 HLD 将根的父亲设为根，必须特别判断。所有中间结果需在 i64 范围内；若有初始权值，在结果上另行相加。

依赖：查看 3.7「树链剖分、LCA 与子树区间」（第 32 页）；完整代码已展开依赖，可单独使用。

```
// 依赖手册 3.7 的 HLD（树链剖分、LCA 与子树区间），独立文件已展开。
class TreePointDiff {
private:
    HLD h;
    vector<i64> diff;

public:
    // 为非空连通无向树 g[1..n] 初始化点差分，根为 root，初始增量全为 0。
    // 要求 1<=root<=n；使用迭代预处理，时间和空间 O(n)。
    TreePointDiff(const vector<vector<int>>& g, int root = 1) : h(g, root), diff(g.size(), 0)
    {}

    // 将 u 到 v 路径上的所有点加 val，包含两端；时间 O(log n)。
    void add(int u, int v, i64 val) {
        assert(1 <= u && u <= h.n && 1 <= v && v <= h.n);
        int p = h.lca(u, v);
        diff[u] += val;
        diff[v] += val;
        diff[p] -= val;
        if (h.father[p] != p) {
            diff[h.father[p]] -= val;
        }
    }

    // 返回各点累计增量 ans[1..n]，ans[0]=0；时间和空间 O(n)。
    // 按 dfs 序逆序向父亲累加，不修改 diff，可重复调用并继续添加操作。
    vector<i64> result() const {
        vector<i64> ans = diff;
        for (int i = h.n; i > 1; --i) {
            int u = h.rev[i];
            ans[h.father[u]] += ans[u];
        }
        return ans;
    }
```

```
};
```

调用与说明：TreePointDiff tree(g); tree.add(u,v,3); auto ans=tree.result(); ans[x] 为点 x 的累计增量。u==v 时仅该点增加。

3.9.2 边差分（TreeEdgeDiff）

用途与条件：输入非空连通无向树 g[1..n]，默认根为 1。批量给路径上的边加值，结果 ans[u] 对应父节点到 u 的边；根没有父边，其结果为 0。

复杂度：预处理 O(n)，单次路径加 O(log n)，还原 O(n)，空间 O(n)。

易错点：令 p=lca(u,v)，两端各加 val，p 减两次 val。u==v 时不修改任何边。边的结果放在较深端点，按输入边 (u,v) 输出时选择深度较大的端点。所有中间结果需在 i64 范围内；初始边权在结果上另行相加。

依赖：查看 3.7「树链剖分、LCA 与子树区间」（第 32 页）；完整代码已展开依赖，可单独使用。

```
// 依赖手册 3.7 的 HLD（树链剖分、LCA 与子树区间），独立文件已展开。
class TreeEdgeDiff {
private:
    HLD h;
    vector<i64> diff;

public:
    // 为非空连通无向树 g[1..n] 初始化边差分，根为 root，初始增量全为 0。
    // 要求 1<=root<=n；使用迭代预处理，时间和空间 O(n)。
    TreeEdgeDiff(const vector<vector<int>>& g, int root = 1) : h(g, root), diff(g.size(), 0) {
    }

    // 将 u 到 v 路径上的所有边加 val；u==v 时无边被修改，时间 O(log n)。
    void add(int u, int v, i64 val) {
        assert(1 <= u && u <= h.n && 1 <= v && v <= h.n);
        int p = h.lca(u, v);
        diff[u] += val;
        diff[v] += val;
        diff[p] -= val;
        diff[p] -= val;
    }

    // 返回各边累计增量，ans[u] 对应父节点到 u 的边，根及 ans[0] 为 0。
    // 逆序向父亲累加，不修改 diff，可重复调用并继续更新；时间和空间 O(n)。
    vector<i64> result() const {
        vector<i64> ans = diff;
        for (int i = h.n; i > 1; --i) {
            int u = h.rev[i];
            ans[h.father[u]] += ans[u];
        }
        ans[h.rev[1]] = 0;
        return ans;
    }
};
```

调用与说明：TreeEdgeDiff tree(g); tree.add(u,v,2); auto ans=tree.result(); ans[x] 为父节点到 x 的边的累计增量。对于根为 1 的链 1—2—3，add(1,3,2) 的结果为 {0,0,2,2}。

3.10 虚树：关键点及其 LCA

用途与条件：依赖 HLD。将关键点按 dfs 序排列，加入相邻点 LCA 后建虚树。返回 {父节点,子节点} 边，编号仍是原树编号。 keys、nodes 和 edges 是无占位的列表（容器位置从 0 开始）；其中保存的原树顶点编号为 1..n。

复杂度：k 个关键点：O(k log k+k log n)，空间 O(k)；虚树节点不超过 2k-1。

易错点：虚边权可以取深度差或根距离差。只有一个关键点时边集为空但节点仍存在。查询间不要 O(n) 清空全部原树数组。

依赖：查看 3.7「树链剖分、LCA 与子树区间」（第 32 页）；完整代码已展开依赖，可单独使用。

```
// Dependency: HLD.
// 根据已预处理的 HLD 和关键点列表 v 构造虚树，自动补充必要 LCA 并去重。
// 返回 {节点列表,父到子的虚边列表}，均保存原树的 1 下标编号；空关键点返回空结果。
```

```
// 列表不带占位，非空时首节点为虚树根；k 个关键点耗时 O(k log k+k log n)。
// LCA（Lowest Common Ancestor）：最近公共祖先；补齐相邻关键点的 LCA 以连接虚树。
pair<vector<int>, vector<pair<int, int>>> virtual_tree(HLD& h, vector<int> v) {
    if (v.empty()) return {{}, {}};
    // 按原树 dfs 序比较顶点 a、b，保证关键点和补入的 LCA 按树上遍历顺序排列。
    auto cmp = [&](int a, int b) { return h.pos[a] < h.pos[b]; };
    sort(v.begin(), v.end(), cmp);
    v.erase(unique(v.begin(), v.end()), v.end());
    int k = v.size();
    for (int i = 1; i < k; ++i) {
        v.push_back(h.lca(v[i - 1], v[i]));
    }
    sort(v.begin(), v.end(), cmp);
    v.erase(unique(v.begin(), v.end()), v.end());
    // st：当前虚树构造路径上的祖先栈，按 DFS 序处理节点。
    vector<int> st;
    vector<pair<int, int>> edges;
    for (int u : v) {
        while (!st.empty() &&
               !(h.pos[st.back()] <= h.pos[u] && h.pos[u] < h.pos[st.back()] + h.sz[st.back()]
            )) {
            st.pop_back();
        }
        if (!st.empty()) {
            edges.push_back({st.back(), u});
        }
        st.push_back(u);
    }
    return {v, edges};
}
```

调用与说明：auto [nodes,edges]=virtual_tree(h,keys)；nodes.front() 是非空虚树的根。

3.11 二分图最大匹配：匈牙利算法（DFS 增广路）

用途与条件：使用 DFS 寻找增广路求无权二分图最大匹配，也称 Kuhn 算法，国内竞赛中通常称为匈牙利算法。左部 nL 个点、右部 nR 个点；g[u] 只保存从左到右的边。适合中等规模二分图。左右两侧各自从 1 开始编号，g 分配 nL+1 行，第 0 行不使用。

复杂度：时间 O(nL • m)，空间 O(nL+nR+m)。大图优先考虑 Hopcroft-Karp 或 Dinic。

易错点：左右两侧独立编号。每次寻找增广路都需要新的访问标记。最大匹配不是一般图匹配，也不是最大权匹配。

```
// Kuhn 算法（以 Harold W. Kuhn 命名）：国内竞赛中通常称为匈牙利算法。
// 使用 DFS（Depth-First Search，深度优先搜索）寻找增广路，求无权二分图最大匹配。
struct BipartiteMatch {
    int nL, nR, timer = 0;
    vector<vector<int>> g;
    // mr[v]：右侧顶点 v 匹配的左侧顶点，-1 表示未匹配。
    // seen[v]：右侧顶点最近被搜索的轮次；timer：当前搜索轮次。
    // nL、nR：左右两侧的顶点数量，各自从 1 开始编号。
    vector<int> mr, seen;
    // 初始化左部 l 个点、右部 r 个点的二分图，两侧各从 1 编号；mr 初值为 -1。
    BipartiteMatch(int l, int r) : nL(l), nR(r), g(l + 1), mr(r + 1, -1), seen(r + 1) {}
    // 内部增广搜索：从左点 u 尝试找到可用右点或重排旧匹配，成功返回 true 并更新 mr。
    // 使用当前 timer 防止本轮重复访问右点，由 solve 为每轮设置新时间戳。
    bool dfs(int u) {
        for (int v : g[u]) {
            if (seen[v] != timer) {
                seen[v] = timer;
                if (mr[v] == -1 || dfs(mr[v])) {
                    mr[v] = u;
                    return true;
                }
```

```
            }
        }
        return false;
    }
    // 重置并求最大匹配，返回匹配边数，mr[v] 保存右点 v 匹配的左点，-1 表示未匹配。
    // 调用前把左到右的边放入 g；时间 O(nL*m)。
    int solve() {
        fill(mr.begin(), mr.end(), -1);
        int ans = 0;
        for (int u = 1; u <= nL; ++u) {
            ++timer;
            ans += dfs(u);
        }
        return ans;
    }
};
```

调用与说明：BipartiteMatch b(nL,nR); b.g[u].push_back(v); b.solve(); mr[v] 为匹配到的左点，-1 表示未匹配。

3.12 Dinic 最大流与最小割

用途与条件：有向非负整数容量网络。add_edge 自动建立反向残量边。结束后从 s 沿正残量边可达的点构成最小割源侧。 顶点编号从 1 开始；传入邻接表时分配 n+1 行，第 0 行不使用。

复杂度：一般图时间 $O(n^2m)$，空间 O(n+m)。

易错点：s 与 t 必须不同。无向容量边按建模需要添加双向有容量的边。本实现修改残量网络；再次调用返回新增流量。递归深度可达 n。

```
const i64 INF = (1LL << 62);

struct Dinic {
    struct E {
        // E（edge）：残量边；v：终点；rev：反向边在 g[v] 中的下标。
        int v, rev;
        // cap（capacity）：当前剩余容量，增广时会改变。
        i64 cap;
    };
    int n;
    vector<vector<E>> g;
    // level[u]：层次图中的层数，-1 表示不可达；it[u]：当前弧的邻接表下标。
    vector<int> level, it;
    // 创建 n 个顶点的空残量网络及层次、当前弧数组，顶点编号为 1..n。
    Dinic(int n) : n(n), g(n + 1), level(n + 1), it(n + 1) {}
    // 添加容量为 c 的有向边 u→v 和容量为 0 的反向残量边；c>=0，自环直接忽略。
    void add_edge(int u, int v, i64 c) {
        if (u == v) return;
        assert(c >= 0);
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({v, b, c});
        g[v].push_back({u, a, 0});
    }
    // 内部建层：沿正残量边从 s 广搜，更新 level，返回汇点 t 是否可达；时间 O(n+m)。
    // BFS（Breadth-First Search）：广度优先搜索，用于构造层次图。
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto e : g[u]) {
                if (e.cap > 0 && level[e.v] < 0) {
```

```
                    level[e.v] = level[u] + 1;
                    q.push(e.v);
                }
            }
        }
        return level[t] >= 0;
    }
    // 在层次图中从 u 向汇点 t 尝试推送至多 f 流量，返回实际流量并更新正反残量。
    // 使用当前弧 it 跳过已尝试边；要求先 bfs 建层并初始化 it。
    // DFS（Depth-First Search）：深度优先搜索，用于沿层次图增广。
    i64 dfs(int u, int t, i64 f) {
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); ++i) {
            E& e = g[u][i];
            if (e.cap > 0 && level[e.v] == level[u] + 1) {
                i64 z = dfs(e.v, t, min(f, e.cap));
                if (z) {
                    e.cap -= z;
                    g[e.v][e.rev].cap += z;
                    return z;
                }
            }
        }
        return 0;
    }
    // 在当前残量网络上增加 s→t 流，返回本次新增流量，最多 limit；要求 s!=t、limit>=0。
    // 会修改残量网络，重复调用只返回新增部分；未被 limit 截断时可用残量可达点求最小割。
    i64 flow(int s, int t, i64 limit = INF) {
        assert(s != t);
        i64 ans = 0;
        while (ans < limit && bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (i64 f = dfs(s, t, limit - ans)) {
                ans += f;
                if (ans == limit) break;
            }
        }
        return ans;
    }
};
```

调用与说明：Dinic d(n); d.add_edge(u,v,c); i64 ans=d.flow(s,t); 默认最多推 INF，若流可能更大应另行调整数值设计。

3.13 最小费用流：逐次最短增广路

用途与条件：允许负费用边，但初始网络不得有正容量负费用环。每轮使用 Bellman-Ford，适合作为容易检查的中小规模版本。 顶点编号从 1 开始；传入邻接表时分配 n+1 行，第 0 行不使用。

复杂度：若增广 A 次，时间 O(A·n·m)，空间 O(n+m)。大规模使用势能+Dijkstra 版本。

易错点：容量非负，s≠t。返回 {流量,费用}；费用使用 i128。调用修改残量图。模型含负费用环时本算法不适用，不会自动完成消圈。

```
struct MinCostFlow {
    struct E {
        // E（edge）：残量边；v：终点；rev：反向边在 g[v] 中的下标。
        int v, rev;
        // cap（capacity）：剩余容量；cost：单位流量费用，反向边费用取反。
        i64 cap, cost;
    };
    int n;
    vector<vector<E>> g;
    // 创建 n 个顶点的空费用残量网络，顶点编号为 1..n。
    MinCostFlow(int n) : n(n), g(n + 1) {}
```

```
    // 添加容量 cap、单位费用 cost 的有向边，并建立费用取反的残量边。
    // 要求端点有效且不同、cap>=0、cost!=LLONG_MIN；允许负费用。
    void add_edge(int u, int v, i64 cap, i64 cost) {
        assert(u != v && cap >= 0 && cost != LLONG_MIN);
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({v, b, cap, cost});
        g[v].push_back({u, a, 0, -cost});
    }
    // 从 s 向 t 增广至多 limit 单位流，返回 {本次流量,本次总费用}，并修改残量网络。
    // 要求 s!=t、limit>=0，初始网络不能有正容量负费用环；精确需求必须检查流量等于 limit。
    // 每轮用 Bellman-Ford，若增广 A 次，时间 O(A*n*m)，费用用 i128。
    pair<i64, i128> flow(int s, int t, i64 limit) {
        assert(s != t);
        i64 f = 0;
        i128 cost = 0, INF = i128(1) << 120;
        while (f < limit) {
            vector<i128> dist(n + 1, INF);
            // pv[v]：增广路径上 v 的前驱顶点，-1 表示尚未记录。
            // pe[v]：前驱边在 g[pv[v]] 中的下标，仅在 pv[v] 有效时使用。
            vector<int> pv(n + 1, -1), pe(n + 1, -1);
            dist[s] = 0;
            for (int z = 0; z < n - 1; ++z) {
                bool changed = false;
                for (int u = 1; u <= n; ++u) {
                    if (dist[u] != INF) {
                        for (int i = 0; i < (int)g[u].size(); ++i) {
                            auto e = g[u][i];
                            if (e.cap > 0 && dist[e.v] > dist[u] + e.cost) {
                                dist[e.v] = dist[u] + e.cost;
                                pv[e.v] = u;
                                pe[e.v] = i;
                                changed = true;
                            }
                        }
                    }
                }
                if (!changed) break;
            }
            if (pv[t] == -1) break;
            i64 add = limit - f;
            for (int v = t; v != s; v = pv[v]) {
                add = min(add, g[pv[v]][pe[v]].cap);
            }
            for (int v = t; v != s; v = pv[v]) {
                E& e = g[pv[v]][pe[v]];
                // 正向剩余容量减少，反向剩余容量增加，使后续增广可以撤回已有流量。
                e.cap -= add;
                g[v][e.rev].cap += add;
            }
            f += add;
            cost += i128(add) * dist[t];
        }
        return {f, cost};
    }
};
```

调用与说明：auto [f,c]=net.flow(s,t,need)；需要恰好 need 单位流时必须检查 f==need。最大费用问题可在无负环条件满足时对费用取反。

4. 字符串

4.1 前缀函数与 KMP 匹配

用途与条件：pi[i] 为 s[0..i] 的最长真前后缀长度；匹配函数返回模式串在文本中的所有起点，允许重叠。字符串及其位置数组保留 0 下标；自动机状态编号也从 0 开始。

复杂度：时间 O(n+m)，空间 O(m+匹配数)。

易错点：匹配成功后跳到 pi[j-1]，不能清零，否则会漏重叠匹配。本实现约定空模式匹配文本的全部 n+1 个边界。

```
// 返回前缀函数 pi，pi[i] 是 s[0..i] 的最长真前缀与后缀的公共长度。
// 字符串和返回数组均为 0 下标，空串返回空数组；时间和空间 O(|s|)。
// KMP（Knuth-Morris-Pratt）：字符串匹配算法，以三位提出者的姓氏命名。
vector<int> prefix_function(const string& s) {
    // pi[i]：s[0..i] 的最长相等真前缀与真后缀的长度，不包含整个串自身。
    vector<int> pi(s.size());
    for (int i = 1; i < (int)s.size(); ++i) {
        int j = pi[i - 1];
        while (j && s[i] != s[j]) {
            j = pi[j - 1];
        }
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}
// 返回模式串 p 在 text 中所有匹配起点，使用 0 下标，允许重叠匹配。
// 空模式返回 0..text.size() 的全部边界；时间 O(|text|+|p|)，不修改输入。
vector<int> kmp(const string& text, const string& p) {
    vector<int> ans;
    if (p.empty()) {
        ans.resize(text.size() + 1);
        iota(ans.begin(), ans.end(), 0);
        return ans;
    }
    auto pi = prefix_function(p);
    int j = 0;
    for (int i = 0; i < (int)text.size(); ++i) {
        while (j && text[i] != p[j]) {
            j = pi[j - 1];
        }
        if (text[i] == p[j]) ++j;
        if (j == (int)p.size()) {
            ans.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return ans;
}
```

调用与说明：kmp("ababa","aba") 返回 {0,2}。若 n%(n-pi[n-1])==0，则非空串最短循环节长度为 n-pi[n-1]。

4.2 Z 函数

用途与条件：z[i] 为 s 与 s.substr(i) 的最长公共前缀长度，本实现 z[0]=n。 字符串及其位置数组保留 0 下标；自动机状态编号也从 0 开始。

复杂度：时间 O(n)，空间 O(n)。

易错点：维护匹配区间 [l,r)。用 p+分隔符+t 匹配时，分隔符必须不在原字符集中。

```
// 返回 z[i]=s 与后缀 s[i..] 的最长公共前缀长度，位置从 0 开始。
// 约定非空串 z[0]=|s|，空串返回空数组；时间和空间 O(|s|)。
vector<int> z_function(const string& s) {
    int n = s.size();
```

```
    // z[i]：s 与从 i 开始的后缀的最长公共前缀长度，本实现规定 z[0]=n。
    vector<int> z(n);
    if (n) z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            ++z[i];
        }
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

调用与说明：z_function("aaaa") 返回 {4,3,2,1}。

4.3 Manacher：奇偶回文半径

用途与条件：d1[i] 为以 i 为中心的奇回文半径，长度 2d1[i]-1；d2[i] 为以 i-1 与 i 间隙为中心的偶回文半径，长度 2d2[i]。 字符串及其位置数组保留 0 下标；自动机状态编号也从 0 开始。

复杂度：时间 O(n)，空间 O(n)。

易错点：无需插入特殊字符，避免分隔符冲突。回文子串出现次数为 sum(d1)+sum(d2)，不等于不同回文串数量。

```
// 返回 {奇回文半径 d1,偶回文半径 d2}，均为 0 下标。
// d1[i] 含中心 i，回文长度为 2*d1[i]-1；d2[i] 的中心在 i-1 与 i 之间，长度为 2*d2[i]。
// 空串返回两个空数组；时间和空间 O(|s|)。
pair<vector<int>, vector<int>> manacher(const string& s) {
    int n = s.size();
    // d1[i]：以 i 为中心的奇回文半径，包含中心。
    // d2[i]：以 i-1 与 i 之间为中心的偶回文半径，不含虚拟中心。
    vector<int> d1(n), d2(n);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = i > r ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) {
            ++k;
        }
        d1[i] = k--;
        if (i + k > r) {
            l = i - k;
            r = i + k;
        }
    }
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) {
            ++k;
        }
        d2[i] = k--;
        if (i + k > r) {
            l = i - k - 1;
            r = i + k;
        }
    }
    return {d1, d2};
}
```

调用与说明："abba" 的 d2[2]=2，因此最长偶回文长度为 4。

4.4 双模字符串哈希

用途与条件：预处理后 O(1) 得到子串 [l,r) 哈希。相同参数构造的对象可跨串比较。 字符串及其位置数组保留 0 下标；自动机状态编号也从 0 开始。

复杂度：预处理时间与空间 O(n)，单次查询 O(1)。

易错点：双哈希仍可能碰撞，不提供严格正确性保证。需要确定性时用 KMP、后缀数组等算法；比较时同时检查长度。

```
struct StringHash {
    static constexpr i64 M1 = 1000000007, M2 = 1000000009, B = 911382323;
    // pre1[i]、pre2[i]：前 i 个字符在两组模数下的哈希，初值均为 0。
    // p1[i]、p2[i]：对应模数下底数的 i 次幂。
    vector<i64> pre1, pre2, p1, p2;
    // 预处理字符串 s 的双模前缀哈希和幂表，字符串位置为 0 下标；时间和空间 O(|s|)。
    StringHash(const string& s)
        : pre1(s.size() + 1), pre2(s.size() + 1), p1(s.size() + 1, 1), p2(s.size() + 1, 1) {
        for (int i = 0; i < (int)s.size(); ++i) {
            int x = (unsigned char)s[i] + 1;
            pre1[i + 1] = (pre1[i] * B + x) % M1;
            pre2[i + 1] = (pre2[i] * B + x) % M2;
            p1[i + 1] = p1[i] * B % M1;
            p2[i + 1] = p2[i] * B % M2;
        }
    }
    // 返回子串 s[l..r) 的两个模数下哈希，要求 0<=l<=r<=|s|；时间 O(1)。
    // 比较子串时需同时比较长度；即使双哈希相等，也仍存在碰撞可能。
    pair<i64, i64> get(int l, int r) const {
        return {(pre1[r] - pre1[l] * p1[r - l] % M1 + M1) % M1,
                (pre2[r] - pre2[l] * p2[r - l] % M2 + M2) % M2};
    }
};
```

调用与说明：StringHash h("abcabc"); h.get(0,3)==h.get(3,6)。

4.5 普通字典树

用途与条件：维护小写字母 a-z 组成的字符串，支持插入、查询完整单词次数和前缀次数，允许重复插入。字符串使用 0 下标；节点从 1 开始，根节点为 1。

复杂度：插入、单词查询和前缀查询均为 O(|s|)；总空间 O(26(L+1))，L 为插入字符串的总长度。

易错点：end 统计完整单词，cnt 统计前缀，均包含重复串。支持空串：count(空串) 返回空串插入次数，prefix_count(空串) 返回全部字符串数量。仅接受 a-z；计数使用 int，插入总数不能超过其范围。

```
// Trie（名称源自 retrieval）：字典树。
class Trie {
private:
    struct Node {
        int ch[26] = {};  // 各字符对应的子节点，0 表示不存在
        int cnt = 0;      // 以当前路径为前缀的字符串数量，包含重复串
        int end = 0;      // 在当前节点结束的字符串数量
    };

    vector<Node> tr;  // tr[0] 占位，tr[1] 为根节点

    // 查找字符串 s 对应的节点；路径不存在时返回 0。
    int find(const string &s) const {
        int u = 1;
        for (char c : s) {
            assert('a' <= c && c <= 'z');
            int v = c - 'a';
            if (tr[u].ch[v] == 0) return 0;
            u = tr[u].ch[v];
        }
        return u;
    }
```

```
public:
    // 建立空字典树。
    Trie() : tr(2) {}

    // 清空所有字符串，恢复根节点。
    void clear() {
        tr.assign(2, Node{});
    }

    // 插入字符串 s，允许重复插入，时间复杂度 O(|s|)。
    void insert(const string &s) {
        int u = 1;
        tr[u].cnt++;
        for (char c : s) {
            assert('a' <= c && c <= 'z');
            int v = c - 'a';
            if (tr[u].ch[v] == 0) {
                int id = (int)tr.size();
                tr.emplace_back();
                tr[u].ch[v] = id;
            }
            u = tr[u].ch[v];
            tr[u].cnt++;
        }
        tr[u].end++;
    }

    // 返回完整字符串 s 的出现次数，不存在时返回 0。
    // 时间复杂度 O(|s|)。
    int count(const string &s) const {
        int u = find(s);
        return u == 0 ? 0 : tr[u].end;
    }

    // 返回以 s 为前缀的字符串数量，包含重复串。
    // 时间复杂度 O(|s|)；空前缀返回所有字符串的数量。
    int prefix_count(const string &s) const {
        int u = find(s);
        return u == 0 ? 0 : tr[u].cnt;
    }
};
```

调用与说明：Trie trie; 依次 insert("apple")、insert("apple")、insert("app") 后，count("apple")==2、count("ap")==0、prefix_count("app")==3；clear() 清空后可重新插入。

4.6 AC 自动机：多模式出现次数

用途与条件：小写字母 a-z，插入所有非空模式后 build，再统计各模式在文本中的出现次数。重复模式允许共用终止节点。 字符串及其位置数组保留 0 下标；自动机状态编号也从 0 开始。

复杂度：设节点数 S、文本长 N、字符集大小 26：构建 O(26S)，匹配与反向累加 O(N+S)，空间 O(26S)。

易错点：build 补全转移后不能再 insert。count 返回按节点编号的出现次数，模式答案读其终止节点。不要逐字符沿 fail 链枚举全部后缀。

```
// AC（Aho-Corasick）：多模式字符串匹配自动机，以两位提出者的姓氏命名。
// BFS（Breadth-First Search）：广度优先搜索，用于按层构造失配链接。
struct Aho {
    struct Node {
        // go[c]：字符 c 对应的转移，构建前 0 表示无子节点，构建后补齐自动机转移。
        array<int, 26> go{};
        // fail：失配链接，指向当前串最长的、仍为某个模式前缀的真后缀节点。
        int fail = 0;
    };
```

```
    vector<Node> t{{}};
    // order：不含根的 BFS 顺序；逆序汇总时将出现次数传给 fail 指向的节点。
    vector<int> order;
    // 插入非空小写模式串 s，返回其终止节点编号；重复模式可返回同一节点。
    // 只能在 build 前调用，时间 O(|s|)。
    int insert(const string& s) {
        assert(!s.empty());
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            assert(0 <= c && c < 26);
            if (!t[u].go[c]) {
                t[u].go[c] = t.size();
                t.push_back({});
            }
            u = t[u].go[c];
        }
        return u;
    }
    // 在全部模式插入后构造 fail 指针并补全字符转移，记录 BFS 顺序 order。
    // 本实现应只调用一次，之后不能继续 insert；S 个节点耗时 O(26S)。
    void build() {
        queue<int> q;
        for (int v : t[0].go) {
            if (v) q.push(v);
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            order.push_back(u);
            for (int c = 0; c < 26; ++c) {
                int v = t[u].go[c];
                if (v) {
                    t[v].fail = t[t[u].fail].go[c];
                    q.push(v);
                }
                else {
                    t[u].go[c] = t[t[u].fail].go[c];
                }
            }
        }
    }
    // 统计小写文本 s 中各自动机节点所代表模式的出现次数，允许重叠。
    // 需先 build；返回值按节点编号索引，模式答案取其 insert 返回的终止节点。
    // 时间 O(|s|+S)，不修改自动机，可以对不同文本重复调用。
    vector<i64> count(const string& s) const {
        vector<i64> cnt(t.size());
        int u = 0;
        for (char ch : s) {
            u = t[u].go[ch - 'a'];
            ++cnt[u];
        }
        for (int i = (int)order.size() - 1; i >= 0; --i) {
            int v = order[i];
            cnt[t[v].fail] += cnt[v];
        }
        return cnt;
    }
};
```

调用与说明：Aho ac; int p=ac.insert("aba"); ac.build(); ac.count("ababa")[p] 返回 2。

4.7 后缀数组与 LCP：倍增版

用途与条件：sa[i] 为第 i 小后缀的起点。lcp[i] 为 sa[i-1] 与 sa[i] 的 LCP，lcp[0]=0。支持任意字节串。 字符串及其位置数组保留 0 下标；自动机状态编号也从 0 开始。

复杂度：使用比较排序的倍增：O(n log² n) 时间；Kasai 求 LCP 为 O(n)；空间 O(n)。

易错点：超大字符串可改计数排序倍增或 SA-IS。不同子串数 = n(n+1)/2 - sum(lcp)。任意两后缀 LCP 可对 lcp 建 RMQ。

```
// SA（Suffix Array）：后缀数组。
// LCP（Longest Common Prefix）：最长公共前缀。
// RMQ（Range Minimum Query）：区间最小值查询，可用于查询任意两后缀的 LCP。
// SA-IS（Suffix Array Induced Sorting）：后缀数组诱导排序，是说明中提到的另一种实现。
struct SuffixArray {
    // sa[i]：字典序排名为 i 的后缀起点；rk[pos]：起点为 pos 的后缀排名，均从 0 开始。
    // lcp[i]：sa[i-1] 与 sa[i] 两个后缀的最长公共前缀长度，lcp[0]=0。
    vector<int> sa, rk, lcp;
    // 构造字符串 s 的后缀排序 sa、逆排名 rk 和相邻后缀公共前缀长度 lcp。
    // 均用 0 下标，lcp[0]=0，支持空串；比较排序倍增耗时 O(n log²n)，空间 O(n)。
    SuffixArray(const string& s) {
        int n = s.size();
        sa.resize(n);
        rk.resize(n);
        lcp.assign(n, 0);
        iota(sa.begin(), sa.end(), 0);
        for (int i = 0; i < n; ++i) rk[i] = (unsigned char)s[i];
        // tmp：本轮倍增的新排名，避免计算中覆盖上一轮 rk。
        vector<int> tmp(n);
        for (int k = 1; k < n; k *= 2) {
            // 读取当前轮位置 i 的排名，超过字符串末尾时返回 -1，作为缺失后半段的最小排名。
            auto val = [&](int i) { return i < n ? rk[i] : -1; };
            // 比较后缀 a、b 的前后两段排名，每段长 k；供当前倍增轮排序与判断新排名。
            auto cmp = [&](int a, int b) {
                return pair{rk[a], val(a + k)} < pair{rk[b], val(b + k)};
            };
            sort(sa.begin(), sa.end(), cmp);
            tmp[sa[0]] = 0;
            for (int i = 1; i < n; ++i) {
                tmp[sa[i]] = tmp[sa[i - 1]] + cmp(sa[i - 1], sa[i]);
            }
            rk = tmp;
            if (rk[sa.back()] == n - 1) break;
        }
        for (int i = 0; i < n; ++i) rk[sa[i]] = i;
        for (int i = 0, h = 0; i < n; ++i) {
            if (rk[i] == 0) {
                h = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (i + h < n && j + h < n && s[i + h] == s[j + h]) {
                ++h;
            }
            lcp[rk[i]] = h;
            if (h) --h;
        }
    }
};
```

调用与说明：SuffixArray a("banana"); a.sa 为 {5,3,1,0,4,2}。

4.8 后缀自动机：不同子串与出现次数

用途与条件：小写字母 a-z。每个状态代表长度区间 (len[link[v]],len[v]]。支持在线 extend，构建结束后统计 endpos 大小。 字符串及其位置数组保留 0 下标；自动机状态编号也从 0 开始。

复杂度：固定字符集下构建 O(n)，空间 O(26n)；本版出现次数按长度排序，O(n log n)。
易错点：克隆状态自身贡献次数为 0。occurrences 使用副本累加，可以重复调用。不要把状态数误当不同子串数。

```
// SAM（Suffix Automaton）：后缀自动机。
struct SAM {
    struct Node {
        // go[c]：字符 c 对应的转移状态，-1 表示没有转移。
        array<int, 26> go;
        // link：后缀链接，根为 -1；len：该状态代表的最大子串长度。
        int link = -1, len = 0;
        // cnt：构建时记录的末端贡献，新建普通状态为 1、克隆为 0。
        // 实际出现次数由 occurrences 按后缀链接汇总到副本，不能直接读取此字段。
        i64 cnt = 0;
        // 创建自动机状态：所有字符转移设为 -1，其他字段使用默认初值。
        Node() {
            go.fill(-1);
        }
    };
    vector<Node> t{Node()};
    // last：当前整个字符串对应的状态；0 号为根状态。
    int last = 0;
    // 在当前字符串末尾追加一个小写字母 ch，更新自动机、后缀链接和 last。
    // 必要时克隆状态，克隆自身出现次数为 0；固定字符集下整个构建过程为线性时间。
    void extend(char ch) {
        int c = ch - 'a', cur = t.size();
        assert(0 <= c && c < 26);
        t.emplace_back();
        t[cur].len = t[last].len + 1;
        t[cur].cnt = 1;
        int p = last;
        while (p != -1 && t[p].go[c] == -1) {
            t[p].go[c] = cur;
            p = t[p].link;
        }
        if (p == -1) {
            t[cur].link = 0;
        }
        else {
            int q = t[p].go[c];
            if (t[p].len + 1 == t[q].len) {
                t[cur].link = q;
            }
            else {
                int clone = t.size();
                t.push_back(t[q]);
                t[clone].len = t[p].len + 1;
                t[clone].cnt = 0;
                while (p != -1 && t[p].go[c] == q) {
                    t[p].go[c] = clone;
                    p = t[p].link;
                }
                t[q].link = t[cur].link = clone;
            }
        }
        last = cur;
    }
    // 返回当前字符串不同非空子串的数量，累加各非根状态代表的长度区间；时间 O(状态数)。
    i64 distinct() const {
        i64 ans = 0;
        for (int v = 1; v < (int)t.size(); ++v) {
```

```
            ans += t[v].len - t[t[v].link].len;
        }
        return ans;
    }
    // 返回每个状态的 endpos 大小，即其代表的非空子串在当前字符串中的出现次数。
    // 用副本按长度降序累加，可重复调用；根状态的值不作为空串出现次数使用。
    // S 个状态耗时 O(S log S)，额外空间 O(S)。
    vector<i64> occurrences() const {
        // ord：状态按 len 从大到小排列的顺序，让较长状态先向后缀链接贡献次数。
        vector<int> ord(t.size());
        iota(ord.begin(), ord.end(), 0);
        // 按状态最大长度降序排序，使出现次数先从较长状态传给较短的后缀链接目标。
        sort(ord.begin(), ord.end(), [&](int a, int b) { return t[a].len > t[b].len; });
        vector<i64> cnt;
        for (auto& v : t) cnt.push_back(v.cnt);
        for (int v : ord) {
            if (t[v].link != -1) {
                cnt[t[v].link] += cnt[v];
            }
        }
        return cnt;
    }
};
```

调用与说明：SAM s; for(char c:string("ababa")) s.extend(c); s.distinct()==9。查询模式时从根沿 go 走，失败为不存在，否则读取 cnt[state]。

4.9 最小表示法

用途与条件：返回字符串循环移位中字典序最小者的起点；空串返回 0。 字符串及其位置数组保留 0 下标；自动机状态编号也从 0 开始。

复杂度：时间 O(n)，空间 O(n)（复制双倍串）。

易错点：多个相同最小表示会返回其中最靠前的候选。用于环结构判等时还需比较长度。

```
// 返回字典序最小循环移位的 0 下标起点；多个最优起点取最小者，空串返回 0。
// 返回位置而非字符串；时间 O(n)，复制双倍串使用 O(n) 空间。
int min_rotation(const string& s) {
    int n = s.size();
    if (!n) return 0;
    string t = s + s;
    int i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        if (t[i + k] == t[j + k]) {
            ++k;
            continue;
        }
        if (t[i + k] > t[j + k]) {
            i += k + 1;
            if (i == j) ++i;
        }
        else {
            j += k + 1;
            if (i == j) ++j;
        }
        k = 0;
    }
    return min(i, j);
}
```

调用与说明：min_rotation("baca") 返回 3，最小循环串为 "abac"。

5. 数学

5.1 快速幂

用途与条件：计算 a^b mod mod；底数 a 使用 i64，非负指数 b 使用 u64，模数要求 1≤mod≤LLONG_MAX。支持负底数和零指数。

复杂度：时间 O(log(b+1))，空间 O(1)。

易错点：mod 必须为正，不能传入负指数。乘法先转为 i128，防止取模前溢出。mod=1 时结果为 0；b=0 时返回 1%mod。费马逆元 qpow(a,p-2,p) 仅适用于素数 p 且 a%p!=0。

```
// 快速幂：计算 a 的 b 次方对 mod 取模，返回 [0,mod-1] 内的余数。
// a 可为负，b 为非负指数，要求 mod>0；b=0 返回 1%mod，mod=1 返回 0。
// 用 i128 进行乘法以避免中间溢出；时间 O(log(b+1))，额外空间 O(1)。
i64 qpow(i64 a, u64 b, i64 mod) {
    a %= mod;
    if (a < 0) a += mod;

    i64 res = 1 % mod;
    while (b) {
        if (b & 1) res = i128(res) * a % mod;
        a = i128(a) * a % mod;
        b >>= 1;
    }
    return res;
}
```

调用与说明：qpow(2,10,1000)==24，qpow(-2,3,7)==6，qpow(2,0,1)==0。

5.2 逆元

5.2.1 费马小定理求逆元

用途与条件：求满足 a*x%mod==1 的逆元。费马小定理法要求 mod 是素数，且 a%mod!=0。

复杂度：时间 O(log mod)，空间 O(1)。

易错点：mod 必须是素数且大于 1，本函数不检查素性。合数模数请使用下一小节的扩展欧几里得法。

依赖：查看 5.1「快速幂」（第 49 页）；完整代码已展开依赖，可单独使用。

```
// 依赖 qpow：见 5.1「快速幂」（代码/05_数学/5.1 快速幂.cpp）。
// 要求 mod 是素数，且 a % mod != 0。
// 用费马小定理返回 a 在模 mod 下的逆元，结果在 [0,mod-1]。
// 要求 mod 是大于 1 的素数且 a 不被 mod 整除；本函数不检查素性。
// 支持负 a，依赖 qpow；时间 O(log mod)，空间 O(1)。
i64 inverse_fermat(i64 a, i64 mod) {
    assert(mod > 1 && a % mod != 0);
    return qpow(a, mod - 2, mod);
}
```

调用与说明：inverse_fermat(3,11)==4，inverse_fermat(-3,11)==7。

5.2.2 扩展欧几里得求逆元

用途与条件：要求 m>1；逆元存在当且仅当 gcd(a,m)=1，m 可以是合数。exgcd 要求非负输入，求出满足 ax+by=gcd(a,b) 的 x、y；当最大公约数为 1 时，x 对模数取模即为逆元。

复杂度：求逆元时间和递归空间均为 O(log m)。

易错点：不互质时返回 nullopt，使用返回值前先判断是否存在。扩欧系数中间量使用 i128。

```
// 将整数 a 规范为模 m 的非负余数，返回 [0,m-1] 内的值；要求 m>0，时间 O(1)。
i64 norm(i64 a, i64 m) {
    a %= m;
    return a < 0 ? a + m : a;
}
// 扩展欧几里得：返回 gcd(a,b)，并通过引用写入满足 a*x+b*y=gcd(a,b) 的系数。
// 要求 a、b 非负；系数用 i128 保存，递归耗时与空间为对数级。
i64 exgcd(i64 a, i64 b, i128& x, i128& y) {
    if (!b) {
        x = 1;
```

```
        y = 0;
        return a;
    }
    i128 u, v;
    i64 g = exgcd(b, a % b, u, v);
    x = v;
    y = u - i128(a / b) * v;
    return g;
}
// 返回 a 在模 m 下的逆元，要求 m>1，支持合数模数和负 a。
// 若 gcd(a,m)!=1 则返回 nullopt，否则返回 [0,m-1] 内的逆元；时间 O(log m)。
optional<i64> inverse(i64 a, i64 m) {
    assert(m > 1);
    i128 x, y;
    if (exgcd(norm(a, m), m, x, y) != 1) return nullopt;
    x %= m;
    if (x < 0) x += m;
    return (i64)x;
}
```

调用与说明：inverse(3,11) 为 4，inverse(3,10) 为 7，inverse(6,9) 无值。例如 auto inv=inverse(3,10); if(inv) cout<<*inv;

5.3 线性筛：最小质因子、欧拉函数与莫比乌斯函数

用途与条件：一次筛出 1..n 的最小质因子 lp、φ 和 μ。φ(1)=μ(1)=1。 下标表示整数本身；phi[1]=mu[1]=1，lp[0] 等位置不代表普通数组元素。

复杂度：时间 O(n)，空间 O(n)。

易错点：乘法 i*p 用 i64 判断上界。μ 取值为 -1、0、1；常用于莫比乌斯反演。n 至少为 1。

```
struct Sieve {
    // primes：素数列表；lp（least prime factor）：最小质因子，lp[1]=0。
    // phi：欧拉函数值；mu：莫比乌斯函数值，phi[1]=mu[1]=1。
    vector<int> primes, lp, phi, mu;
    // 在线性时间内筛出 1..n 的素数、最小质因子 lp、欧拉函数 phi 和莫比乌斯函数 mu。
    // 要求 n>=1；phi[1]=mu[1]=1，lp[1]=0；时间和空间 O(n)。
    Sieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1) {
        assert(n >= 1);
        phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; ++i) {
            if (!lp[i]) {
                lp[i] = i;
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : primes) {
                if (1LL * i * p > n) break;
                int x = i * p;
                lp[x] = p;
                if (i % p == 0) {
                    phi[x] = phi[i] * p;
                    mu[x] = 0;
                    break;
                }
                phi[x] = phi[i] * (p - 1);
                mu[x] = -mu[i];
            }
        }
    }
    // 判断 x 是否为素数，要求 0<=x<=n；调用：s.is_prime(x)。
    bool is_prime(int x) const {
        assert(0 <= x && x < (int)lp.size());
```

```
        return x >= 2 && lp[x] == x;
    }
};
```

调用与说明：Sieve s(100); s.is_prime(7) 为 true，s.is_prime(12) 为 false；s.phi[12]==4，s.mu[12]==0。反复 x/=lp[x] 可以分解不超过 n 的正整数。

5.4 组合数

5.4.1 阶乘预处理求组合数

用途与条件：模数 p 为素数，预处理 0..n 的阶乘和逆阶乘，要求 n<p；单次查询 C(n,k)。
复杂度：预处理 O(n+log p)，查询 O(1)，空间 O(n)。
易错点：p 必须为素数，预处理上限 n<p；合数模数不可直接使用阶乘逆元。
依赖：查看 5.1「快速幂」（第 49 页）；完整代码已展开依赖，可单独使用。

```
// 依赖 qpow：见 5.1「快速幂」（代码/05_数学/5.1 快速幂.cpp），复制本节时一并复制。
struct Comb {
    i64 p;
    // fac[i]：i! 模 p；ifac[i]：i! 在模 p 下的乘法逆元。
    vector<i64> fac, ifac;
    // 预处理 0..n 的阶乘和逆阶乘，用于模素数 p 的组合数查询。
    // 要求 0<=n<p 且 p 是素数（不检查素性）；时间 O(n+log p)，空间 O(n)。
    Comb(int n, i64 p) : p(p), fac(n + 1, 1), ifac(n + 1, 1) {
        assert(n < p && p >= 2);
        for (int i = 1; i <= n; ++i) {
            fac[i] = i128(fac[i - 1]) * i % p;
        }
        ifac[n] = qpow(fac[n], p - 2, p);
        for (int i = n; i > 0; --i) {
            ifac[i - 1] = i128(ifac[i]) * i % p;
        }
    }
    // 返回 C(n,k)%p；k<0 或 k>n 返回 0，有效查询要求 0<=n<=预处理上限。
    // 使用阶乘和逆阶乘，单次时间 O(1)。
    i64 C(int n, int k) const {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return i128(fac[n]) * ifac[k] % p * ifac[n - k] % p;
    }
};
```

调用与说明：Comb c(1000000,1000000007); c.C(5,2)==10。

5.4.2 Lucas 定理

用途与条件：模数 p 为较小素数，用 p 进制逐位分解大参数组合数，依赖阶乘预处理。
复杂度：预处理 O(p+log p)，单次查询 O(log_p(max(n,k)+1))，空间 O(p)。
易错点：p 必须为素数且完整预处理到 p-1；大模数不适合本实现。
依赖：查看 5.4.1「阶乘预处理求组合数」（第 51 页）；完整代码已展开依赖，可单独使用。

```
struct Lucas : Comb {
    // 预处理完整的 0..p-1 阶乘表；p 必须是可承受 O(p) 时空开销的素数。
    Lucas(int p) : Comb(p - 1, p) {}
    // 用 Lucas 定理返回大参数 C(n,k)%p，要求构造时完整预处理到 p-1。
    // p 必须为素数；逐位处理 p 进制，时间 O(log_p(max(n,k)+1))，k>n 返回 0。
    i64 lucas(u64 n, u64 k) const {
        assert((u64)fac.size() == (u64)p);
        i64 r = 1;
        while (n || k) {
            r = i128(r) * C(n % p, k % p) % p;
            n /= p;
            k /= p;
        }
        return r;
```

```
    }
};
```

调用与说明：Lucas c(7); c.lucas(100,20);

5.5 扩展中国剩余定理

用途与条件：合并 x≡a(mod m)、x≡b(mod n)，模数可不互质，要求 m,n>0。依赖 modular 的 exgcd。
复杂度：单次合并 O(log min(m,n))；空间 O(log min(m,n))。
易错点：返回状态 0 表示无解，1 表示成功，2 表示合并模数超出 i64。不能把溢出误当无解。成功时 0≤余数<模数。
依赖：查看 5.2.2「扩展欧几里得求逆元」（第 49 页）；完整代码已展开依赖，可单独使用。

```
// 依赖 exgcd：见 5.2.2「扩展欧几里得求逆元」（代码/05_数学/5.2.2 扩展欧几里得求逆元.cpp）。
// CRT（Chinese Remainder Theorem）：中国剩余定理，此处支持非互质模数的扩展形式。
struct CRTResult {
    // status：0 无解、1 成功、2 合并模数溢出；只有成功时才使用 r、m。
    int status;
    // r：规范化余数；m：合并后的模数，成功时解为 x ≡ r (mod m)。
    i64 r, m;
};
// 合并 x≡a(mod m) 与 x≡b(mod n)，要求 m,n>0，允许模数不互质。
// 返回 {status,r,m}：0 表示无解，1 表示成功，2 表示合并模数超出 i64。
// 成功时解为 x≡r(mod 返回模数)，且 0<=r<返回模数；依赖 exgcd，耗时为对数级。
CRTResult crt_merge(i64 a, i64 m, i64 b, i64 n) {
    assert(m > 0 && n > 0);
    a %= m;
    if (a < 0) a += m;
    b %= n;
    if (b < 0) b += n;
    i128 x, y;
    i64 g = exgcd(m, n, x, y);
    i128 diff = i128(b) - a;
    if (diff % g) return {0, 0, 0};
    i128 mod = n / g;
    i128 t = (diff / g * x) % mod;
    if (t < 0) t += mod;
    i128 lcm = i128(m / g) * n;
    if (lcm > LLONG_MAX) return {2, 0, 0};
    i128 r = (i128(a) + i128(m) * t) % lcm;
    return {1, (i64)r, (i64)lcm};
}
```

调用与说明：crt_merge(2,6,5,9) 返回余数 14、模数 18。处理多组同余式可从 x≡0(mod 1) 开始逐个合并。

5.6 Miller–Rabin：64 位素性测试

用途与条件：判断 u64 范围整数是否为素数；n<2 返回 false，使用覆盖该范围的确定性底数。
复杂度：固定底数轮数，每轮 O(log n) 次模乘；额外空间 O(1)。
易错点：使用 __uint128_t 模乘，底数集合针对完整 u64 范围；n<2 返回 false，不适用于任意精度整数。

```
using u128 = __uint128_t;
// 返回 (a*b)%m，要求 m>0；使用 u128 保存乘积，支持完整 u64 范围，O(1)。
u64 mul(u64 a, u64 b, u64 m) { return u128(a) * b % m; }
// 完整 u64 模数专用快速幂；5.1「快速幂」的 mod 为 i64，不能覆盖这里的范围。
// 内部模快速幂：用于素性测试，返回 a^b%m；调用场景要求 m>1。
// 模数支持完整 u64 范围，乘法通过 mul 防溢出；时间 O(log(b+1))。
u64 pw(u64 a, u64 b, u64 m) {
    u64 r = 1;
    for (; b; b >>= 1, a = mul(a, a, m)) {
        if (b & 1) {
            r = mul(r, a, m);
        }
    }
```

```
    return r;
}
// 判断无符号 64 位整数 n 是否为素数，n<2 返回 false。
// 使用覆盖完整 u64 范围的确定性 Miller-Rabin 底数，固定轮数下耗时 O(log n) 次模乘。
bool is_prime(u64 n) {
    if (n < 2) return false;
    for (u64 p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n % p == 0) return n == p;
    }
    u64 d = n - 1;
    int s = 0;
    while (!(d & 1)) {
        d >>= 1;
        ++s;
    }
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
        u64 x = pw(a % n, d, n);
        if (x == 1 || x == n - 1) continue;
        bool ok = false;
        for (int r = 1; r < s; ++r) {
            x = mul(x, x, n);
            if (x == n - 1) {
                ok = true;
                break;
            }
        }
        if (!ok) return false;
    }
    return true;
}
```

调用与说明：is_prime(97)==true；is_prime(1)==false。

5.7 Pollard–Rho：质因数分解

用途与条件：分解 n>=1 的 u64 整数，输出所有质因子及其重数；依赖 Miller－Rabin。
复杂度：素性测试 O(log n) 次模乘乘以固定底数个数；Rho 随机算法无最坏时间保证，常见启发式估计约 O(nˆ(1/4))。
易错点：用 __uint128_t 模乘，平方加常数也在 128 位中完成。分解 1 得到空表，禁止传 0。结果需要排序才能统计质因子次数。
依赖：查看 5.6「Miller－Rabin：64 位素性测试」（第 52 页）；完整代码已展开依赖，可单独使用。

```
// Pollard-Rho 随机寻找并返回 n 的一个非平凡因子，要求 n 是合数且 n>=4。
// 偶数直接返回 2；奇数失败时重新尝试，无确定的最坏运行时间保证。
u64 rho(u64 n) {
    if (n % 2 == 0) return 2;
    static mt19937_64 rng(712367821);
    while (true) {
        u64 c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
        // 当前 Rho 轮的迭代函数：返回 (v²+c)%n，用 128 位中间量避免平方和加法溢出。
        auto f = [&](u64 v) { return (u64)((u128(mul(v, v, n)) + c) % n); };
        while (d == 1) {
            x = f(x);
            y = f(f(y));
            d = gcd(x > y ? x - y : y - x, n);
        }
        if (d != n) return d;
    }
}
// 把 n 的所有质因子（含重数）追加到 out，要求 n>=1；n=1 不添加任何元素。
// 不清空、不排序 out；先判断素数，再用 Rho 分治，运行时间具有随机性。
void factor(u64 n, vector<u64>& out) {
```

```
    assert(n >= 1);
    if (n == 1) return;
    if (is_prime(n)) {
        out.push_back(n);
        return;
    }
    u64 d = rho(n);
    factor(d, out);
    factor(n / d, out);
}
```

调用与说明：vector<u64> f; factor(60,f); sort(f.begin(),f.end()); // {2,2,3,5}

5.8 数论分块

用途与条件：枚举 floor(n/i) 相同的最大连续区间 [l,r]；本节区间特例为双闭。示例计算 Σ floor(n/i)。
复杂度：时间 O(sqrt(n))，空间 O(1)。
易错点：n 为非负 i64，答案用 i128。r=n/(n/l)；有多个整除项时取各项右端点最小值。

```
// 计算 Σ(i=1..n) floor(n/i)，按商相同的最大连续区间合并贡献。
// 要求 n>=0，n=0 返回 0；结果用 i128，时间 O(sqrt(n))，额外空间 O(1)。
i128 quotient_sum(i64 n) {
    assert(n >= 0);
    i128 ans = 0;
    for (i64 l = 1; l <= n;) {
        i64 q = n / l, r = n / q;
        ans += i128(r - l + 1) * q;
        if (r == n) break;
        l = r + 1;
    }
    return ans;
}
```

调用与说明：quotient_sum(5)==10。若有权函数前缀和 S，则该块贡献为 (S[r]-S[l-1])*q。

5.9 NTT：模 998244353 多项式卷积

用途与条件：原根为 3，输入系数先规范到 [0,MOD)。返回普通卷积，空输入返回空数组。 例外：多项式系数按次数从 0 编号，a[i] 是 x^i 的系数；FFT/NTT 蝶形缓冲区也从 0 开始。
复杂度：时间 O(L log L)，空间 O(L)，L 为不小于 n+m-1 的最小二次幂。
易错点：L 必须≤2^23。不能只改 MOD 就适配任意模数。整数卷积只有在可证明系数不跨模时才能直接恢复；否则需要多模数 CRT 等方案。
依赖：查看 5.1「快速幂」（第 49 页）；完整代码已展开依赖，可单独使用。

```
const int MOD = 998244353;
// 依赖 qpow：见 5.1「快速幂」（代码/05_数学/5.1 快速幂.cpp），复制本节时一并复制。
// 对系数数组 a 原地做 NTT；inv=false 为正变换，true 为逆变换并乘长度逆元。
// 要求长度是 1..2^23 内的二次幂，系数在 [0,998244353)；时间 O(n log n)。
// NTT（Number Theoretic Transform）：数论变换，在有限域中进行离散变换。
// inv（inverse）：是否执行逆变换，true 表示逆变换。
void ntt(vector<int>& a, bool inv) {
    int n = a.size();
    assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << 23));
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) {
            j ^= bit;
        }
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        int wlen = qpow(3, (MOD - 1) / len, MOD);
        if (inv) {
```

```
            wlen = qpow(wlen, MOD - 2, MOD);
        }
        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j], v = 1LL * a[i + j + len / 2] * w % MOD;
                a[i + j] = u + v;
                if (a[i + j] >= MOD) a[i + j] -= MOD;
                a[i + j + len / 2] = u - v;
                if (a[i + j + len / 2] < 0) a[i + j + len / 2] += MOD;
                w = 1LL * w * wlen % MOD;
            }
        }
    }
    if (inv) {
        int ni = qpow(n, MOD - 2, MOD);
        for (int& x : a) {
            x = 1LL * x * ni % MOD;
        }
    }
}
// 返回多项式 a、b 在模 998244353 下的普通卷积，系数按次数从 0 编号。
// 输入系数需已在 [0,MOD)，任一为空返回空数组；补齐后的变换长度 L<=2^23。
// 返回长度 |a|+|b|-1，不修改调用者的数组；时间 O(L log L)，空间 O(L)。
vector<int> convolution(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <<= 1;
    assert(n <= (1 << 23));
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < n; ++i) {
        a[i] = 1LL * a[i] * b[i] % MOD;
    }
    ntt(a, true);
    a.resize(need);
    return a;
}
```

调用与说明：convolution({1,2},{3,4}) 返回 {3,10,8}。

5.10 FFT：实系数卷积

用途与条件：使用复数蝶形计算实系数多项式卷积，适合误差可控的数值卷积。 例外：多项式系数按次数从 0 编号，不添加占位。

复杂度：时间 O(L log L)，空间 O(L)。

易错点：浮点误差与长度和系数规模有关。整数答案用 llround 前必须确认误差足够小；大整数精确卷积应使用 NTT 多模数方案。

```
using cd = complex<double>;
// 对复数数组 a 原地做 FFT；inv=false 为正变换，true 为逆变换并除以长度。
// 要求长度为正二次幂；浮点计算存在误差，时间 O(n log n)。
// FFT（Fast Fourier Transform）：快速傅里叶变换。
// cd（complex double）：复数类型别名；inv（inverse）：是否执行逆变换。
void fft(vector<cd>& a, bool inv) {
    int n = a.size();
    assert(n > 0 && (n & (n - 1)) == 0);
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) {
            j ^= bit;
```

```
        }
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    const double PI = acos(-1.0);
    for (int len = 2; len <= n; len <<= 1) {
        cd step = polar(1.0, 2 * PI / len * (inv ? -1 : 1));
        for (int i = 0; i < n; i += len) {
            cd w = 1;
            for (int j = 0; j < len / 2; ++j) {
                cd u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= step;
            }
        }
    }
    if (inv) {
        for (cd& x : a) x /= n;
    }
}
// 返回实系数多项式 a、b 的近似卷积，系数按次数从 0 编号；空输入返回空数组。
// 输出长度 |a|+|b|-1，不修改输入；变换长度 L 下时间 O(L log L)、空间 O(L)。
// 用于整数卷积时须先验证误差再舍入，不能直接当作精确大整数卷积。
vector<double> convolution(const vector<double>& a, const vector<double>& b) {
    if (a.empty() || b.empty()) return {};
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <<= 1;
    vector<cd> x(n), y(n);
    copy(a.begin(), a.end(), x.begin());
    copy(b.begin(), b.end(), y.begin());
    fft(x, false);
    fft(y, false);
    for (int i = 0; i < n; ++i) {
        x[i] *= y[i];
    }
    fft(x, true);
    vector<double> ans(need);
    for (int i = 0; i < need; ++i) ans[i] = x[i].real();
    return ans;
}
```

调用与说明：convolution({1.0,2.0},{3.0,4.0}) 近似返回 {3,10,8}。

5.11 高斯消元：实数线性方程组

用途与条件：输入增广矩阵 a[1..n][1..m+1]，第 m+1 列为常数项；第 0 行、第 0 列占位。返回 0 无解、1 唯一解、2 无穷多解；解向量 x[1..m]，自由变量设为 0。

复杂度：时间 O(nm·min(n,m))，空间 O(nm)。

易错点：采用 long double 与列主元，仍需根据数据尺度调整 eps。病态矩阵可能严重放大误差；模素数消元需替换为模逆运算。

```
// 对增广矩阵 a[1..n][1..m+1] 做实数高斯消元，末列为常数项，0 行/列占位。
// 返回 {状态,x}：0 无解、1 唯一解、2 无穷多解；有解时 x[1..m] 给出自由变量取 0 的一组解。
// 要求 n>0、各行尺寸一致、eps>0；矩阵按值传入，时间 O(n*m*min(n,m))。
pair<int, vector<long double>> gauss(vector<vector<long double>> a, long double eps = 1e-12L)
{
    int n = (int)a.size() - 1;
    assert(n > 0);
    int m = (int)a[1].size() - 2, row = 1;
    // where[j]：第 j 个未知数的主元所在行，0 表示没有主元，即自由变量。
    vector<int> where(m + 1, 0);
    for (int col = 1; col <= m && row <= n; ++col) {
```

```
        int sel = row;
        for (int i = row; i <= n; ++i) {
            if (fabsl(a[i][col]) > fabsl(a[sel][col])) {
                sel = i;
            }
        }
        if (fabsl(a[sel][col]) < eps) continue;
        swap(a[sel], a[row]);
        where[col] = row;
        long double d = a[row][col];
        for (int j = col; j <= m + 1; ++j) {
            a[row][j] /= d;
        }
        for (int i = 1; i <= n; ++i) {
            if (i != row) {
                long double f = a[i][col];
                for (int j = col; j <= m + 1; ++j) {
                    a[i][j] -= f * a[row][j];
                }
            }
        }
        ++row;
    }
    vector<long double> x(m + 1);
    for (int j = 1; j <= m; ++j) {
        if (where[j]) {
            x[j] = a[where[j]][m + 1];
        }
    }
    for (int i = 1; i <= n; ++i) {
        long double v = 0;
        for (int j = 1; j <= m; ++j) {
            v += a[i][j] * x[j];
        }
        if (fabsl(v - a[i][m + 1]) > eps) return {0, {}};
    }
    return {row == m + 1 ? 1 : 2, x};
}
```

调用与说明：gauss({{0,0,0,0},{0,1,1,3},{0,1,-1,1}}) 返回唯一解 x[1]=2,x[2]=1。

5.12 矩阵快速幂

用途与条件：正方形矩阵、正模数下快速求 A^e。适合固定维度线性递推、路径计数等问题。输入元素须在 [0,mod)。 矩阵分配 (d+1)×(d+1)，行列使用 1..d，第 0 行、第 0 列忽略。

复杂度：d 阶矩阵：O(d³ log e) 时间，O(d²) 空间。

易错点：乘法不交换，转移方向必须统一。若状态为列向量，则新状态=A*旧状态。本实现逐项使用 i128 累加后取模。

```
using Matrix = vector<vector<i64>>;
// 返回同阶方阵 a*b 对 mod 取模的结果；行列为 1..n，0 行/列占位。
// 要求 mod>0、元素在 [0,mod)，输入尺寸一致；不修改输入，时间 O(n³)，输出空间 O(n²)。
Matrix multiply(const Matrix& a, const Matrix& b, i64 mod) {
    int n = (int)a.size() - 1;
    Matrix c(n + 1, vector<i64>(n + 1));
    for (int i = 1; i <= n; ++i) {
        for (int k = 1; k <= n; ++k) {
            if (a[i][k]) {
                for (int j = 1; j <= n; ++j) {
                    c[i][j] = (i128(c[i][j]) + i128(a[i][k]) * b[k][j]) % mod;
                }
            }
        }
```

```
    }
    return c;
}
// 返回方阵 a 的非负整数 e 次幂模 mod；行列使用 1 下标，元素需在 [0,mod)。
// 要求 mod>0；e=0 返回模意义的单位矩阵，时间 O(n³log(e+1))，空间 O(n²)。
Matrix matrix_power(Matrix a, u64 e, i64 mod) {
    assert(mod > 0);
    int n = (int)a.size() - 1;
    Matrix r(n + 1, vector<i64>(n + 1));
    for (int i = 1; i <= n; ++i) r[i][i] = 1 % mod;
    for (; e; e >>= 1, a = multiply(a, a, mod)) {
        if (e & 1) {
            r = multiply(r, a, mod);
        }
    }
    return r;
}
```

调用与说明：Matrix a={{0,0,0},{0,1,1},{0,1,0}}; matrix_power(a,n,1000000007)[1][2] 为 Fibonacci(n)。

5.13 线性递推第 n 项：多项式约简

用途与条件：已知 f[t]=c[0]f[t-1]+...+c[k-1]f[t-k]，给前 k 项，求 f[n]。模数为正，输入系数和初值已规范。 例外：初值 init[0..k-1] 对应 f[0..k-1]，c[0] 对应 f[t-1]，多项式次数从 0 开始。

复杂度：时间 O(k² log n)，空间 O(k)。

易错点：特征关系为 x^k=c[0]x^(k-1)+...+c[k-1]，不要颠倒系数。这里不负责从数列推导递推式。

```
// 已知 f[t]=c[0]f[t-1]+...+c[k-1]f[t-k]，返回 f[n]%mod。
// init[0..k-1] 为前 k 项，c 长度相同且 k>0；要求 mod>0，初值和系数已在 [0,mod)。
// 用多项式约简求第 n 项，时间 O(k²log(n+1))，空间 O(k)。
i64 linear_nth(const vector<i64>& init, const vector<i64>& c, u64 n, i64 mod) {
    int k = c.size();
    assert(k > 0 && init.size() == c.size() && mod > 0);
    // 内部合并：将次数小于 k 的多项式 a、b 相乘，再按递推特征关系约简并取模。
    // 返回长度 k 的系数数组，不修改输入；时间 O(k²)。
    auto combine = [&](const vector<i64>& a, const vector<i64>& b) {
        vector<i64> t(2 * k - 1);
        for (int i = 0; i < k; ++i) {
            for (int j = 0; j < k; ++j) {
                t[i + j] = (i128(t[i + j]) + i128(a[i]) * b[j]) % mod;
            }
        }
        for (int i = 2 * k - 2; i >= k; --i) {
            for (int j = 1; j <= k; ++j) {
                t[i - j] = (i128(t[i - j]) + i128(t[i]) * c[j - 1]) % mod;
            }
        }
        t.resize(k);
        return t;
    };
    // r：快速幂累积的余式系数；x：当前二进制幂对应的余式，数组下标表示次数。
    vector<i64> r(k), x(k);
    r[0] = 1 % mod;
    if (k == 1) {
        x[0] = c[0];
    }
    else {
        x[1] = 1 % mod;
    }
    for (; n; n >>= 1, x = combine(x, x)) {
        if (n & 1) {
            r = combine(r, x);
```

```
        }
    }
    i64 ans = 0;
    for (int i = 0; i < k; ++i) {
        ans = (i128(ans) + i128(r[i]) * init[i]) % mod;
    }
    return ans;
}
```

调用与说明：linear_nth({0,1},{1,1},10,1000000007)==55。

5.14 博弈

5.14.1 Nim

用途与条件：正常 Nim，每次从一堆取走正数个石子，无法行动者输。a[1..n] 为堆大小。
复杂度：时间 O(n)，空间 O(1)。
易错点：忽略 a[0]；异或和为零则先手败。

```
// 判断正常 Nim 先手是否必胜：a[1..n] 为各堆石子数，忽略 a[0]。
// 所有堆大小异或非零返回 true，空局面返回 false；时间 O(n)。
bool nim_win(const vector<u64>& a) {
    u64 x = 0;
    for (int i = 1; i < (int)a.size(); ++i) {
        x ^= a[i];
    }
    return x != 0;
}
```

调用与说明：nim_win({0,1,2,3})==false。

5.14.2 反 Nim

用途与条件：每次从一堆取石子，取走最后一颗者输；a[1..n] 为堆大小。
复杂度：时间 O(n)，空间 O(1)。
易错点：全为大小 1 的非空堆时单独判断奇偶；当前空局面返回 true。

```
// 判断取最后一颗石子者输的反 Nim 先手胜负，堆数组为 a[1..n]。
// 存在大于 1 的堆时看异或和，否则看 1 石子堆是否为偶数；本实现空局面返回 true。
bool misere_nim_win(const vector<u64>& a) {
    u64 x = 0;
    int ones = 0;
    bool big = false;
    for (int i = 1; i < (int)a.size(); ++i) {
        u64 v = a[i];
        x ^= v;
        ones += v == 1;
        big |= v > 1;
    }
    return big ? x != 0 : ones % 2 == 0;
}
```

调用与说明：misere_nim_win({0,1,1})==true。

5.14.3 减法游戏 SG

用途与条件：正常玩法的减法游戏：每步可取走的数量来自 moves，计算状态 0..N 的 SG 值。
复杂度：时间 O(N*(|moves|+1))，空间 O(N+|moves|)。
易错点：moves 中每个数必须为正；SG 异或判定针对正常玩法的独立游戏之和。

```
// 计算减法游戏状态 0..N 的 SG 值；moves 是每步可取走数量的无占位列表。
// 要求 N>=0、每个动作 d>0；返回 sg[0]=0 的数组，时间 O(N*(|moves|+1))。
// SG（Sprague-Grundy）：斯普拉格-格兰迪函数。
// mex（minimum excluded value）：集合中未出现的最小非负整数。
vector<int> subtraction_sg(int N, const vector<int>& moves) {
    int K = moves.size();
    // sg[x]：剩余 x 时的 SG 值；seen[v]：SG 值 v 最近在哪个状态的后继中出现。
    vector<int> sg(N + 1), seen(K + 1, -1);
```

```
    for (int x = 1; x <= N; ++x) {
        for (int d : moves) {
            assert(d > 0);
            if (d <= x && sg[x - d] <= K) {
                seen[sg[x - d]] = x;
            }
        }
        int g = 0;
        while (g <= K && seen[g] == x) {
            ++g;
        }
        sg[x] = g;
    }
    return sg;
}
```

调用与说明：subtraction_sg(10,{1,2}) 中 sg[x]=x%3。

5.15 高精度整数：分块加减乘除

用途与条件：非负整数的精确运算。struct big_int 使用默认公开成员和动态数组；低位块从 arr[0] 开始，零表示为 {0}。只修改 width（1～9）即可联动 base=10^width、读入和输出。

复杂度：设 n、m 为操作数块数：比较和加减 O(max(n,m))，乘除普通 int 为 O(n)，高精度乘法 O(nm)，高精度除法 O(nm log base)；结果存储 O(n+m)。

易错点：减法要求 a>=b，除数必须非零；输入为非空十进制数字字符串。普通整数乘除参数按非负 32 位 int 设计，保持整数宏关闭。arr 公开便于理解，但不要手动破坏块范围和无高位零的约定。动态容量不代表朴素算法适合任意规模。

```
// 返回 10^k，用于编译期计算分块基数；要求 0<=k<=9。
constexpr int power10(int k) {
    int result = 1;
    while (k--) result *= 10;
    return result;
}

// 非负高精度整数：低位在前，从 0 下标开始，零统一表示为 {0}。
struct big_int {
    static constexpr int width = 3; // 只修改这里，每块 1～9 位
    static_assert(1 <= width && width <= 9, "width must be between 1 and 9");
    static constexpr int base = power10(width);
    vector<int> arr;

    // 将非负 int 转为高精度整数，默认构造为 0。
    big_int(int x = 0) {
        assert(x >= 0);
        do {
            arr.push_back(x % base);
            x /= base;
        } while (x);
    }

    // 将非空十进制数字字符串转为高精度整数，自动去除前导零。
    big_int(const string& s) {
        assert(!s.empty());
        for (int r = (int)s.size(); r > 0; r -= width) {
            int x = 0;
            for (int l = max(0, r - width); l < r; ++l) {
                assert(s[l] >= '0' && s[l] <= '9');
                x = x * 10 + s[l] - '0';
            }
            arr.push_back(x);
        }
        trim();
```

```
    }

    // 返回有效块数。
    int size() const { return (int)arr.size(); }

    // 删除最高端多余的零块，至少保留一块表示 0。
    void trim() {
        while (arr.size() > 1 && arr.back() == 0) arr.pop_back();
    }

    // 判断当前整数是否为 0。
    bool is_zero() const { return arr.size() == 1 && arr[0] == 0; }
};

// 输出 a，最高块正常输出，其余块补足 width 位；不自动换行。
void print(const big_int& a) {
    cout << a.arr.back();
    for (int i = a.size() - 2; i >= 0; --i) {
        string s = to_string(a.arr[i]);
        cout << string(big_int::width - (int)s.size(), '0') << s;
    }
}

// 比较 a、b：小于返回 -1，相等返回 0，大于返回 1。
int compare(const big_int& a, const big_int& b) {
    if (a.size() != b.size()) return a.size() < b.size() ? -1 : 1;
    for (int i = a.size() - 1; i >= 0; --i) {
        if (a.arr[i] != b.arr[i]) return a.arr[i] < b.arr[i] ? -1 : 1;
    }
    return 0;
}

// 返回 a+b，从低位向高位逐块相加并进位。
big_int jia(const big_int& a, const big_int& b) {
    big_int result;
    int n = max(a.size(), b.size());
    result.arr.assign(n + 1, 0);
    int carry = 0;
    for (int i = 0; i < n; ++i) {
        int x = i < a.size() ? a.arr[i] : 0;
        int y = i < b.size() ? b.arr[i] : 0;
        int cur = x + y + carry;
        result.arr[i] = cur % big_int::base;
        carry = cur / big_int::base;
    }
    result.arr[n] = carry;
    result.trim();
    return result;
}

// 返回 a-b，要求 a>=b；从低位向高位逐块相减并借位。
big_int jian(const big_int& a, const big_int& b) {
    assert(compare(a, b) >= 0);
    big_int result;
    result.arr.assign(a.size(), 0);
    int borrow = 0;
    for (int i = 0; i < a.size(); ++i) {
        int y = i < b.size() ? b.arr[i] : 0;
        int cur = a.arr[i] - y - borrow;
        borrow = cur < 0;
        if (borrow) cur += big_int::base;
```

```
        result.arr[i] = cur;
    }
    result.trim();
    return result;
}

// 返回高精度 a 与非负 int b 的乘积，中间运算使用 i64。
big_int chen(const big_int& a, int b) {
    assert(b >= 0);
    if (b == 0 || a.is_zero()) return big_int(0);
    big_int result;
    result.arr.assign(a.size(), 0);
    i64 carry = 0;
    for (int i = 0; i < a.size(); ++i) {
        i64 cur = (i64)a.arr[i] * b + carry;
        result.arr[i] = (int)(cur % big_int::base);
        carry = cur / big_int::base;
    }
    while (carry) {
        result.arr.push_back((int)(carry % big_int::base));
        carry /= big_int::base;
    }
    result.trim();
    return result;
}

// 返回两个高精度整数的乘积，采用逐块竖式乘法。
big_int chen(const big_int& a, const big_int& b) {
    if (a.is_zero() || b.is_zero()) return big_int(0);
    big_int result;
    result.arr.assign(a.size() + b.size(), 0);
    for (int i = 0; i < a.size(); ++i) {
        i64 carry = 0;
        for (int j = 0; j < b.size(); ++j) {
            i64 cur = result.arr[i + j] + (i64)a.arr[i] * b.arr[j] + carry;
            result.arr[i + j] = (int)(cur % big_int::base);
            carry = cur / big_int::base;
        }
        // 此位置尚未被之前的外层循环写入。
        result.arr[i + b.size()] = (int)carry;
    }
    result.trim();
    return result;
}

// 计算高精度 a 除以正 int b，返回 {商, 余数}，从高位向低位处理。
pair<big_int, int> divmod(const big_int& a, int b) {
    assert(b > 0);
    big_int quotient;
    quotient.arr.assign(a.size(), 0);
    i64 remainder = 0;
    for (int i = a.size() - 1; i >= 0; --i) {
        i64 cur = remainder * big_int::base + a.arr[i];
        quotient.arr[i] = (int)(cur / b);
        remainder = cur % b;
    }
    quotient.trim();
    return {quotient, (int)remainder};
}

// 计算高精度 a 除以非零高精度 b，返回 {商, 余数}。
```

```
// 采用竖式除法，每块商在 [0,base-1] 内二分。
pair<big_int, big_int> divmod(const big_int& a, const big_int& b) {
    assert(!b.is_zero());
    if (compare(a, b) < 0) return {big_int(0), a};
    if (b.size() == 1) {
        auto [q, r] = divmod(a, b.arr[0]);
        return {q, big_int(r)};
    }
    big_int quotient, remainder;
    quotient.arr.assign(a.size(), 0);
    for (int i = a.size() - 1; i >= 0; --i) {
        // remainder = remainder*base + a.arr[i]。
        if (remainder.is_zero()) {
            remainder.arr[0] = a.arr[i];
        }
        else {
            remainder.arr.insert(remainder.arr.begin(), a.arr[i]);
        }
        int l = 0, r = big_int::base;
        while (l + 1 < r) {
            int m = (l + r) / 2;
            big_int product = chen(b, m);
            if (compare(product, remainder) <= 0) {
                l = m;
            }
            else {
                r = m;
            }
        }
        quotient.arr[i] = l;
        if (l != 0) {
            remainder = jian(remainder, chen(b, l));
        }
    }
    quotient.trim();
    return {quotient, remainder};
}

// 返回高精度 a 除以正 int b 的整数商。
big_int chu(const big_int& a, int b) { return divmod(a, b).first; }

// 返回高精度 a 除以非零高精度 b 的整数商。
big_int chu(const big_int& a, const big_int& b) { return divmod(a, b).first; }
```

调用与说明：字符串读入：string s1,s2; cin>>s1>>s2; big_int a(s1),b(s2);。加法 print(jia(a,b));；乘法 print(chen(a,b));；商余 auto [q,r]=divmod(a,b); print(q);。print 不换行，按需 cout<<'\n';。需要负差时先 compare(a,b)，若 a<b 则输出负号并 print(jian(b,a))。阶乘使用 chen(result,i)。不使用 explicit 或输入输出运算符重载。

6. 动态规划与优化

6.1 背包

6.1.1 0/1 背包

用途与条件：每件物品最多选一次，容量倒序更新。 dp[0..W] 表示容量不超过上限的最大价值，允许不选。
复杂度：每件物品 O(W)。空间 O(W)。
易错点：物品重量必须为正；价值乘积及转移加法不能溢出；dp 初始全为 0，不表示恰好装满。

```
// 将一件重量 w、价值 v 的物品加入 0/1 背包，原地更新 dp[0..W]，容量倒序。
// 要求 w>0；默认 dp 为容量不超过上限的最优值且允许不选，加法不得溢出；时间 O(W)。
// DP（Dynamic Programming）：动态规划；dp[j] 表示容量不超过 j 时可获得的最大价值。
void zero_one(vector<i64>& dp, int w, i64 v) {
    assert(w > 0);
    for (int c = (int)dp.size() - 1; c >= w; --c) {
        dp[c] = max(dp[c], dp[c - w] + v);
    }
}
```

调用与说明：vector<i64> dp(W+1,0); zero_one(dp,w,v); 答案为 dp[W]。

6.1.2 完全背包

用途与条件：每种物品无限使用，容量正序更新。 dp[0..W] 表示容量不超过上限的最大价值，允许不选。
复杂度：每种物品 O(W)。空间 O(W)。
易错点：物品重量必须为正；价值乘积及转移加法不能溢出；dp 初始全为 0，不表示恰好装满。

```
// 将可无限使用的重量 w、价值 v 的物品加入完全背包，原地更新 dp，容量正序。
// 要求 w>0，dp 使用容量 0..W；当前初始化约定为不要求恰好装满，时间 O(W)。
// DP（Dynamic Programming）：动态规划；dp[j] 表示容量不超过 j 时可获得的最大价值。
void complete(vector<i64>& dp, int w, i64 v) {
    assert(w > 0);
    for (int c = w; c < (int)dp.size(); ++c) {
        dp[c] = max(dp[c], dp[c - w] + v);
    }
}
```

调用与说明：vector<i64> dp(W+1,0); complete(dp,w,v); 答案为 dp[W]。

6.1.3 多重背包：二进制拆分

用途与条件：每种物品最多 count 件，二进制拆分后调用 0/1 更新。 dp[0..W] 表示容量不超过上限的最大价值，允许不选。
复杂度：每种物品 O(W*log(count+1))。空间 O(W)。
易错点：物品重量必须为正；价值乘积及转移加法不能溢出；dp 初始全为 0，不表示恰好装满。
依赖：查看 6.1.1「0/1 背包」（第 64 页）；完整代码已展开依赖，可单独使用。

```
// 将最多 count 件重量 w、价值 v 的物品加入多重背包，二进制拆分后更新 dp。
// 要求 w>0、count>=0，价值乘积及转移和不溢出；时间 O(W*log(count+1))。
// DP（Dynamic Programming）：动态规划；dp[j] 表示容量不超过 j 时可获得的最大价值。
void bounded(vector<i64>& dp, int w, i64 v, int count) {
    assert(w > 0 && count >= 0);
    for (i64 k = 1; count > 0; k *= 2) {
        int take = (int)min<i64>(k, count);
        count -= take;
        i64 weight = 1LL * w * take;
        if (weight < (int)dp.size()) {
            zero_one(dp, (int)weight, v * take);
        }
    }
}
```

调用与说明：vector<i64> dp(W+1,0); bounded(dp,w,v,count); 答案为 dp[W]。

6.2 最长严格上升子序列及方案恢复

用途与条件：输入 a[1..n]，a[0] 忽略；返回最长严格上升子序列的原数组 1 下标。返回值是无占位的下标列表，内部 tail 也是无占位列表。

复杂度：时间 O(n log n)，空间 O(n)。

易错点：严格上升用 lower_bound；最长不下降改为 upper_bound。恢复的方案不保证字典序最小。

```
// 返回 a[1..n] 的一条最长严格上升子序列对应的原数组下标列表，忽略 a[0]。
// 结果列表不带占位，不保证字典序最小；时间 O(n log n)，空间 O(n)。
// LIS（Longest Increasing Subsequence）：最长严格上升子序列。
vector<int> lis_indices(const vector<i64>& a) {
    // tail[k]：长度为 k+1 的严格上升子序列可取得的最小末尾值。
    vector<i64> tail;
    // at[k]：tail[k] 对应的原数组下标；prev[i]：恢复方案时 i 的前驱下标，-1 表示无前驱。
    vector<int> at, prev(a.size(), -1);
    for (int i = 1; i < (int)a.size(); ++i) {
        int k = lower_bound(tail.begin(), tail.end(), a[i]) - tail.begin();
        if (k) prev[i] = at[k - 1];
        if (k == (int)tail.size()) {
            tail.push_back(a[i]);
            at.push_back(i);
        }
        else {
            tail[k] = a[i];
            at[k] = i;
        }
    }
    vector<int> ans;
    if (at.empty()) return ans;
    for (int i = at.back(); i != -1; i = prev[i]) ans.push_back(i);
    reverse(ans.begin(), ans.end());
    return ans;
}
```

调用与说明：lis_indices({0,3,1,2,2,4}) 返回长度为 3 的原数组下标列表，例如 {2,4,5}，对应值 {1,2,4}。

6.3 最长公共子序列：滚动数组

用途与条件：返回两个字符串的 LCS 长度。状态 dp[j] 表示当前前缀与 b 的前 j 个字符的 LCS。

复杂度：时间 O(nm)，空间 O(m)。

易错点：滚动数组不能直接恢复方案。若需要恢复，可保存完整状态或采用 Hirschberg 分治。字符相等转移来自上一行上一列。

```
// 返回字符串 a、b 的最长公共子序列长度，允许不连续；空串对应答案 0。
// 滚动数组不保存恢复方案所需的信息；时间 O(|a|*|b|)，空间 O(|b|)。
// LCS（Longest Common Subsequence）：最长公共子序列。
// DP（Dynamic Programming）：动态规划。
int lcs(const string& a, const string& b) {
    // dp[j]：当前已处理的 a 前缀与 b 的前 j 个字符的 LCS 长度，原地滚动更新。
    vector<int> dp(b.size() + 1);
    for (char x : a) {
        int diag = 0;
        for (int j = 1; j <= (int)b.size(); ++j) {
            int old = dp[j];
            if (x == b[j - 1]) {
                dp[j] = diag + 1;
            }
            else {
                dp[j] = max(dp[j], dp[j - 1]);
            }
            diag = old;
        }
    }
```

```
    return dp.back();
}
```

调用与说明：lcs("abcde","ace")==3。

6.4 数位 DP：相邻数字不同

用途与条件：示例统计 [0,N] 中十进制相邻位不相等的整数，包含 0。展示 tight、前导零与记忆化的处理方式。 例外：pos 为字符串位置，从 0 开始；prev=10 是未开始哨兵。

复杂度：十进制位数 D：O(D·11·10) 时间，O(D·11) 空间。

易错点：前导零不参与相邻位比较，prev=10 表示尚未开始。只缓存非 tight 状态。区间答案 count(R)-count(L-1)，要求 L≥0。

```
// 统计闭区间 [0,N] 中十进制相邻数字不同的整数个数，计入 0；N<0 返回 0。
// 前导零不参与比较；可用 count(R)-count(L-1) 求区间答案，时间与十进制位数成正比。
// DP（Dynamic Programming）：动态规划；此处按数位及上一位数字划分状态。
// DFS（Depth-First Search）：深度优先搜索，此处配合记忆化实现数位 DP。
u64 count_no_equal_adjacent(i64 N) {
    if (N < 0) return 0;
    string s = to_string(N);
    // memo[pos][prev]：不受上界约束时的补全数量；prev=10 表示尚未出现非前导零数字。
    u64 memo[20][11]{};
    bool vis[20][11]{};
    // 内部数位搜索：pos 为当前 0 下标位，prev 为上一位数字，10 表示尚未开始。
    // tight 表示前缀是否贴住上界；返回当前状态的合法补全数量，仅缓存不贴上界的状态。
    function<u64(int, int, bool)> dfs = [&](int pos, int prev, bool tight) -> u64 {
        if (pos == (int)s.size()) return 1;
        if (!tight && vis[pos][prev]) return memo[pos][prev];
        int lim = tight ? s[pos] - '0' : 9;
        u64 ans = 0;
        for (int d = 0; d <= lim; ++d) {
            int next = (prev == 10 && d == 0) ? 10 : d;
            if (prev != 10 && d == prev) continue;
            ans += dfs(pos + 1, next, tight && d == lim);
        }
        if (!tight) {
            vis[pos][prev] = true;
            memo[pos][prev] = ans;
        }
        return ans;
    };
    return dfs(0, 10, true);
}
```

调用与说明：count_no_equal_adjacent(20)==20，0..20 中只有 11 不满足。

6.5 SOS DP：子集和与超集和

用途与条件：对 n 位掩码计算 f[S]=Σ a[T]，其中 T 是 S 的子集。超集和使用相反方向。 例外：下标直接表示位掩码，0 是空集，不能改成从 1 开始。

复杂度：时间 O(n·2^n)，空间 O(2^n)。

易错点：n 必须足够小以容纳 2^n 状态，通常不超过 22 左右；实际以内存限制为准。容斥逆变换将 += 改成 -=。

```
// 返回每个掩码 S 的子集和 Σ(T⊆S) f[T]；按值处理，不修改调用者的数组。
// 要求 0<=n<31、f.size()==2^n 且内存足够，累加不溢出；时间 O(n*2^n)。
// SOS DP（Sum Over Subsets Dynamic Programming）：子集和动态规划。
// f[mask] 初始为单个集合的值，逐位转移后为 mask 的所有子集的值之和。
vector<i64> subset_sum(vector<i64> f, int n) {
    assert(0 <= n && n < 31 && f.size() == (1ULL << n));
    for (int b = 0; b < n; ++b) {
        for (int s = 0; s < (1 << n); ++s) {
            if (s >> b & 1) {
                f[s] += f[s ^ (1 << b)];
            }
```

```
        }
    }
    return f;
}
// 返回每个掩码 S 的超集和 Σ(T⊇S) f[T]，掩码 0 表示空集。
// 要求 0<=n<31、f.size()==2^n 且内存足够，累加不溢出；时间 O(n*2^n)。
// f[mask] 初始为单个集合的值，逐位转移后为 mask 的所有超集的值之和。
vector<i64> superset_sum(vector<i64> f, int n) {
    assert(0 <= n && n < 31 && f.size() == (1ULL << n));
    for (int b = 0; b < n; ++b) {
        for (int s = 0; s < (1 << n); ++s) {
            if (!(s >> b & 1)) {
                f[s] += f[s ^ (1 << b)];
            }
        }
    }
    return f;
}
```

调用与说明：subset_sum({1,2,3,4},2) 返回 {1,3,4,10}。枚举 S 的非空子集：for(int T=S;T;T=(T-1)&S)。

6.6 树形 DP：最大权独立集

用途与条件：在树上选若干互不相邻节点，使权值和最大。允许空集；输入连通无向树。使用迭代后序避免深递归。 邻接表 g 和权值 w 均为 n+1 个位置，使用 1..n；内部后序列表 ord 从 0 存放。

复杂度：时间 O(n)，空间 O(n)。

易错点：dp[u][0] 表示不选 u，dp[u][1] 表示选 u。父亲选中时子节点不能选；全为负权时答案为 0。

```
// 返回连通无向树的最大权独立集权值和，允许不选任何点；全负权时返回 0。
// g 和 w 均使用 1..n，0 号占位，要求权值和不溢出；迭代遍历耗时、空间均为 O(n)。
// DP（Dynamic Programming）：动态规划；此处在树上按子树合并状态。
i64 tree_independent_set(const vector<vector<int>>& g, const vector<i64>& w) {
    int n = (int)g.size() - 1;
    if (!n) return 0;
    vector<int> father(n + 1, -1), ord{1};
    father[1] = 1;
    for (int i = 0; i < n; ++i) {
        for (int v : g[ord[i]]) {
            if (v != father[ord[i]]) {
                father[v] = ord[i];
                ord.push_back(v);
            }
        }
    }
    // dp[u][0]：不选 u 的子树最大权值；dp[u][1]：选 u 的子树最大权值。
    vector<array<i64, 2>> dp(n + 1);
    for (int i = n - 1; i >= 0; --i) {
        int u = ord[i];
        dp[u][1] = w[u];
        for (int v : g[u]) {
            if (v != father[u]) {
                dp[u][0] += max(dp[v][0], dp[v][1]);
                dp[u][1] += dp[v][0];
            }
        }
    }
    return max(dp[1][0], dp[1][1]);
}
```

调用与说明：三点链权值 {2,3,2}，答案为 4，选择两端。

6.7 李超线段树：直线最小值与 DP 优化

用途与条件：离线收集所有将要查询的 x 坐标，支持任意斜率直线 y=kx+b 插入和单点最小值查询；适合 dp[i]=min_j(k[j]x[i]+b[j])。 构造参数 x 使用 1 下标，x[0] 忽略；内部坐标压缩缓冲区使用 0 下标。
复杂度：坐标数 M：每次插入与查询 O(log M)，空间 O(M)。
易错点：查询 x 必须在构造坐标中。直线值用 i128，要求实际值绝对值<2^126。无直线时返回 INF。本版本插入整条直线，不是线段。

```
// DP（Dynamic Programming）：动态规划；李超树可维护直线形式的转移最小值。
struct LiChao {
    struct Line {
        i64 k = 0, b = 0;
        // 计算当前直线在 x 处的 y=k*x+b，使用 i128 中间量；有效值绝对值须小于 2^126。
        i128 eval(i64 x) const { return i128(k) * x + b; }
    };
    const i128 INF = i128(1) << 126;
    // xs：排序去重后的所有查询横坐标，下标从 0 开始。
    vector<i64> xs;
    // t[p]：线段树节点保存的候选直线；has[p] 表示该节点是否已有直线。
    vector<Line> t;
    vector<char> has;
    // 收集全部将被查询的横坐标 x[1..m]，忽略 x[0]，排序去重后建立空李超树。
    // 至少需要一个有效坐标；初始化时间 O(m log m)，存储 O(m)。
    LiChao(vector<i64> x) : xs(move(x)) {
        assert(!xs.empty());
        xs.erase(xs.begin());
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty());
        t.resize(xs.size() * 4);
        has.resize(xs.size() * 4);
    }
    // 向李超树插入整条直线 v={k,b}，维护已收集坐标处的最小值；时间 O(log m)。
    void add(Line v) {
        add(1, 0, xs.size(), v);
    }
    // 内部递归插入：节点 p 覆盖压缩坐标下标区间 [l,r)，其中下标从 0 开始。
    // 保留在中点更优的直线，再递归处理另一条线可能更优的一侧。
    void add(int p, int l, int r, Line v) {
        if (!has[p]) {
            has[p] = 1;
            t[p] = v;
            return;
        }
        int m = (l + r) / 2;
        bool left = v.eval(xs[l]) < t[p].eval(xs[l]);
        bool mid = v.eval(xs[m]) < t[p].eval(xs[m]);
        if (mid) swap(t[p], v);
        if (r - l == 1) return;
        if (left != mid) {
            add(p * 2, l, m, v);
        }
        else {
            add(p * 2 + 1, m, r, v);
        }
    }
    // 返回所有已插入直线在横坐标 x 处的最小值；x 必须在构造时收集的坐标中。
    // 尚未插入直线时返回 INF，时间 O(log m)，不修改树。
    i128 query(i64 x) {
        int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(idx < (int)xs.size() && xs[idx] == x);
        int p = 1, l = 0, r = xs.size();
```

```
        i128 ans = INF;
        while (true) {
            if (has[p]) {
                ans = min(ans, t[p].eval(x));
            }
            if (r - l == 1) break;
            int m = (l + r) / 2;
            if (idx < m) {
                p *= 2;
                r = m;
            }
            else {
                p = p * 2 + 1;
                l = m;
            }
        }
        return ans;
    }
};
```

调用与说明：LiChao lc({0,0,1,2}); lc.add({2,3}); lc.add({-1,5}); lc.query(2)==3；首个 0 是占位，第二个 0 是真实坐标。

6.8 分治优化 DP

用途与条件：计算 cur[mid]=min_{0≤j<mid}(prev[j]+cost(j,mid))。适用于每行最优决策位置随 mid 单调不减的转移。cost 为 O(1)。 输入普通数组应采用 1 下标；DP 下标是前缀长度，prev[0] 表示空前缀。cost(j,i) 对应末组原数组闭区间 [j+1,i]。

复杂度：每层 O(n log n)，空间 O(n+log n)。

易错点：必须先证明决策单调性，常由四边形不等式导出。不能对任意 DP 使用。prev 不可达为 INF；有效加法结果须严格落在 (-INF,INF)。

```
const i64 INF = (1LL << 62);

// 计算一层 cur[i]=min(0<=j<i){prev[j]+cost(j,i)}，返回 cur[0..n]，cur[0]=INF。
// prev 的下标是前缀长度，cost(j,i) 对应原数组 [j+1,i]；要求最优决策位置单调不减。
// 不可达 prev 用 INF，有限转移结果须在 (-INF,INF)；cost 为 O(1) 时耗时 O(n log n)。
template <class Cost>
// DP（Dynamic Programming）：动态规划；利用最优决策下标的单调性分治计算一层。
vector<i64> divide_dp_layer(const vector<i64>& prev, Cost cost) {
    int n = prev.size() - 1;
    // prev：上一层 DP 值；cur：正在计算的当前层 DP 值，INF 表示尚无有效转移。
    vector<i64> cur(n + 1, INF);
    // 内部决策分治：求 cur[l..r]，最优决策只在 [optL,optR] 中搜索。
    // 先计算中点最优决策，再收窄左右搜索范围；只有决策单调性成立时才正确。
    function<void(int, int, int, int)> go = [&](int l, int r, int optL, int optR) {
        if (l > r) return;
        int mid = (l + r) / 2, best = optL;
        for (int j = optL; j <= min(mid - 1, optR); ++j) {
            if (prev[j] == INF) continue;
            i128 v = i128(prev[j]) + cost(j, mid);
            if (v < cur[mid]) {
                cur[mid] = (i64)v;
                best = j;
            }
        }
        go(l, mid - 1, optL, best);
        go(mid + 1, r, best, optR);
    };
    if (n) go(1, n, 0, n - 1);
    return cur;
}
```

调用与说明：分组 DP 可令 prev[0]=0，其他为 INF，逐层调用。cost(j,i) 为最后一组 [j+1,i] 的代价。

7. 计算几何

7.1 二维叉积

向量叉积公式：`cross(a,b) = a.x * b.y - a.y * b.x`。
三点叉积公式：`cross(a,b,c) = (b.x-a.x)*(c.y-a.y) - (b.y-a.y)*(c.x-a.x)`，即向量 AB 与 AC 的叉积。

cross(a,b,c)	几何含义
大于 0	c 在有向直线 a→b 左侧（要求 a != b）
小于 0	c 在有向直线 a→b 右侧（要求 a != b）
等于 0	三点共线，包含点重合的情况
绝对值	三角形 abc 面积的两倍

用途与条件：计算两个二维向量的叉积，或通过三点叉积判断左右方向、共线及三角形面积。采用 x 轴向右、y 轴向上的坐标系。
复杂度：时间与额外空间 O(1)。
易错点：坐标绝对值不超过 10^18；使用 i64 存坐标，i128 保存中间运算及结果。交换两个向量会使叉积变号；叉积为零只说明共线，判断点在线段上还要检查坐标范围。整数面积优先保存两倍面积，避免整除丢失半整数。

```
// 二维整数点，也用于表示二维向量。
struct Point {
    i64 x, y;
    // 按 (x,y) 字典序比较当前点和 b，供严格排序使用；不使用浮点容差，O(1)。
    bool operator<(Point b) const { return tie(x, y) < tie(b.x, b.y); }
    // 判断当前点与 b 的两个整数坐标是否完全相等，供点集去重使用，O(1)。
    bool operator==(Point b) const { return x == b.x && y == b.y; }
};
// 返回二维向量 a 与 b 的叉积，绝对值为平行四边形面积。
// 正值表示逆时针，负值表示顺时针，0 表示共线或存在零向量。
// 坐标绝对值不超过 10^18；先转为 i128 再乘法，时间 O(1)。
i128 cross(Point a, Point b) {
    return i128(a.x) * b.y - i128(a.y) * b.x;
}

// 返回向量 AB 与 AC 的叉积，即三角形 ABC 的两倍有向面积。
// a != b 时，正值表示 c 在 a→b 左侧，负值表示右侧；0 表示三点共线。
// 坐标绝对值不超过 10^18；先转为 i128 再减法，时间 O(1)。
i128 cross(Point a, Point b, Point c) {
    i128 x1 = i128(b.x) - a.x;
    i128 y1 = i128(b.y) - a.y;
    i128 x2 = i128(c.x) - a.x;
    i128 y2 = i128(c.y) - a.y;
    return x1 * y2 - y1 * x2;
}
```

调用与说明：cross(Point{3,0}, Point{0,2}) == 6；cross(Point{0,0}, Point{3,0}, Point{0,2}) == 6。后者表示第三个点在有向直线左侧，三角形面积为 3。

使用示例

```
Point a{0, 0}, b{3, 0}, c{0, 2};
i128 v = cross(a, b, c);  // 结果为 6，c 在 a→b 左侧
i128 area2 = v < 0 ? -v : v;  // 三角形面积的两倍
```

7.2 线段相交：整数精确判断

用途与条件：判断两条闭线段是否相交，包含端点、共线重叠和退化点。
复杂度：时间与额外空间 O(1)。
易错点：坐标绝对值要求不超过 10^18；包含端点接触、共线重叠和退化线段，使用整数精确比较。
依赖：查看 7.1「二维叉积」（第 70 页）；完整代码已展开依赖，可单独使用。

```
// 返回 x 的符号：负数为 -1，零为 0，正数为 1；时间 O(1)。
```

```
int sign(i128 x) { return (x > 0) - (x < 0); }
// 判断点 p 是否在线段 ab 上，包含端点，也支持 a=b 的退化线段。
// 使用精确叉积和坐标范围判断，坐标满足整数几何范围约定；时间 O(1)。
bool on_segment(Point p, Point a, Point b) {
    return cross(a, b, p) == 0 && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}


// 判断闭线段 ab 与 cd 是否有交点，包含端点接触、共线重叠和退化点。
// 使用整数精确判断，坐标绝对值不超过 10^18；时间 O(1)。
bool intersect(Point a, Point b, Point c, Point d) {
    int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
    int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
    if (!x && on_segment(c, a, b)) return true;
    if (!y && on_segment(d, a, b)) return true;
    if (!z && on_segment(a, c, d)) return true;
    if (!w && on_segment(b, c, d)) return true;
    return x * y < 0 && z * w < 0;
}
```

调用与说明：intersect({0,0},{2,0},{1,0},{3,0})==true。

7.3 凸包：单调链

用途与条件：求整数点集的逆时针凸包，去重且删除边上中间共线点；返回无占位顶点列表。

复杂度：时间 O(n log n)，空间 O(n)。

易错点：要求坐标绝对值≤10^18，以保证差值及叉积落在数值范围内。排序使用严格字典序，不加 eps。全共线凸包只保留两个端点。

依赖：查看 7.1「二维叉积」（第 70 页）；完整代码已展开依赖，可单独使用。

```
// 返回点集 p 的逆时针凸包顶点列表，不带占位、不重复首点，删除边上中间共线点。
// 输入可无序、有重复，不修改调用者的点集；全共线时只留端点，时间 O(n log n)。
vector<Point> convex_hull(vector<Point> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = p.size();
    if (n <= 1) return p;
    vector<Point> h(2 * n);
    int k = 0;
    for (auto v : p) {
        while (k >= 2 && cross(h[k - 2], h[k - 1], v) <= 0) {
            --k;
        }
        h[k++] = v;
    }
    for (int i = n - 2, t = k + 1; i >= 0; --i) {
        while (k >= t && cross(h[k - 2], h[k - 1], p[i]) <= 0) {
            --k;
        }
        h[k++] = p[i];
    }
    h.resize(k - 1);
    return h;
}
```

调用与说明：convex_hull({{0,0},{1,0},{1,1},{0,1}}) 返回四个逆时针顶点。

7.4 多边形面积

用途与条件：顶点按边界顺序给出；返回两倍有向面积，逆时针为正。

复杂度：时间 O(n)，额外空间 O(1)。

易错点：点列表无占位、不重复首点；坐标及叉积累加须在数值范围内。

依赖：查看 7.1「二维叉积」（第 70 页）；完整代码已展开依赖，可单独使用。

```
// Dependency: Point, cross, on_segment.
// 返回按边界顺序排列的多边形顶点 p 的两倍有向面积，逆时针为正、顺时针为负。
// p 是无占位循环列表，不需重复首点；要求叉积累加不溢出 i128，时间 O(n)。
i128 area2(const vector<Point>& p) {
    i128 a = 0;
    int n = p.size();
    for (int i = 0; i < n; ++i) {
        auto q = p[(i + 1) % n];
        a += i128(p[i].x) * q.y - i128(p[i].y) * q.x;
    }
    return a;
}
```

调用与说明：area2({{0,0},{4,0},{4,4},{0,4}})==32。

7.5 点与多边形的位置关系

用途与条件：判断点与简单多边形的位置；返回 0 外部、1 边界、2 内部。
复杂度：时间 O(n)，额外空间 O(1)。
易错点：点列表无占位、不重复首点；坐标及叉积累加须在数值范围内。
依赖：查看 7.1「二维叉积」（第 70 页）；完整代码已展开依赖，可单独使用。

```
// 返回 x 的符号：负数为 -1，零为 0，正数为 1；时间 O(1)。
int sign(i128 x) { return (x > 0) - (x < 0); }
// 判断点 p 是否在线段 ab 上，包含端点，也支持 a=b 的退化线段。
// 使用精确叉积和坐标范围判断，坐标满足整数几何范围约定；时间 O(1)。
bool on_segment(Point p, Point a, Point b) {
    return cross(a, b, p) == 0 && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}


// 判断点 q 与简单多边形 p 的位置关系：0 为外部、1 为边界、2 为内部。
// p 是按边界顺序排列的无占位循环列表，顺逆时针均可；时间 O(n)。
int inside(const vector<Point>& p, Point q) {
    // wn（winding number）：绕数；点不在边界时，非零表示在多边形内部。
    int wn = 0, n = p.size();
    for (int i = 0; i < n; ++i) {
        Point a = p[i], b = p[(i + 1) % n];
        if (on_segment(q, a, b)) return 1;
        if (a.y <= q.y && b.y > q.y && cross(a, b, q) > 0) {
            ++wn;
        }
        if (a.y > q.y && b.y <= q.y && cross(a, b, q) < 0) {
            --wn;
        }
    }
    return wn ? 2 : 0;
}
```

调用与说明：inside({{0,0},{4,0},{4,4},{0,4}},{2,2})==2。

7.6 旋转卡壳：凸包直径

用途与条件：求逆时针凸多边形最远点对距离平方；不足两个点返回 0。
复杂度：时间 O(n)，额外空间 O(1)。
易错点：输入必须逆时针且去除边上中间共线点，可用凸包模板预处理；坐标绝对值不超过 10^18。
依赖：查看 7.1「二维叉积」（第 70 页）；完整代码已展开依赖，可单独使用。

```
// 返回整数点 a、b 的欧氏距离平方，避免开方；坐标绝对值不超过 10^18，时间 O(1)。
i128 dist2(Point a, Point b) {
    i128 x = i128(a.x) - b.x, y = i128(a.y) - b.y;
    return x * x + y * y;
```

```
}
// 用旋转卡壳返回凸多边形最远两点距离的平方；不足两个点返回 0。
// p 必须逆时针排列且无边上中间共线点，可直接使用 convex_hull 的结果；时间 O(n)。
i128 diameter2(const vector<Point>& p) {
    int n = p.size();
    if (n < 2) return 0;
    if (n == 2) return dist2(p[0], p[1]);
    i128 ans = 0;
    int j = 1;
    for (int i = 0; i < n; ++i) {
        int ni = (i + 1) % n;
        while (cross(p[i], p[ni], p[(j + 1) % n]) > cross(p[i], p[ni], p[j])) {
            j = (j + 1) % n;
        }
        ans = max({ans, dist2(p[i], p[j]), dist2(p[ni], p[j])});
    }
    return ans;
}
```

调用与说明：diameter2({{0,0},{1,0},{1,1},{0,1}})==2。

7.7 直线交点

用途与条件：直线由点和非零方向向量表示；返回唯一交点，平行或重合返回 nullopt。
复杂度：时间和空间 O(1)。
易错点：方向向量必须非零；EPS 为绝对容差，应按坐标尺度调整。平行和重合都返回 nullopt。

```
// ld（long double）：扩展精度浮点类型；P（point）：二维点，也用于表示向量。
using ld = long double;
// EPS（epsilon）：浮点比较容差，需结合题目坐标尺度选择。
const ld EPS = 1e-12L;
struct P {
    ld x, y;
    // 返回当前二维向量与 b 的逐坐标和，不修改原向量；时间 O(1)。
    P operator+(P b) const { return {x + b.x, y + b.y}; }
    // 返回当前二维向量减去 b 的结果，可表示从 b 指向当前点的位移；时间 O(1)。
    P operator-(P b) const { return {x - b.x, y - b.y}; }
    // 返回当前向量乘标量 k 的结果，不修改原向量；时间 O(1)。
    P operator*(ld k) const { return {x * k, y * k}; }
};
// 返回二维向量 a、b 的叉积，正负表示旋转方向；浮点结果需结合 EPS 判断，O(1)。
ld cross(P a, P b) { return a.x * b.y - a.y * b.x; }
// 返回二维向量 a、b 的点积，可用于投影和长度平方计算；时间 O(1)。
ld dot(P a, P b) { return a.x * b.x + a.y * b.y; }


// 返回直线 p+t*v 与 q+s*w 的唯一交点；v、w 必须是非零方向向量。
// 在 EPS 意义下平行或重合时返回 nullopt，不区分这两种情况；时间 O(1)。
optional<P> line_intersection(P p, P v, P q, P w) {
    ld d = cross(v, w);
    if (fabsl(d) < EPS) return nullopt;
    return p + v * (cross(q - p, w) / d);
}
```

调用与说明：line_intersection({0,0},{1,1},{0,1},{1,-1}) 为 (0.5,0.5)。直线平行和重合均返回无单一交点。

7.8 圆与圆交点

用途与条件：两圆半径非负；返回交点数及交点列表，-1 表示重合且交点无穷。
复杂度：时间和空间 O(1)。
易错点：半径必须非负；EPS 为绝对容差，应按坐标尺度调整。重合的零半径圆只有一个交点。

```
// ld（long double）：扩展精度浮点类型；P（point）：二维点，也用于表示向量。
using ld = long double;
// EPS（epsilon）：浮点比较容差，需结合题目坐标尺度选择。
const ld EPS = 1e-12L;
struct P {
    ld x, y;
    // 返回当前二维向量与 b 的逐坐标和，不修改原向量；时间 O(1)。
    P operator+(P b) const { return {x + b.x, y + b.y}; }
    // 返回当前二维向量减去 b 的结果，可表示从 b 指向当前点的位移；时间 O(1)。
    P operator-(P b) const { return {x - b.x, y - b.y}; }
    // 返回当前向量乘标量 k 的结果，不修改原向量；时间 O(1)。
    P operator*(ld k) const { return {x * k, y * k}; }
};
// 返回二维向量 a、b 的叉积，正负表示旋转方向；浮点结果需结合 EPS 判断，O(1)。
ld cross(P a, P b) { return a.x * b.y - a.y * b.x; }
// 返回二维向量 a、b 的点积，可用于投影和长度平方计算；时间 O(1)。
ld dot(P a, P b) { return a.x * b.x + a.y * b.y; }


// 求圆心 a、半径 r 与圆心 b、半径 s 两圆的交点，要求半径非负。
// 返回 {数量,交点列表}：0/1/2 为有限交点数，-1 表示重合且有无穷交点。
// 按 EPS 判定，重合的零半径圆按一个交点处理；时间和空间 O(1)。
pair<int, vector<P>> circle_intersection(P a, ld r, P b, ld s) {
    assert(r >= 0 && s >= 0);
    P v = b - a;
    ld d = sqrtl(dot(v, v));
    if (d < EPS) {
        if (fabsl(r - s) > EPS) return {0, {}};
        if (r < EPS) return {1, {a}};
        return {-1, {}};
    }
    if (d > r + s + EPS || d < fabsl(r - s) - EPS) return {0, {}};
    ld x = (r * r - s * s + d * d) / (2 * d), h2 = r * r - x * x;
    if (h2 < -EPS) return {0, {}};
    P base = a + v * (x / d);
    if (h2 <= EPS) return {1, {base}};
    P off = P{-v.y, v.x} * (sqrtl(max((ld)0, h2)) / d);
    return {2, {base + off, base - off}};
}
```

调用与说明：circle_intersection({0,0},1,{2,0},1).first==1。

7.9 有界半平面交：逐边裁剪

用途与条件：用凸多边形逐个裁剪半平面，保留有向直线 a→b 左侧（含边界）。依赖浮点几何 P/cross/EPS。例外：poly 是无占位的循环顶点列表，位置从 0 开始，与本章其他几何函数一致。
复杂度：单次 O(当前顶点数)，m 个半平面总体最坏 O(m²)；空间 O(m)。
易错点：这是适合中小规模的裁剪版，不是 O(m log m) 排序双端队列版。必须给出题目保证覆盖解集的初始包围框；任意猜大框不能可靠判断无界。

```
// ld（long double）：扩展精度浮点类型；P（point）：二维点，也用于表示向量。
using ld = long double;
// EPS（epsilon）：浮点比较容差，需结合题目坐标尺度选择。
const ld EPS = 1e-12L;
struct P {
    ld x, y;
    // 返回当前二维向量与 b 的逐坐标和，不修改原向量；时间 O(1)。
    P operator+(P b) const { return {x + b.x, y + b.y}; }
    // 返回当前二维向量减去 b 的结果，可表示从 b 指向当前点的位移；时间 O(1)。
    P operator-(P b) const { return {x - b.x, y - b.y}; }
    // 返回当前向量乘标量 k 的结果，不修改原向量；时间 O(1)。
    P operator*(ld k) const { return {x * k, y * k}; }
```

```
};
// 返回二维向量 a、b 的叉积，正负表示旋转方向；浮点结果需结合 EPS 判断，O(1)。
ld cross(P a, P b) { return a.x * b.y - a.y * b.x; }
// 返回二维向量 a、b 的点积，可用于投影和长度平方计算；时间 O(1)。
ld dot(P a, P b) { return a.x * b.x + a.y * b.y; }


// Dependency: P, cross, EPS.
// 用有向直线 a→b 裁剪凸多边形 poly，保留其左侧及边界，返回新的循环顶点列表。
// 要求 a!=b，顶点按边界顺序排列且不带占位；空结果表示在容差意义下无交集。
// 不修改输入，每次耗时 O(顶点数)；连续裁剪需从保证覆盖解集的包围框开始。
vector<P> clip_halfplane(const vector<P>& poly, P a, P b) {
    vector<P> out;
    int n = poly.size();
    P v = b - a;
    assert(dot(v, v) > 0);
    for (int i = 0; i < n; ++i) {
        P p = poly[i], q = poly[(i + 1) % n];
        // dp、dq：点 p、q 相对裁剪直线的有向叉积值，此处 dp 不是动态规划。
        ld dp = cross(v, p - a), dq = cross(v, q - a);
        // ip、iq：对应端点是否位于保留半平面内，包含容差范围内的边界。
        bool ip = dp >= -EPS, iq = dq >= -EPS;
        if (ip) out.push_back(p);
        if (ip != iq) {
            // Classification uses EPS; clamp near-boundary interpolation.
            ld t = dp / (dp - dq);
            t = max((ld)0, min((ld)1, t));
            out.push_back(p + (q - p) * t);
        }
    }
    return out;
}
```

调用与说明：初始矩形按逆时针给出。每条限制调用 poly=clip_halfplane(poly,a,b)；空数组表示交集为空（在浮点容差意义下）。

8. 比赛速查与检查清单

8.1 算法选型

需求	首选与条件
静态区间和 / 最值	前缀和 / ST 表；ST 要求幂等合并
单点加、区间和	树状数组
区间加、区间和	懒标记线段树
静态区间第 k 小	离散化 + 主席树
多个离线区间统计	可逆增删时考虑莫队
树上路径 / 子树操作	树链剖分 + 线段树
每次只关心少量树上节点	虚树
无权 / 0-1 权 / 非负权最短路	BFS / 0-1 BFS / Dijkstra
负权最短路	Bellman-Ford；先确认负环语义
有向图环依赖	SCC 缩点成 DAG
二选一约束	能转成二元析取子句时用 2-SAT
无向图连通脆弱性	割点与桥；重边需按边 id 区分
单模式 / 多模式匹配	KMP / AC 自动机
回文子串	Manacher；不同回文串统计需回文树等其他方法
子串排序 / 子串集合	后缀数组 / 后缀自动机
模素数多项式乘法	模数与长度满足要求时用 NTT
多条直线最小值	李超线段树；先收集查询坐标

8.2 复杂度与内存估算

以下仅是数量级估算，不是“必过”保证。运行时间取决于时限、机器、语言、常数与访存方式。

n	常见可尝试的复杂度
20	2^n、n • 2^n；阶乘通常已不可行
500	n^3 可能偏紧，需结合时限
5000	n^2，注意内存与常数
200000	n log n、n $\log^2$ n；后者需估常数
1000000	n 或低常数 n log n

- `int[n]` 约 4n 字节，`i64[n]` 约 8n 字节，另加容器本身开销。
- `vector<vector<T>>` 每行有额外容器开销；大量小对象可考虑平铺。
- 主席树预估节点数约为插入次数 × (树高+1)，以实际 `sizeof(Node)` 估内存。
- 长链上的递归 DFS、Tarjan、Dinic 可能耗尽调用栈；深度较大时优先改迭代，不依赖本机恰好能跑过。
- `__int128` 不支持直接流输入输出。输出辅助函数见下节。

8.3 128 位整数输出

```
string to_string_i128(i128 x) {
    if (x == 0) return "0";
    bool neg = x < 0;
    __uint128_t y = neg ? __uint128_t(-(x + 1)) + 1 : __uint128_t(x);
    string s;
    while (y) { s.push_back(char('0' + y % 10)); y /= 10; }
    if (neg) s.push_back('-');
    reverse(s.begin(), s.end());
    return s;
}
```

8.4 常用结论与适用条件

- 树的边数为 n-1，且任意两点间恰有一条简单路径。只检查边数不足以保证输入是一棵树，还需连通。
- 非负边权树的直径可用两次遍历求最远点；含负边时不能无条件使用该结论，应做树形 DP。
- 无向图最小边覆盖大小 = n − 最大匹配大小，前提为没有孤立点。二分图最小点覆盖大小 = 最大匹配大小。
- DAG 最小不相交路径覆盖（按顶点、边必须为原图边）可拆点为二分图，答案为 n − 最大匹配。
- 最大流 = 最小割容量。网络中使用“无穷容量”时，应取严格大于任意可行有限割的上界并确保求和不溢出。
- 欧拉回路需满足度数条件及所有有边顶点的相应连通条件，单看度数不充分。
- 普通 Nim：堆大小异或和为 0 时先手败。独立正常玩法 SG 游戏可异或组合，反常玩法不能直接照搬。
- 欧拉定理：gcd(a,m)=1 时 $a^{\varphi(m)} \equiv 1 \pmod m$。不互质时指数不能直接对 $\varphi(m)$ 取模。
- 简单格点多边形满足 Pick 定理；自交多边形不直接适用。

8.5 交题前检查

1. 题意：是否多测？是否允许空集、重复元素、自环、重边、负权？求的是次数还是不同对象数量？
2. 边界：n=0/1、全相等、全负、链、星、完全图、断开图、无解、恰好卡端点。
3. 下标：普通数组和顶点直接使用题目的 1 编号，数组区间 [l,r] 直接传入；字符串、掩码、次数等例外按条目约定。
4. 数值：乘法在相乘前提升类型；哨兵不参与普通算术；取模先处理负数；浮点按量级选容差。
5. 状态：多测重建容器；递归栈容量；残量图、自动机 build、持久化版本的生命周期是否正确。
6. 复杂度：总输入规模，不仅单测规模；内存估算；哈希和随机算法的最坏情况风险。
7. 输出：大小写、空格、换行、小数位、无解格式、是否仍有调试输出。

8.6 对拍方法

先写小规模暴力算法作为独立答案来源，再随机生成合法输入，对比最优值或结果集合。

- 固定随机种子，失败时输出种子和完整输入，保证可复现。
- 非唯一解只比较合法性与目标值，不要求输出方案完全一样。
- 单测之外加入极端结构，例如长链、重复值、空区间（仅适用于支持空区间的接口）。
- 浮点结果使用与题目尺度相符的绝对和相对误差判定。
- 本项目的检查见 `测试/verify.cpp` 与 `验证报告.md`；它们不代替具体题目的建模验证。

8.7 本版范围

本版覆盖常用及一部分进阶竞赛模板，重在可以直接维护、理解与验证。没有把所有专题都压进同一个版本：LCT、Segment Tree Beats、一般图带花匹配、后缀树、多项式全家桶、三维几何等尚未收录。遇到相应题目时，应补充专门模板及对应验证，而不是把现有模板强行改用。